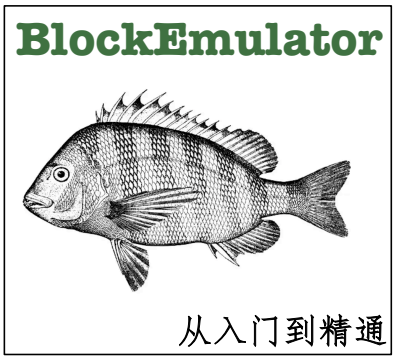
【从入门到精通】

开源区块链实验平台 BlockEmulator

使用指南



黄华威研究组

中山大学软件工程学院

2026 年 7 月 19 日

前 言

这本小册子是 BlockEmulator 的官方中文版用户使用指南，基于 BlockEmulator 1.0 使用指南 (2024 December 版本) 扩展编写而成；旨在为 BlockEmulator 的用户，尤其是新手，提供一个易上手、足够详细的使用指南。

该使用指南的主要贡献者描述如下：

- 黄华威：规划统筹、写作技术部分之外的内容、把控写作质量、营造团队文化、管理该文档版本，等等。
- 技术部分撰写主要贡献者为黄华威研究组[1]的叶光、殷昭伉、熊剑波同学。他们也是 BlockEmulator v2.0 版本（BlockEmulator-X，详见章节 7、章节 8与章节 9）的主要贡献者。
- 陈钦德是 Broker2Earn 算法分支（章节 6）的主要贡献者。
- 熊剑波是 BlockEmualtor-Web（章节 9）的主要贡献者。

BlockEmulator 开源项目的介绍可以从官网主页上[2]查看。事实上，这个项目所有的贡献者包括：林建入老师、叶光、殷昭伉、熊剑波、陈钦德、郑简、彭肖文、林岳、詹建洲、张深扬、黄振毅、李灿林、许淼泳、谢葆洲、胡飞鸿、周阳、吴均豪、罗肖飞博士、袁剑文博士、李涛涛博士、杨青林博士。指导老师是黄华威。

其中，林建入老师是全栈工程师，也是黄华威研究团队的 CTO，他有丰富的工程开发经验，他也曾是中山大学软件工程学院的区块链实验课授课老师之一。

彭肖文、林岳、詹建洲、张深扬、黄振毅、李灿林、许淼泳是黄华威研究组已经毕业的硕士生，目前均在互联网大厂工作。他们对 BlockEmulator 的早期版本做出了重要的贡献。

郑简是黄华威研究组在读 Ph.D 学生；陈钦德于 2026 年 6 月博士毕业，谢葆洲、胡飞鸿、熊剑波、周阳是在读的硕士生；吴均豪从本科实习生开始加入本项目直到他现在留在组内继续攻读博士；袁剑文博士是研究组在站博士后研究员；罗

[1]黄华威研究组主页：`http://xintelligence.pro/`

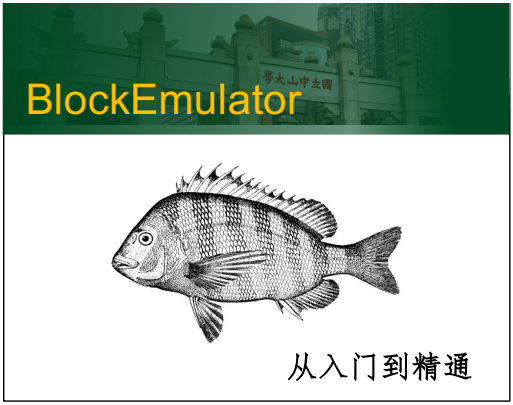
[2]BlockEmulator 主页：`https://www.blockemulator.com`

肖飞博士与李涛涛博士曾是研究组的博士后研究员；杨青林曾是访问研究员。他们的研究课题几乎都会使用 BlockEmulator 当做实验工具，来验证他们提出的最新区块链协议、算法与机制。

在此，对以上所有对 BlockEmulator 做出过独特贡献的每一位 team member 致以真挚感谢！

黄华威

2026 年 7 月 19 日



HuangLab appreciates the great efforts made by every member, current and graduated, who have made a unique contribution to the success of BlockEmulator.

目 录

1 背景介绍 *1*

1.1 BlockEmulator 项目简介 1

1.2 项目开源初衷 2

1.3 BlockEmulator 的亮点 3

1.4 官方技术论文 3

1.5 研究组产出的相关论文 4

1.6 BlockEmulator 视频教程 6

1.7 BlockEmulator 第一年发展历程 . 7

1.8 BlockEmulator 社区持续发展 . . 9

2 设计原理 *12*

2.1 设计思路 12

2.2 架构与组件 14

2.3 内置共识机制 28

3 *BlockEmulator* 技术论文 *46*

3.1 技术论文撰写背景 46

3.2 技术论文简介 47

3.3 BlockEmulaotr 技术论文的引用 . 48

4 开始使用它做实验 *55*

4.1 启动方式 55

4.2 事前准备 56
4.3 启动 BlockEmulator 58
4.4 运行结束后的数据收集 64
4.5 设计用户自定义的协议 68
4.6 手动编译源码运行 BlockEmulator 77
5 BlockEmulator v1.0 的其他分支 79
5.1 Fine-tune-lock 分支 79
5.2 tMPT 分支 83
6 Broker2Earn 分支 91
6.1 Broker2Earn 工作简介 91
6.2 Broker2Earn 协议设计说明 . . . 93
6.3 Broker2Earn 算法的解释 96
6.4 调用 Broker2Earn 算法做实验 . . 97
6.5 Broker2Earn 实验案例展示 . . . 104
7 BlockEmulator-X: 更好用的版本 109
7.1 BlockEmulator-X 诞生的背景 . . 109
7.2 BlockEmulator-X 的定位与价值 . 110
7.3 启动使用 BlockEmulator-X . . . 111
7.4 BE-X 的系统架构 113
7.5 两类节点的工作流程 114
7.6 BlockEmulator v2.0 (BE-X) 的更新内容 117
7.7 存储机制 119
7.8 数据结构 124
7.9 网络与通信 129

7.10 节点执行 134
7.11 日志功能 135
8 用 BE-X 定制化测试智能合约 137
8.1 测试智能合约的使用场景 137
8.2 使用 BE-X 测试智能合约的优势 139
8.3 测试步骤概述 139
8.4 Step1：编写并编译智能合约 . . . 141
8.5 Steps 2&3：生成 “Trace 文件” 144
8.6 Finally：开始运行实验并查看实验结果 151
8.7 可能会遇到的技术问题与解决方案 153
9 用 BlockEmulator-Web 启动 BE-X 154
9.1 BlockEmulator-Web 是什么? . . 154
9.2 使用 BE-Web 前的准备工作 . . . 156
9.3 启动 BE-Web 中的所有服务 . . . 158
9.4 BE-Web 的界面布局与功能区介绍 162
9.5 运行实验：从开始到结束 168
9.6 查看实验结果 170
9.7 其他补充信息 172
9.8 使用过程中可能遇见的问题 . . . 173
10 使用 BE 可能会遇到的技术问题 176
10.1 用户二次开发中遇到的 Bug 及解决方案 176
10.2 PBFT 共识间隔参数的异常设置 177
10.3 CLPACommitteeModule 出现死循环 182

10.4 AddAccount 函数抛出空指针异常 186
11 历史版本更新 192
12 用户社区的建设与维护 202
12.1 微信群社区的问题解答 202
12.2 GitHub 项目 Issues 页面的问答 . 205
12.3 用户社区其他常见问题的汇总 . . 210
13 未来规划 224
13.1 BlockEmulator 的后续版本 . . . 224
13.2 BlockEmulator 社区 224

第一章　背景介绍

1.1　BlockEmulator 项目简介

BlockEmulator 是由中山大学黄华威研究组（HuangLab[1] ）开源的可支持多种**共识协议**的区块链实验平台，可用于对用户开发的自定义区块链协议、算法与机制进行性能评估，其特色是支持区块链**跨分片机制**。

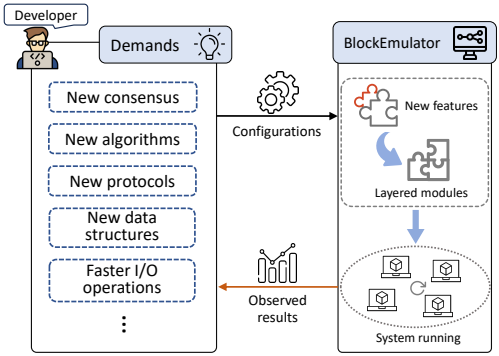


图 1.1: The design principle of BlockEmulator.

如图 1.1 所示，此实验平台开源的主要目的是为了帮助用户（研究者、学生）快速验证他们提出的新型区块链共识协议、算法、机制、数据结构等等其他设计。BlockEmulator 被设计为轻量化区块链系统架构的实验平台。它简化了工业级区块链系统的实现流程，这是因为 BlockEmulator 只关注实现区块链的核心功能，比如交易池、区块打包、区块共识、交易上

[1] http://www.xintelligence.pro

链等核心环节，忽略了那些与区块链体系架构不相关的细节。

BlockEmulator 支持常见的主流共识协议，例如拜占庭容错协议 PBFT (Practical Byzantine Fault Tolerance)。特别地，BlockEmulator 对主流的“区块链分片机制”进行了系统底层级别的设计与实现。其中，BlockEmulator 实现的“跨分片交易”处理机制包含以下两个具有代表性的分片协议：Monoxide (NSDI'2019) 中提出的“Relay 交易机制”[16]，以及 BrokerChain (INFOCOM'2022) 中的“broker 机制”[7]。

1.2 项目开源初衷

该实验平台主要面向区块链研究人员，当他们需要对提出的新型区块链共识、新算法、新机制进行验证时，可以帮助他们快速搭建一个轻量化的区块链底层协议验证与性能测试平台，并对实验数据进行自动记录，方便实验人员绘制实验图。

图 1.2: The slogan of BlockEmulator.

关于开源初衷，我们已经写到了 BlockEmulator 的官网主页，即“Making Blockchain Experiments Easy（让区块链研究更简单）”，如图1.2所示。

1.3 BlockEmulator 的亮点

1. **轻量化**：代码轻量化。
2. **快速搭建**：快速搭建区块链实验环境，可远程部署到云端运行。
3. **可定制化**：基于 Golang 语言实现，支持用户定制化二次开发。
4. **易于实验**：支持主流区块链（如以太坊）的历史交易数据的回放，可以自动输出、保存各项实验指标（如交易吞吐量、交易确认时延、交易池拥塞程度等等）以及系统运行的日志。

1.4 官方技术论文

我们从 2023 年开始撰写了一篇严谨的技术论文，题目为 “BlockEmulator: An Emulator Enabling to Test Blockchain Sharding Protocols” [8]。请前往 chapter 3.1 查看该技术文档的详细介绍。最终，这篇论文于 2025 年发表在 IEEE TSC 期刊。如果您使用了 BlockEmulator 做了实验或者基于它二次开发，请在您的论文中引用它：

```
@article{huang2023blockemulator,
  title     = {{BlockEmulator: An Emulator Enabling to Test
    Blockchain Sharding Protocols}},
  author    = {Huang, Huawei and Ye, Guang and Yang, Qinglin
    and Chen, Qinde and  Yin, Zhaokang and Luo, Xiaofei and
    Lin, Jianru and Zheng, Jian and Li, Taotao and Zheng,
    Zibin},
  journal   = {{IEEE Transactions on Services Computing
    (TSC)}},
  year      = {2025},
  volume    = {18},
  number    = {2},
  pages     = {690--703}
}
```

1.5 研究组产出的相关论文

从 2021 年开始，HuangLab 使用自己开发的 BlockEmulator 各个时期的版本陆续产出了一些区块链论文。随着这些论文陆续发表，BlockEmulator 的各项功能也逐渐完善。如下几篇论文欢迎查阅：

- **BrokerChain 分片协议原始版本：** “BrokerChain: A Cross-Shard Blockchain Protocol for Account/Balance-based State Sharding” [7] **(INFOCOM 2022)** 【PDF[1]】【公众号文章[2]】
- **BrokerChain 分片协议完整版本：** “BrokerChain: A Blockchain Sharding Protocol by Exploiting Broker Accounts” [9] **(ToN 2025)** 【PDF[3]】
- **ShardCutter 分片间账户图划分、负载均衡方案：** “ShardCutter: ShardCutter: A Blockchain Sharding Protocol Achieving Transaction Workload Balance Across State Shards” [11] **(ToN 2026)** 【PDF[4]】
- **Broker2Earn 激励机制协议：** “Broker2Earn: Towards Maximizing Broker Revenue and System Liquidity for Sharded Blockchains” [1] **(INFOCOM 2024)** 【PDF[5]】【公众号文章[6]】
- **LiqudityPool 小散户质押参与金额敏感 DeFi 协议的收益保障方案：** “LiquidityPool: Game-Theoretic Analysis of Stakeholder Revenue in Ranking-Dependent DeFi” [2] **(WWW 2026)** 【PDF[7]】

[1] https://www.researchgate.net/publication/356789473_BrokerChain_A_Cross-Shard_Blockchain_Protocol_for_AccountBalance-based_State_Sharding

[2] https://mp.weixin.qq.com/s/5MelID6kVMQeM1LAET-37w, 《基于 “做市商账户” 的区块链跨分片协议—BrokerChain》

[3] https://www.researchgate.net/publication/390218703_BrokerChain_A_Blockchain_Sharding_Protocol_by_Exploiting_Broker_Accounts

[4] https://www.researchgate.net/publication/400699615_ShardCutter_A_Blockchain_Sharding_Protocol_achieving_Transaction_Workload_Balance_across_State_Shardss

[5] https://www.researchgate.net/publication/379213048_Broker2Earn_Towards_Maximizing_Broker_Revenue_and_System_Liquidity_for_Sharded_Blockchains

[6] https://mp.weixin.qq.com/s/WxuNjGmqil5GmYYSFJ0qVw

[7] https://www.researchgate.net/publication/400068018_LiquidityPool_Game-Theoretic_Analysis_of_Stakeholder_Revenue_in_Ranking-Dependent_DeFi

- **Fine-Tuned Lock (FTL) 账户跨片迁移方案:**"Account Migration across Blockchain Shards using Fine-tuned Lock Mechanism" [5] **(INFOCOM 2024)**【PDF[1]】【公众号文章[2]】【知乎文章[3]】
- **Justitia 补偿分片间手续费促使交易上链公平性方案:** "Justitia: An Incentive Mechanism towards the Fairness of Cross-shard Transactions" [22] **(INFOCOM 2025)**【PDF[4]】【公众号文章 [5]】
- **MVCom 扩展版本:** "Scheduling Most Valuable Committees for the Sharded Blockchain" [6] (**ToN 2023**)【PDF[6]】【公众号文章[7]】
- **MVCom 会议版本:** "MVCom: Scheduling Most Valuable Committees for the Large-Scale Sharded Blockchain" [4] **(ICDCS 2021)** 【PDF[8]】【公众号文章[9]】
- **CLPA 账户图划分算法:**"Achieving Scalability and Load Balance across Blockchain Shards for State Sharding" [13] **(SRDS 2022)**【PDF[10]】【公众号文章[11]】
- **tMPT 共识节点分片间重分配方案:** "tMPT: Reconfiguration across Blockchain Shards via Trimmed Merkle Patricia Trie" [10] **(IWQoS 2023)**【PDF[12]】【公众号文章[13]】

---

[1] https://www.researchgate.net/publication/379210418_Account_Migration_across_Blockchain_Shards_using_Fine-tuned_Lock_Mechanism

[2] https://mp.weixin.qq.com/s/WxuNjGmqil5GmYYSFJOqVw

[3] https://zhuanlan.zhihu.com/p/670072518，《HuangLab 两篇区块链论文被 INFOCOM'24 接收》

[4] http://xintelligence.pro/archives/1371，《两篇区块链论文被顶会 INFOCOM 2025 接收 (附论文下载)》

[5] https://mp.weixin.qq.com/s/rU3zytC2xurI1S7VH1msZw

[6] https://www.researchgate.net/publication/370671128_Scheduling_Most_Valuable_Committees_for_the_Sharded_Blockchain

[7] https://mp.weixin.qq.com/s/Li7WJQdnFKWko8kBnCd80A

[8] https://www.researchgate.net/publication/350152541_MVCom_Scheduling_Most_Valuable_Committees_for_the_Large-Scale_Sharded_Blockchain

[9] https://mp.weixin.qq.com/s/3EUO5Pt5-4hwGWtCJaFnIQ

[10] https://www.researchgate.net/publication/364255361_Achieving_Scalability_and_Load_Balance_across_Blockchain_Shards_for_State_Sharding

[11] https://mp.weixin.qq.com/s/UMsQ7VIzyU-nzW5MzUXPfg

[12] https://www.researchgate.net/publication/370633426_tMPT_Reconfiguration_across_Blockchain_Shards_via_Trimmed_Merkle_Patricia_Trie

[13] https://mp.weixin.qq.com/s/M7KxjWTyheygrgKgDfz97g

1.6 BlockEmulator 视频教程

- 快速上手 BlockEmulator 的视频教程 [1]
- 如何将 BlockEmulator 部署在多终端，以及如何查看实验数据 [2]
- 如何使用 BlockEmulator-X 的视频教程 (英文配音)[3]

[1]快速上手 BlockEmulator: https://www.bilibili.com/video/BV1go4y177JY/?vd_source=208afae902c32b68341f17d8578dcdfe

[2]将 BlockEmulator 部署在多终端: https://www.bilibili.com/video/BV1L1421o7nz/?vd_source=c5d7d7737dddc0717729ba9c9f3ef544

[3]BlockEmulator-X 的视频教程: https://www.bilibili.com/video/BV1zuVN6UECD/?vd_source=208afae902c32b68341f17d8578dcdfe

1.7 BlockEmulator 第一年发展历程

（本小节内容来源自黄华威老师在知乎上的专栏文章 [1]。）

2024 年 5 月 26 日中午在 BlockEmulator 用户群，有位来自北京理工大学的张同学问到了 BrokerChain 论文（“BrokerChain: A Cross-Shard Blockchain Protocol for Account/Balance-based State Sharding” [7]）的系统原型实现代码的细节，而且提到如何基于 BlockEmulator [2] 的早期版本做二次开发。随后，BlockEmulator 后期成熟版本的贡献者之一殷昭伉同学回复了张同学的问题。

不过，我发现昭伉还有 BlockEmulator 社区用户普遍对 BrokerChain [INFOCOM 2022] 与 BlockEmulator 的关系，以及 BlockEmulator 的诞生背景、历史发展过程不太了解。

趁此良机，我写了这篇小作文，希望可以帮助 BlockEmulator 社区用户还有读者朋友们了解这个实验工具的诞生背景与第一年发展历程。

1.7.1 BlockEmulator 的诞生背景

BrokerChain 与 BlockEmulator 的关系

BrokerChain 是我们 HuangLab 在 2021 年设计并实现的分片区块链 (分片区块链系统框架 + 独特的 Broker 中间商机制)，论文正式发表在 INFOCOM 2022。

BrokerChain 论文中提到的实验框架其实是 BlockEmulator[3] 的最早

[1] 《BlockEmulator 诞生背景与第一年的发展历程》：`https://zhuanlan.zhihu.com/p/699908738`，作者：区块链黄博士

[2]《BlockEmulator：可支持分片协议的区块链仿真测试平台》：`https://zhuanlan.zhihu.com/p/665703706`

[3] BlockEmulator 主页：`https://www.blockemulator.com`

版本，是论文二作彭肖文同学 2021 年（当时他处于研一暑假期间）使用了 python 和 java 写的，比较简陋，甚至某些非关键环节（比如 PBFT 协议的消息广播过程）使用了 simulation 代码来模拟。后来论文在 2022 年发表之后，黄华威老师带领 team 把这个实验框架逐步扩展并完善每一个重要模块，开发语言换成了 Go 语言，后来就逐渐形成了 BlockEmulator[1]。在 2022 年 2023 年逐步完善 BlockEmulator 的过程中，它的每个中间版本都分别用来支撑了 HuangLab 每一篇陆续发表出来的 blockchain 论文（论文列表请前往 chapter 1.5 查看）。

BlockEmulator 项目开源

随着这些论文陆续发表，很多同学纷纷通过邮件向黄老师索要各个论文背后的代码，于是黄华威老师带领 team 在 2023 年 5 月 11 日正式开源了 BlockEmulator，里边嵌入了大部分研究组发表出来的论文的算法代码。可以这样理解：BlockEmulator 是一个篮子，里边陆续装填了 HuangLab 若干区块链论文中提出的机制与算法。

黄老师开源 BlockEmulator 的愿景是希望它可以为在区块链方向从事科研的同学提供一个方便做实验的平台，同时也希望大家一起把 blockchain sharding 这个小众方向给带起来。

1.7.2 BlockEmulator 的发展演化

开源的同时，HuangLab 也陆续收到来自社区用户的反馈，这对 BlockEmulator 也是一个接受检验与提高的过程。事实也的确如此，不少同学在

[1]知乎文章《BlockEmulator：可支持分片协议的区块链仿真测试平台》https://zhuanlan.zhihu.com/p/665703706

各个平台分别反馈了他们在使用过程中遇到的问题，比如在 B 站的 block-Emulator 视频教程页面 [1]，在 Github 的 issues 页面[2] 纷纷留言。

每一个技术问题黄老师都转发给负责 BlockEmulator 代码维护的同学，比如叶光、殷昭伉、陈钦德，有时也会让林建入老师来回复。他们也各自做出了及时的反馈并提供了非常棒的解决方案。

图 1.3: The Eng. version homepage of BlockEmulator.

比起刚开源时，现在 BlockEmulator 更加稳定，而且各种使用文档、视频教程都陆续完善。甚至，为了扩大它在国际上的受众范围，在 2024 年初时，黄老师带领 team 把官网从中文版改为了现在的英文版，如图 1.3 所示。

1.8 BlockEmulator 社区持续发展

根据 2026 年 7 月的统计数据显示（如图1.4所示），BlockEmulator 官网已得到来自近 100 个国家与地区用户的访问。

Github 项目页面显示（如图 1.5 所示），BlockEmulator 项目已经获得了 350 个星，97 个 forks。

[1] https://www.bilibili.com/video/BV1go4y177JY，“BlockEmulator 视频教程”
[2] https://github.com/HuangLab-SYSU/block-emulator/issues

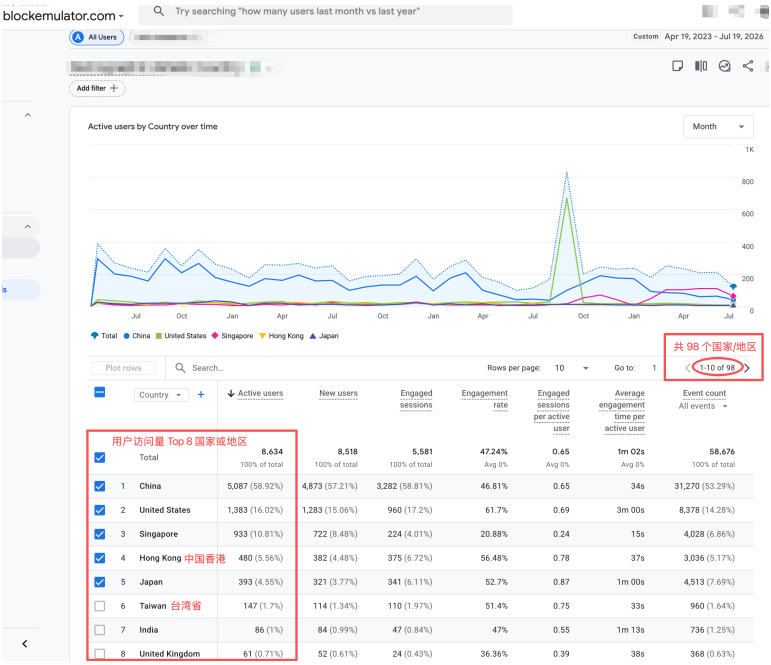


图 1.4: The visitor data of the blockEmulator homepage.

虽然 BlockEmulator 还未获得世界范围内的广泛关注，但是 HuangLab 成员仍然为它能帮助一些同行而感到高兴。

之后，如果同学们在使用 BlockEmulator 的过程中遇到任何问题，可以先去 Github 的 issues 页面（HuangLab-SYSU/block-emulator/issues ）看一下你遇到的问题是否已经被前人问过了。如果没有的话，欢迎在 issues 页面或者在 BlockEmulator 用户微信群里提出来，负责 BlockEmulator 维护的同学会第一时间给大家解答。

感谢大家的关注，如果你认为 BlockEmulator 是个有用的实验工具，欢迎转告给其他正在区块链方向默默耕耘的同学与朋友。顺便，加入 Block-

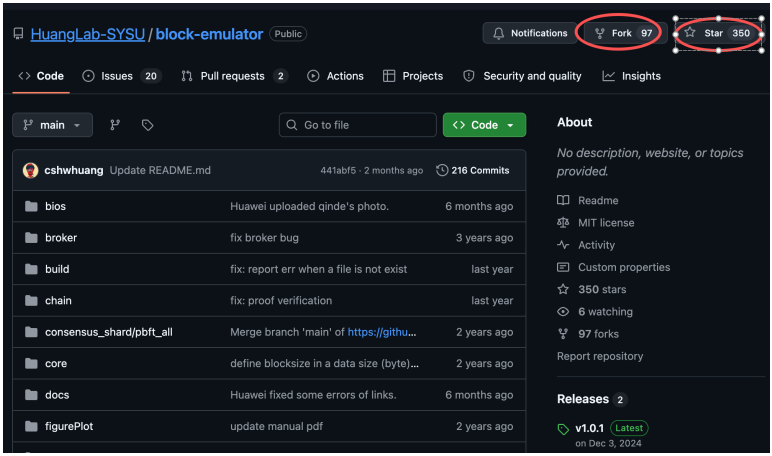


图 1.5: The GitHub project of blockEmulator.

Emulator 社群的方法在官网[1]的底部，感兴趣的同学请自行探索。

[1]BlockEmulator 主页：`https://www.blockemulator.com`

第二章　设计原理

2.1　设计思路

2.1.1　BlockEmulator 的定位是什么？

BlockEmulator 设计定位是一个可支持多种共识协议的区块链实验平台，以支持区块链跨分片机制为特色。

该实验平台主要面向区块链研究人员，当他们需要对提出的新型区块链共识协议、新型跨分片机制进行验证时，可以帮助他们快速搭建一个轻量化的区块链底层协议的实验平台，并对实验数据进行收集，方便绘制科研论文所需的实验图。

BlockEmulator 需要实现区块链的各项底层模块，它不仅可以帮助初学者快速入门理解区块链底层原理，也能为有经验的区块链研究者提供一个便捷的区块链新协议/算法/机制的开发测试环境。为此，它需要降低开发和测试的成本和难度，为开发人员和研究人员提供可定制化的二次开发环境，从而可促进区块链技术的创新。

2.1.2　BlockEmulator 能用来做什么？

BlockEmulator 的设计目标是为了帮助用户（研究者、学生）快速实现并测试他们提出的新型区块链共识协议和区块链跨分片机制。它遵循了轻量化区块链系统架构的设计理念，简化了区块链系统实验环境的搭建流

程。这是因为 BlockEmulator 仅仅关注实现区块链的核心功能，比如交易池、区块打包、区块共识、交易上链等核心环节，并且支持常见的几种主流共识协议，如拜占庭容错（Practical Byzantine Fault Tolerance, PBFT）协议与工作量证明机制。

特别地，BlockEmulator 对主流的“区块链分片机制”进行了系统底层级别的设计与实现。其中，“跨分片交易”机制包含以下两个具有代表性的分片协议：Monoxide [16] (NSDI'2019) 方案中提出的“Relay 交易机制”，以及 BrokerChain [7] (INFOCOM'2022) 中的“broker 机制”。

因此，BlockEmulator 可以支持用户快速开发定制化版本的区块链新机制/协议/算法，尤其支持对区块链新型分片机制与协议的功能测试。

2.1.3 BlockEmulator 有什么特点？

- **支持定制化二次开发：** BlockEmulator 采用 Go 语言实现，能够定制化二次开发，满足不同需求。

- **快速搭建：** 不仅可以在本地进行实验，还可以远程部署到云端运行。

- **易于实验：** BlockEmulator 支持主流区块链（如以太坊）历史交易数据回放，可以自动输出、保存区块链实验指标，如系统吞吐量、交易确认时延、交易池拥塞程度等等。

- **容易上手：** 无需复杂设置，用户就能快速上手做实验并收集数据绘制图表。

2.2 架构与组件

2.2.1 系统架构设计

BlockEmulator 采用分层的方法进行设计，每一层都负责独立的工作，并且尽可能的让每一层都只与其邻近层发生交互，实现系统层面的功能结构，帮助用户快速熟悉系统并进行代码复用和改造。

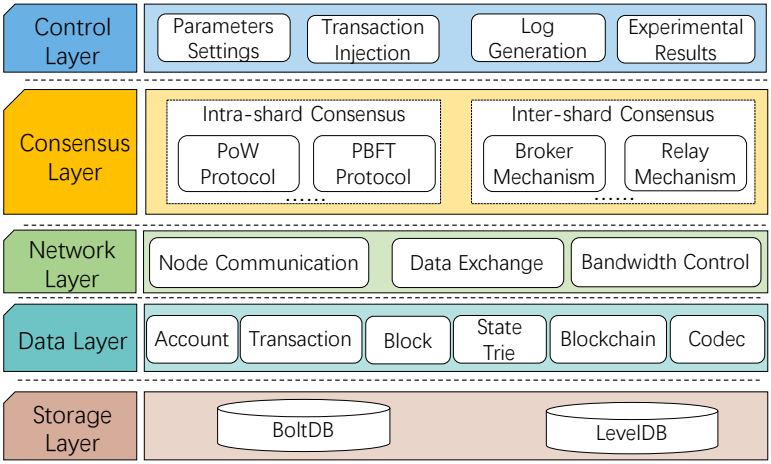


图 2.1: BlockEmulator 的分层系统架构设计。

如图 2.1 所示，BlockEmulator 的整个架构可以划分为具体的五个层。按照层次接近真实数据（交易数据）的远近可以划分为，存储层、数据层、网络层、共识层和控制层。各个层介绍如下。

1. **存储层**：存储层将账本数据存储在磁盘上，包括生成的区块链数据、进行实验时生成的日志文件等。例如，共识节点保存已确认的区块、交易、账户状态和系统日志。存储层的功能基于 boltDB 和 levelDB

实现。特别地，我们在 levelDB 上采用了 Merkle Patricia 树（MPT），以提供验证功能。

这一层的代码有两部分。一部分代码位于 `./storage` 文件夹中。另一部分我们采用了以太坊的源码。

2. **数据层**：数据层在系统中建立了数据的基本结构，包括账户、交易、区块、状态树和节点。数据层与存储层进行交互。在运行 BlockEmulator 时，新生成的区块会通过数据层不断生成和编码。这些新生成的区块触发状态树的修改，状态树中新生成的树节点会被本地编码保存。此外，当实验完成后，可以通过存储层查询存储在文件中的区块数据。查询到的区块数据随后可以通过数据层解码还原为区块。这样，研究人员可以访问生成的区块和账户状态树。因此，用户如果对 BlockEmulator 生成的交易执行结果的正确性有疑问，可以离线检查账户状态。

   这部分的代码位于 `./core` 和 `./chain` 文件夹中。

3. **网络层**：网络层旨在支持部署在不同设备上的共识节点之间的通信。我们通过 TCP 实现了端到端的数据包传输机制，旨在节点之间交换数据。交换的数据包括交易、区块、共识消息等。我们将 TCP 实现为长连接（Keep-Alive connection）的方式，以便在网络中促进大量消息的交换。在这种方式下，两个节点之间的 TCP 连接仅在整个实验过程中初始化和关闭一次。另外，我们为每一个消息设计了传输头，以便于节点在接收到 TCP 报文后能快速识别出是这个报文对应的是哪一类消息。确定了消息类型后，即可触发对应的消息处理函数处理该消息。

   这部分的代码位于 `./networks` 和 `./message` 文件夹中。

4. **共识层**：共识层负责使共识节点针对新区块达成一致。在分片区块链中，节点同时运行着分片内和跨分片的共识机制。分片内的共识机制（如 PoW 和 PBFT）确保一个分片内的区块达成共识。跨分片机制则保证不同分片之间的通信。例如，为了处理一笔跨分片交易，位于付款方和收款方分片的共识节点需要接收来自其他分片的消息。

   这部分的代码主要位于 `./consensus_shard` 文件夹中。

5. **控制层**：控制层负责系统的执行。在实验开始之前，该层需要为实验环境配置做准备。实验环境的准备包括初始化指标测试线程、监听消息、确定每个共识节点的 IP 地址等。当 BlockEmulator 运行时，控制层的 supervisor 节点将按照预定速率将交易注入到分片的交易池中。注入到 BlockEmulator 中的交易可以是来自任何区块链的历史交易，如以太坊。当 BlockEmulator 完成实验后，用户可以从日志中收集实验结果进行分析。

   这部分的代码主要位于 `./supervisor` 和 `./params` 文件夹中。

2.2.2 BlockEmulator 的系统角色

BlockEmulator 的共识体系主要包括两类角色的节点，分别是 worker 节点和 supervisor 节点，如图 2.2 所示，分别介绍如下。

Worker 节点

Worker 节点即负责打包交易、共识区块和维护账本数据的节点。这些节点通过共识机制，保证了区块链的活性和安全性。在 BlockEmulator 中，这类节点都是“全节点”。

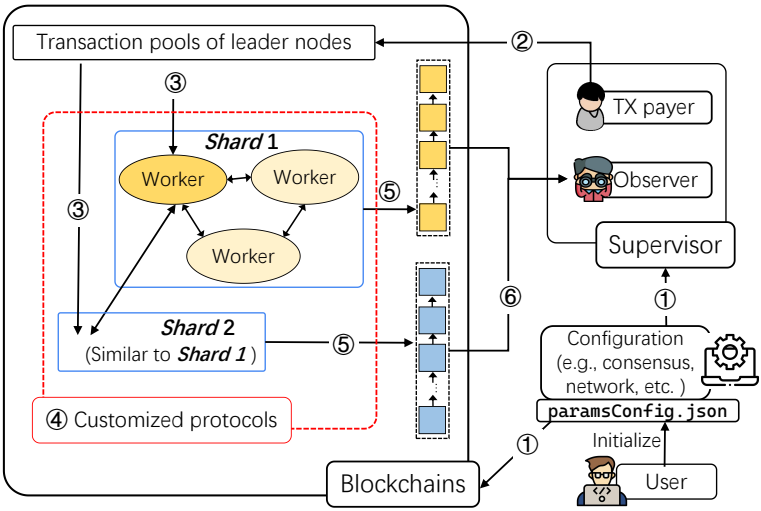


图 2.2: BlockEmulator 的主要角色，与内部工作流程。

Supervisor 节点

为了便于统计实验数据、简化开发代码逻辑，我们在 BlockEmulator 中引入了 supervisor 节点。它融合了多个系统角色，具有以下功能：

- **交易发送者**。Supervisor 节点可以充当交易发送者，负责将交易不断地发送到区块链网络中。
- **实验数据观测员**。Supervisor 节点可以充当区块链网络中的观察者，可持续不断地监听区块链数据，通过计算来统计区块链的各项性能指标。
- **共识协议中的 committee**。有些共识协议（如 CLPA [13]）需要利用 committee 来计算分片区块链重配置的结果，这时，supervisor 可担任这个稍微带有中心化色彩的角色。

- **Broker 角色**。在 BrokerChain 协议 [7] 中，一笔跨分片交易会被某个 broker 账户拆分为两笔耦合的子交易。Supervisor 也可以担任 broker 的角色，这样可以减轻实验代码的开发量。

2.2.3 内部运行流程概述

接下来，我们从系统内部运行的视角介绍当系统启动后，supervisor 与 worker 节点将会如何协作。请注意，这部分内容并不是介绍用户如何启动 BlockEmulator，而是以区块链系统底层的视角介绍 BlockEmulator 内部不同角色的节点是如何运作的。启动 BlockEmulator 的方法请跳转 chapter 4.1 查看。

如图2.2所示，BlockEmulator 内部的工作流程主要包括如下几个步骤：

- Step 1: 在启动之前，用户进行系统配置（包括共识、网络等）。
- Step 2: Supervisor 节点向 worker 分片的交易池中注入交易。
- Step 3: Worker 分片的节点从交易池中读取交易。
- Step 4: Worker 分片根据用户自定义开发的协议进行共识。
- Step 5: Worker 节点们将共识完成后的区块上链，并且更新状态树，将数据保存到本地。
- Step 6: Supervisor 节点观察区块链网络、并且收集区块，根据区块信息测量区块链的各项性能指标。

2.2.4 支持测量的实验指标

Supervisor 节点可测量的指标

BlockEmulator 中的大部分指标由 supervisor 节点来测量。因此，如果要在 BlockEmulator 中测量指标，需要在创建 supervisor 时，指定 supervisor 测量的指标名称。

通过选择 `mearsureModNames...` 的取值，supervisor 节点可以调整需要测量的内容。`mearsureModNames` 在 `./build/build.go` 中是这样定义的:

```
methodID := params.ConsensusMethod
var measureMod []string
if methodID == 0 || methodID == 2 {
    measureMod = params.MeasureBrokerMod
} else {
    measureMod = params.MeasureRelayMod
}
measureMod = append(measureMod, "Tx_Details")

spv:= new(supervisor.Supervisor)
spv.NewSupervisor(supervisor_ip, chainConfig,
    committeeMethod, mearsureModNames...)
```

目前开源的代码中，包含了以下 9 个可测量的实验指标。(位于代码 `./supervisor/measure` 文件中)。

指标类型	说明
`Tx_Details`	即每笔交易的详细数据，适用于以下两种跨分片交易处理模式。Tx_Details 给出了每一笔交易的各种时间，用户可以利用它统计各种细化的指标。
Relay 跨分片交易处理模式	

指标类型	说明
`TPS_Relay`	即每秒处理的交易量（transactions per second, TPS）。它是区块链系统每秒处理交易的平均数目。该指标针对交易中继（relay）机制，一笔跨分片需要上链两次才算处理完成，所以每次上链的权重仅有 1/2；一笔片内交易仅需一次上链便能处理完成，所以每次上链权重为 1。
`TCL_Relay`	即交易确认时延（transaction confirm latency, TCL）。某一笔交易的 TCL 是指该交易从进入交易池到最终确认上链所耗费的时间，而本指标测试的是一批交易的平均 TCL。该指标针对交易中继（relay）机制，由于一笔跨分片需要被确认上链两次，所以它的 TCL 包含了这两次上链的排队和共识时间。
`CrossTxRate_Relay`	即跨分片交易占比（Cross-shard transaction ratio）。它指的是跨分片交易占所有交易的比例。该指标针对交易中继（relay）机制，一笔跨分片需要上链两次，但是在统计跨分片交易占比时，每次上链的权重仅为 1/2。
`TxNumberCount_Relay`	即交易总数（Number of transactions）。它指的是区块链在某一段时间内处理了多少交易。该指标可以用来检验区块链是否正确处理了所有注入的交易。更确切地说，一次实验正确完成的条件是，`TxNumberCount_Relay==`{`TotalDataSize`}，其中 {`TotalDataSize`} 是在 `./paramsConfig.json` 中定义的“注入交易总数”。
Broker account 跨分片交易处理模式	

指标类型	说明
`TPS_Broker`	即每秒处理的交易量（transactions per second, TPS）。它是区块链系统每秒处理交易的平均数目。该指标针对中间人账户（broker account）机制，一笔跨分片需要被拆为两笔 broker 交易，两笔 broker 交易需要都上链才算处理完成，所以每次上链的权重仅有 1/2；一笔片内交易仅需一次上链便能处理完成，所以每次上链权重为 1。
`TCL_Broker`	即交易确认时延（transaction confirm latency, TCL）。某一笔交易的 TCL 是指该交易从进入交易池到最终确认上链所耗费的时间，而本指标测试的是一批交易的平均 TCL。该指标针对中间人账户（broker account）机制，由于一笔跨分片需要被中间人账户拆分为两笔，所以它的 TCL 包含了拆分操作的耗时、两笔交易的排队时间和两笔交易的共识时间。
`CrossTxRate_Broker`	即跨分片交易占比（Cross-shard transaction ratio）。它指的是跨分片交易占所有交易的比例。该指标针对中间人账户（broker account）机制，统计时，一笔交易的权重取决于它需要上链的次数。
`TxNumberCount_Broker`	即交易总数（Number of transactions）。它指的是区块链在某一段时间内处理了多少交易。该指标可以用来检验区块链是否正确处理了所有注入的交易。更确切地说，一次实验正确完成的条件是，`TxNumberCount_Relay=={TotalDataSize}`，其中 `{TotalDataSize}` 是在 `./paramsConfig.json` 中定义的“注入交易总数”。

Tips：

- 如果用户想要新增自定义指标，可以在 `./supervisor/supervisor.go` 文件中的 `NewSupervisor` 函数中，**追踪 `mearsureModNames` 变量**是如何生效的。
- Supervisor 的统计结果，会被存储在 `./{ExpDataRootDir}/result/supervisor_measureOutput/{metric_name}.csv` 文件中。

Worker 节点可测量的指标

仅仅从 supervisor 这样一个区块链 “观察者” 的视角，BlockEmulator 无法对区块链的全部性能指标进行统计与观测。因此，worker 节点被设计为也需要负责统计一些指标。具体来讲，Worker 节点会在出块的时候进行统计某些指标，包括：

指标类型	说明
`Block Height`	当前区块的高度
`EpochID of this block`	出块时，区块链处于哪一个 epoch
`TxPool Size`	出块时，当前分片的交易池大小
`# of all Txs in this block`	该区块内的交易数目
`# of {Broker1,Relay1} Txs in this block`	该区块内的 {Broker1, Relay1} 交易的数目

指标类型	说明
`# of {Broker2,Relay2} Txs in this block`	该区块内的 {Broker2, Relay2} 交易的数目
`TimeStamp-Propose (unix Millisecond)`	该区块提出（即共识开始）的时间戳
`TimeStamp-Commit (unix Millisecond)`	该区块确认（即共识完毕）的时间戳
`SUM of confirm latency (ms, All Txs)`	所有交易的 TCL 之和
`SUM of confirm latency (ms, {Broker1, Relay1} Txs)`	所有 {Broker1, Relay1} 交易的 TCL 之和
`SUM of confirm latency (ms, {Broker2, Relay2} Txs)`	所有 {Broker2, Relay2} 交易的 TCL 之和

Tip:

Worker 的统计结果，将会被存储在 `./{ExpDataRootDir}/result/pbft_shardNum={ShardNums}` 文件夹中。

2.2.5 输出的日志和实验结果

各个节点的日志查看

所有节点的日志默认输出路径为 `./{ExpDataRootDir}/log`，该目录下的文件组织如下：

```
/log
|-- supervisor.log
|-- S0
|   |-- N0.log
|   \-- N1.log
|-- S1
|   |-- N0.log
|   \-- N1.log
```

日志文件记录格式在 `./consensus_shard/pbft_all/pbft_log/log_module.go`(worker 节点)和 `./supervisor/supervisor_log/supervisor_logmod.go`（supervisor 节点)。

Worker 节点和 supervisor 节点的 log 格式均是以下格式：

```
（Worker 节点）
S0N0: 2024/12/09 20:10:40 messageHandle.go:141: S0N0 :
    has broadcast the prepare message
（Supervisor 节点）
Supervisor: 2024/12/09 20:10:42
    committee_clpa_broker.go:258: received from shard
    0 in epoch 0.
```

上述输出，每一行的内容从左到右分别是：节点的名称（包括分片序号和节点序号)，打印日志的时间，触发打印的代码位置，打印的信息。

Supervisor 节点输出的实验指标

Supervisor 的统计结果，将会被放在 `./{ExpDataRootDir}/result/supervisor_measureOutput` 文件夹中。

默认情况下，Supervisor 节点输出的文件组织如下：

```
/ supervisor_measureOutput
   |-- Average_TPS.csv
   |-- CrossTransaction_ratio.csv
   |-- Transaction_Confirm_Latency.csv
   |-- Tx_Details.csv
   \-- Tx_number.csv
```

各个文件包含了一个测试指标的细节，csv 文件中的每一列数据均有对应的列名解释。

Worker 节点输出的实验指标

Worker 的统计结果，将被放在 `./ExpDataRootDir/result/pbft_shardNum=ShardNums` 文件夹中。

默认情况下（以 `ShardNums=4` 为例），Worker 节点输出的文件组织如下：

```
/pbft_shardNum=4
    |-- Shard04.csv
    |-- Shard14.csv
    |-- Shard24.csv
    |-- Shard34.csv
```

其中上述 csv 文件的名称格式为 `Shard{ShardID}{ShardNumber}.csv`。比如，`Shard04.csv` 表示分片数目为 4 的区块链系统中，序号为 0 的分片所对应的结果。

各个文件包含了每个分片 leader 节点输出的指标，csv 文件中的每一列数据均有对应的列名解释。

2.2.6 可配置的系统参数

经过若干次的迭代，BlockEmulator 的系统参数配置功能被整理到了 `./paramsConfig.json` 和 `./ipTable.json` 两个配置文件中。

实验参数配置

`./paramsConfig.json` 文件中包含了关于区块链共识、数据集路径、实验结果输出路径、带宽控制等功能的参数。以下是这些参数的解释：

参数名	参数描述
`ConsensusMethod`	共识节点采用的共识机制。0~3 分别表示{“CLPA_Broker”, “CLPA”, “Broker”, “Relay”}。
`PbftViewChangeTimeOut`	PBFT 共识中，如果一轮共识时间超过了 `PbftViewChangeTimeOut`，便会触发 view change。
`ExpDataRootDir`	实验结果的数据输出路径。
`Block_Interval`	出块间隔（秒）。
`BlockSize`	区块大小（交易为单位）。
`BlocksizeInBytes`	区块大小（字节数为单位）。
`UseBlocksizeInBytes`	交易大小是否采用“字节数为单位”。
`InjectSpeed`	交易注入网络中的速度。
`TotalDataSize`	被注入的交易总数。

参数名	参数描述
TxBatchSize	每次发送的交易数目(较大时可以减少 TCP 请求数)。
BrokerNum	Brokerchain 机制中，broker 账户的数目。
RelayWithMerkleProof	使用 Relay 机制时，是否采用 Merkle Proof 验证交易。
DatasetFile	数据集文件路径。
ReconfigTimeGap	两次 reconfiguration 阶段的时间间隔。每隔 ReconfigTimeGap 秒，触发一次重配置。
Delay	网络延迟（毫秒)。
JitterRange	网络扰动（毫秒)，以均匀分布进行波动。
Bandwidth	限制的带宽大小。

IP 地址配置

./ipTable.json 文件中包含了各个共识节点的 IP 地址。在用户的实验环境，请按需更改这些节点的 IP 地址。

```
{
    "0": {
        "0": "127.0.0.1:32217",
        "1": "127.0.0.1:32227"
    },
    "1": {
        "0": "127.0.0.1:32317",
        "1": "127.0.0.1:32327"
    },
    "2147483647": {
        "0": "127.0.0.1:38800"
```

```
12         }
13     }
```

其中，json 文件的第一级索引表示分片序号，第二级表示节点序号。按照上面代码的逻辑，可知第 0 号分片的第 0 号节点的 IP 地址加端口为：`127.0.0.1:32217`，其他的类似，以此类推。

特别地，第 2147483647 号分片的第 0 号节点（IP 地址加端口为 `127.0.0.1:38800`）表示 supervisor 节点。

2.3 内置共识机制

BlockEmulator 内置了 PBFT 协议当做分片内使用的共识机制，此外还有 3 种原生的“跨分片共识协议”或机制，即交易中继（transaction relaying）机制 [16]，BrokerChain 协议的“做市商”交易机制 [7]，与 CLPA 账户重划分算法 [13]。

2.3.1 跨分片交易中继机制（Transaction Relaying）

交易中继（Transaction Relaying）机制的背景

- 交易中继机制来源于 Monoxide [16]，其采用消息传递的方式，由分片间消息传递进行跨分片交易的处理。

- 一笔跨分片交易需要被“源分片（source shard）”和“目标分片（destination shard）”分别进行“上链”操作：i）源分片对该交易的发送方账户状态进行更新；ii）目标分片对该交易接收方的账户状态进行更新。

交易中继机制工作流程的简要描述

1. Supervisor 节点将每一笔交易发送到该交易发送者账户所在的分片。
2. Worker 节点生成区块。如果区块中的某一笔交易被判定为跨分片交易，则在这一步，该交易的“源分片”中的 worker 节点仅需要更新该跨分片交易发送者的账户状态。
3. 当该**跨分片交易上链**之后，work 节点将当前跨分片交易以消息的形式发送到交易接收者所在的分片（即“目标分片”）。
4. 该交易的目标分片内的 worker 节点收到该跨分片交易之后，会将该跨分片交易发送到交易池，等待打包上链。
5. 目标分片的某个 worker 节点将交易打包至区块，这一步仅需要更新该跨分片交易接收者的账户状态。
6. 至此，该跨分片交易**第二次上链**，则该交易的执行周期结束。

交易中继机制的代码实现

Relay 机制的实现大部分位于 `./consensus_shard/pbft_all/pbftOutside_module.go` 和 `consensus_shard/pbft_all/pbftInside_module.go` 两个文件中，以下是代码运行逻辑的大致介绍。

1. 节点交易池维护 `RelayPool` 结构，负责暂存跨分片交易，并提供 `RelayPool` 的如下两个操作函数：
   - `AddRelayTx(tx*Transaction,shardIDuint64)`：负责将交易添加到指定分片的中继池（Relay Pool）中。
   - `ClearRelayPool()`：负责清空中继池。

2. 当区块链经过共识，准备将一个区块上链之前，worker 节点会检测区块中是否存在跨分片交易（节点会判断交易的发起者与接收者是否属于同一个分片：如果不是，那么这笔交易就是一笔跨分片交易）。如果一笔交易是跨分片交易且没有被**中继处理 (relaying)** 过，那么 worker 节点仅更新交易发起者的账户状态。在这之后，节点会调用 `AddRelayTx` 函数，将此跨分片交易添加到指定的 `RelayPool` 中。

3. 节点遍历并处理区块内所有交易后，将每个 `RelayPool` 中的跨分片交易分别打包为 `message.Relay` 消息，并分别发送到对应分片的节点中（PBFT 中为 leader 节点）。

```go
// 代码位于 ./consensus_shard/pbft_all/toolFuncs.go 文件中。

// help to send Relay message to other shards.
func (p *PbftConsensusNode) RelayMsgSend() {
    for sid := uint64(0); sid < p.pbftChainConfig.ShardNums; sid++ {
        if sid == p.ShardID {
            continue
        }
        relay := message.Relay{
            Txs:           p.CurChain.Txpool.RelayPool[sid],
            SenderShardID: p.ShardID,
            SenderSeq:     p.sequenceID,
        }
        rByte, err := json.Marshal(relay)
        if err != nil {
            log.Panic()
        }
        msg_send := message.MergeMessage(message.CRelay, rByte)
        go networks.TcpDial(msg_send, p.ip_nodeTable[sid][0])
        p.pl.Plog.Printf("S%dN%d : sended relay txs to %d\n", p.ShardID, p.NodeID, sid)
    }
    p.CurChain.Txpool.ClearRelayPool()
}
```

4. 对应分片的 worker 节点收到 `message.Relay` 消息后，他们会将交易从消息中解析出来，然后将这些交易添加到当前节点的交易池中。

```
// 代码位于 ./consensus_shard/pbft_all/pbftOutside_module.go 文件中。

// receive relay transaction, which is for cross shard txs
func (rrom *RawRelayOutsideModule) handleRelay(content []byte) {
    relay := new(message.Relay)
    err := json.Unmarshal(content, relay)
    if err != nil {
        log.Panic(err)
    }
    pbftNode.ShardID, rrom.pbftNode.NodeID, relay.SenderShardID, relay.SenderSeq)
    rrom.pbftNode.CurChain.Txpool.AddTxs2Pool(relay.Txs)
    rrom.pbftNode.seqMapLock.Lock()
    rrom.pbftNode.seqIDMap[relay.SenderShardID] = relay.SenderSeq
    rrom.pbftNode.seqMapLock.Unlock()
}
```

5. 同 2，节点完成交易的打包、共识和上链。因为一笔已经被中继处理过的跨分片交易不会再被放到 `RelayPool` 中，所以此时只需要更新交易接收者的账户状态，便可以完成该跨分片交易的处理。

2.3.2 “做市商”跨分片机制（BrokerChain Protocol）

Broker 机制的背景

BrokerChain 协议提出的“做市商”跨分片交易处理机制，来自发表在 INFOCOM 2022 的论文 *BrokerChain: A Cross-Shard Blockchain Protocol for Account/Balance-based State Sharding* [7]。该论文针对分片区块链系统中广泛存在大量跨分片交易的现象提出了 BrokerChain 跨分片交易共识机制。按照该机制的设计，系统中存在某些特殊的 broker 账户（即“做

市商账户”）。它们中的任意一个的账户状态可以被分割为多份，并且每一份仅分布于一个分片之中。相当于每一个 broker 账户在多个分片中开设了“子账户”。

Broker 机制的原理

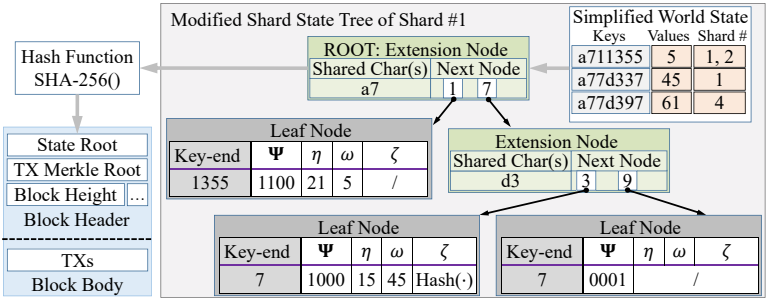


图 2.3: Data structure of the modified *Shard State Tree* (mSST).

尽管某个 broker 有多个“子账户”，但是这些归属于同一个 broker 账户的若干个“子账户”仍然共用同一个“外部账户”地址。如图2.3所示，账户地址为“a711355”的 broker 账户状态被分割到了两个分片中，即 Shard #1 与 #2。但是，从系统外部用户的角度来看，这两个子账户就是同一个账户，因为它们共用同一个账户地址“a711355”。只是在 BrokerChain 的协议层，它们会被判定为不一样的“分账户”。为了达到上述效果，作者设计了图2.3示的新型数据结构叫做“修改之后的分片状态树（modified Shard State Tree，mSST）”。在 mSST 中，每个“外部账户”的叶子结点（Leaf Node）的数据结构中被添加了一个扩展字段，即图2.3中的 Ψ 字段，它可以用来设置一个类似于 id 的标识符，以此来区分归属于同一个 broker 账户的多个子账户。例如图2.3中，中间部位的叶子结点，Key-end 为 1355 对应的 Ψ 字段为“1100”（四位数字代表系统中有 4 个分片；此

处每个数字从左向右的位数代表第几个分片；数字“1”代表在该分片上部署有子账户），意思是账户地址为“a711355”的 broker 的两个子账户被分割到了第 1 号与第 2 号分片，即 Shard #1 与 #2。

按照上述“账户状态分割机制”的设计，当分片间出现跨分片交易的时候，BrokerChain 分片区块链就可以使用这些 broker 账户来帮助处理这些跨分片交易。

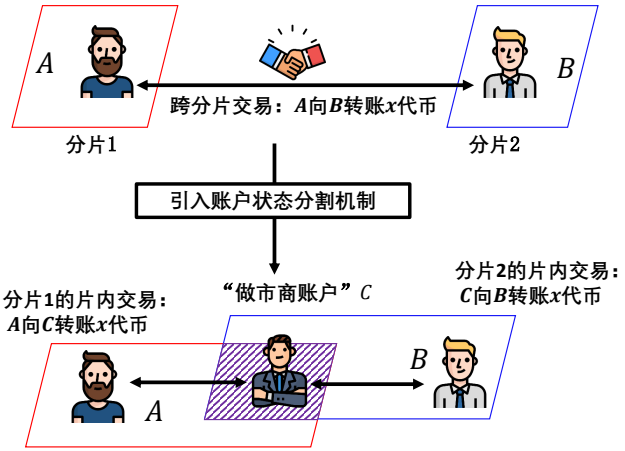


图 2.4: BrokerChain 跨分片协议中的“做市商”交易机制。

图 2.4 展示了“做市商账户”的跨分片交易处理机制。其中，“做市商账户” C 就是一个 broker 账户。通过“账户状态分割”机制，它在分片 1 与分片 2 存在两个“分账户”。那么，当一笔交易的发起者向系统提交了一个原始交易时，这笔交易有较大概率会被协议判定为是一笔跨分片交易。例如，当分片 1 内的账户 A 向另一个分片 2 内的账户 B 发起一笔转账交易时，这笔转账交易就是一笔跨分片交易。那么，在 BrokerChain 的协议层，这笔跨分片交易会被拆解为两笔“耦合”的“片内”子交易:

- **子交易 1：**账户 A 向分片 1 内的 broker 账户 C 进行片内转账。

- **子交易 2：** 分片 2 内的 broker 账户 C 向账户 B 进行片内转账。

当这两笔子交易完成后，系统就实现了那笔原始跨分片交易（A 向 B 转账 x 数量的代币）的上链处理。可见，通过借助做市商账户 C，BrokerChain 分片区块链系统可以大大减少跨分片交易的数量。

BrokerChain 协议中跨分片交易的处理流程简述

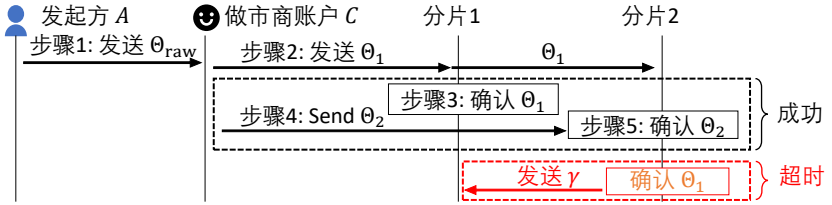


图 2.5: BrokerChain 协议中跨分片交易处理流程。

图 2.5 展示了一笔由 broker 账户服务的跨分片交易的执行时序图，具体流程描述如下：

- 步骤 1：Sender A 发送原始交易消息 θ_{raw} 给 Broker C。
- 步骤 2：Broker C 收到原始消息 θ_{raw} 后，分别给原始交易发送方账户和接收方账户所在分片（图中 Shard1、Shard2 ）发送 θ_1。
- 步骤 3：Shard1 节点接收到 θ_1 后，判断消息合法性，构建 A 和 C 之间的交易（Tx1），如果交易上链，则向 Broker 发送 `Confirm`θ_1 消息。
- 步骤 4：Broker C 接收到 `Confirm`θ_1 消息，验证消息的有效性后，向 Shard2 发送 θ_2。

- 步骤 5：Shard2 节点接收到 θ_2 后，判断消息合法性，构建 C 和 B 之间的交易（Tx2），如果交易上链，则向 Broker C 发送 `Confirm`θ_2 消息完成跨分片交易。

两类角色在 BlockEmulator 框架中的实现逻辑

上述跨分片交易处理流程中提到了两个关键角色：Sender 与 Broker。它们在 BlockEmulator 代码层面的实现逻辑分别解释如下。

- **Sender**：Sender 理应为区块链系统的客户端。在 BlockEmulator 的代码层面，目前 Sender 的角色由 supervisor 节点担当。Sender 负责检测接收的交易是否为跨分片交易，如果是跨分片交易，则生成 θ_{raw}，并发送给某个 broker 账户。

- **Broker** ： Broker 的角色由 supervisor 节点担当，负责处理跨分片交易的请求。

  1. 在分片区块链 BrokerChain 网络中，每一个 broker 账户将会被分割到多个分片中，而且这些 broker 账户不参与区块链网络的状态迁移。
  2. Broker 接收到跨分片交易 θ_{raw} 后，Broker 负责生成 θ_1 ，并将其发送给 supervisor。随后，supervisor 验证消息、生成片内交易 Tx1，并添加 Tx1 到相应的分片交易池。
  3. Broker 负责监控区块链上链情况，当接收到 `Confirm`θ_1 消息后，生成 θ_2 并发送给 supervisor，supervisor 验证消息、生成片内交易 Tx2，并添加 Tx2 到对应分片交易池。
  4. Broker 接收到 `Confirm`θ_2 消息后，跨分片交易 θ_{raw} 处理完成。

5. 具体举例说明：A 为分片 1 中的账户，B 为分片 2 中的账户，如果 A，B 之间发生了转账交易，记为 $\langle A \to B \rangle$。Broker 检测到跨分片交易后，经过一系列信息交换，跨分片交易 $\langle A \to B \rangle$ 被转变为对应于分片 1 和分片 2 的两笔片内交易，即 $\langle \text{Tx1} : A \to \text{Broker} \rangle$ 和 $\langle \text{Tx2} : \text{Broker} \to B \rangle$。

Broker 机制的代码实现

在 BlockEmulator 项目中，Broker 机制的实现分为两个部分：第一部分是 Broker 数据结构，第二部分是 Broker 相关消息数据结构的实现。

Broker 数据结构

Broker 实例包含了与 Broker 相关的变量，用来存储交易处理过程中生成的信息。Broker 的数据结构代码位于 `./broker/broker.go`。

```
type Broker struct {
    BrokerRawMegs  map[string]*message.BrokerRawMeg
    ChainConfig    *params.ChainConfig
    BrokerAddress  []string
    RawTx2BrokerTx map[string][]string
}
```

具体的变量说明如下：

变量	类型	说明
`BrokerRawMegs`	`map[string]*BrokerRawMeg`	θ_{raw} 摘要和 `*BrokerRawMeg` 的映射。
`ChainConfig`	`*params.ChainConfig`	系统配置信息。

变量	类型	说明
BrokerAddress	[]string	Broker 地址。
rawTx2BrokerTx	map[string][]string	原始跨分片交易和 Broker 产生的两个片内交易的映射。

同时 Broker 实例提供了处理跨分片交易必要的方法，代码位于 ./supervisor/committee/committee_broker.go。

```
// generate a new broker
func NewBroker(pcc *params.Cha in config)

// get the digest of rawMeg
func getBrokerRawMagDigest(r *message.BrokerRawMeg)
    []byte

// Get parition (if not exist, return default)
func fetchModifiedMap(key string) uint64

// Handle the raw messsage
func handleBrokerRawMag(brokerRawMags
    []*message.BrokerRawMeg)

// Handle the tx1
func handleTx1ConfirmMag(mag1confirms
    []*message.Mag1Confirm)

// Handle the tx2
func handleTx2ConfirmMag(mag2confirms
    []*message.Mag2Confirm)

//init broker address
func initBrokerAddr(num int) []string
```

Broker 相关消息的数据结构

θ_{raw}：Sender 向 Broker 发送的原始交易的消息。

```
type BrokerRawMeg struct {
    Tx         *core.Transaction
    Broker     utils.Address
```

```
    Hlock      uint64 //ignore
    Snonce     uint64 //ignore
    Bnonce     uint64 //ignore
    Signature []byte // not implemented now.
}
```

θ_1：Broker 向原始交易的发起者所在分片发送的信息。

```
type BrokerType1Meg struct {
    RawMeg    *BrokerRawMeg
    Hcurrent uint64         //ignore
    Signature []byte // not implemented now.
    Broker    utils.Address
}
```

$Confirm\theta_1$：原始交易发起者账户所在分片将 Tx1 上链后，向 Broker 发送的确认信息。

```
type mag1Confirm struct {
    Tx1Hash []byte
    RawMeg  *BrokerRawMeg
}
```

θ_2：Broker 向原始交易的接收者所在分片发送的信息。

```
type BrokerType2Meg struct {
    RawMeg *BrokerRawMeg
    Signature []byte // not implemented now.
    Broker utils.Address
}
```

$Confirm\theta_2$：原始交易的接收者所在分片，将 Tx2 上链后，向 Broker 发送的确认信息。

```
type mag2Confirm struct {
    Tx2Hash []byte
    RawMeg  *BrokerRawMeg
}
```

我们在 BlockEmulator 项目中为上述消息的数据结构都提供了对应的处理方法，例如这里给出了一些相关方法的函数签名：

函数签名	说明
handleBrokerRawMag(brokerRawMags []*message.BrokerRawMeg)	Broker 接收 θ_{raw} ，产生并发送 θ_1。
handleTx1ConfirmMag(mag1confirms []*message.Mag1Confirm)	Broker 接收 Confirmθ_1，产生并发送 θ_2。
handleTx2ConfirmMag(mag2confirms []*message.Mag2Confirm)	Broker 接收 Confirmθ_2，记录跨分片交易上链结果。
fetchModifiedMap(key string) uint64	客户端判断交易中账户所属分片。
dealTxByBroker([]*core.Transaction)[]*core.Transaction	获取注入交易中的跨分片交易，生成 θ_{raw} 发送给 Broker。
handleBrokerType1Mes([]*message.BrokerType1Meg)	supervisor 处理来自 Broker 的 θ_1, 生成片内交易 Tx1, 并加入对应分片交易池。
handleBrokerType2Mes([]*message.BrokerType2Meg)	supervisor 处理来自 Broker 的 θ_2, 生成片内交易 Tx2, 并加入对应分片交易池。
createConfirm([]*core.Transaction)	supervisor 跟踪区块中交易记录，分别产生 Confirmθ_1 和 Confirmθ_2 发送给 Broker。

2.3.3 CLPA 账户重划分算法

由于传统分片区块链的账户划分方式基本是静态的，这会导致有些分片会出现交易负载过高（热分片），而有些分片交易负载较小（冷分片）的情况。在这种时候，**一个合适的账户重新划分算法就能够动态地调整某些账户所在的分片，以达到降低跨分片交易数量、均衡分片间负载的效果**。

BlockEmulator 内置了账户划分的功能，一个可选的账户划分算法为 CLPA（Constrained Label Propagation Algorithm）。此方法来自于发表在 SRDS 2022 的论文 *Achieving Scalability and Load Balance across Blockchain Shards for State Sharding* [13]。

CLPA 的数据结构设计

CLPA 有关的数据结构均在 **package partition (./partition)** 中定义。由于 CLPA 本质上是一个图划分算法，因此实现 CLPA 之前，我们首先需要实现一个 Graph 的数据类，如下：

```
// 图中的顶点
type Vertex struct {
    Addr string // 节点具体属性
    // 其他属性待补充
}
// 图
type Graph struct {
    VertexSet map[Vertex]bool     // 顶点集合
    EdgeSet   map[Vertex][]Vertex //
        以邻接表的形式记录顶点之间的边
}
```

借助上述的 Graph 类，我们参考原文定义了如下的 `CLPAState` 类：

```
// CLPA算法状态， state of CLPA
type CLPAState struct {
    NetGraph          Graph          //
        需运行CLPA算法的图
    PartitionMap      map[Vertex]int // 记录分片信息的
        map，某个节点属于哪个分片
```

```
    Edges2Shard        []int          // Shard 相邻接的边数，对应论文中的 total weight of edges associated with label k
    VertexsNumInShard  []int          // Shard 内节点的数目
    WeightPenalty      float64        // 权重惩罚，对应论文中的 beta
    MinEdges2Shard     int            // 最少的 Shard 邻接边数，最小的 total weight of edges associated with label k
    MaxIterations      int            // 最大迭代次数，constraint，对应论文中的\tau
    CrossShardEdgeNum  int            // 跨分片边的总数
    ShardNum           int            // 分片数目
    GraphHash          []byte
}
```

需要说明的是，将 CLPA 算法应用到分片区块链场景下时，顶点（Vertex）指的是账户（account），边（edge）指的是交易（transaction），**重新调整图的过程其实就是重新调整账户所在分片的过程**（即，对账户图进行重划分其实在执行分片重组，shard reconfiguration）。

CLPA 的函数设计

上述的 CLPAState 类中定义了一些方法，其中可供外部函数调用的方法有：

```
// 加入顶点，将它默认归到一个分片中
func (cs *CLPAState) AddVertex(v Vertex)

// 加入边，将它的端点（如果不存在）默认归到一个分片中
func (cs *CLPAState) AddEdge(u, v Vertex)

// 初始化 CLPA 状态，设置参数
func (cs *CLPAState) Init_CLPAState(wp float64, mIter, sn int)

// 调用 CLPA 划分算法，返回结果是 （账户与账户所属分片的映射，重组后的跨分片边的数目）
func (cs *CLPAState) CLPA_Partition() (map[string]uint64, int)
```

CLPAState 的方法可以在 ./partition/clpa_test.go 文件中进行测试。用户也可以仿照这个文件中使用 CLPA 的方式，将 CLPA 移植到其他区块链中。

Supervisor 节点调用 CLPA 的流程

通过上述的 CLPA 算法函数的描述，supervisor 节点可以地调用 CLPA 算法，具体代码位于 supervisor/committee/committee_clpa.go，逻辑如下：

1. 首先，supervisor 节点需要创建 CLPAState 类，并且设置参数（weightPenalty=0.5，MaxIterations=100，与原论文保持一致）。

```
cs = new(partition.CLPAState)
cs.Init_CLPAState(weightPenalty_input,
    MaxIterations_input, shardNumber_input)
```

2. 接下来，supervisor 节点将交易作为边，置入 CLPAState 中：

```
var u, v Vertex
cs.AddEdge(u, v)
// 如果 u 或 v 不存在于当前 CLPAState 中，
// 则顶点也会被置入CLPAState中，
// 并以静态分片的方式置入对应分片
```

3. 插入若干条边之后，supervisor 节点可以运行 CLPA 算法，以得到更新后的账户划分表和该情况下跨分片交易的数目：

```
// dirtyMap: 算法返回的是更新后的
    dirtyMap，一个账户存在于此 Map
    中，仅当它在本次更新中改变了所属分片
// crossTxNum: 该划分情况下的跨分片交易数目
dirtyMap, crossTxNum := cs.CLPA_Partition()
```

4. 由于 dirtyMap 仅仅记录了某一次运行 CLPA 算法更新后的结果。如果需要多次调用 CLPA 算法，那么算法结果的接收方应当维护一个记录着所有运行结果的 PartitionMap。这样，每次接收到 dirtyMap，PartitionMap 都会得到更新，代码为：

```
var PartitionMap map[string]uint64
// 保存了自 CLPA 算法运行以来的所有账户变动
// (每个账户的所属分片被维护为最新版本)
for key, val := range dirtyMap {
    PartitionMap[key] = val
}
```

5. 最后，当 supervisor 运行完 CLPA 后，将会把 dirtyMap 作为输出结果发送给 worker 分片的所有 leader 节点。

Worker 节点利用 CLPA 进行重组的流程

Worker 节点收到来自于 supervisor 节点的 CLPA 重组消息 handlePartitionMsg 后，将会进入“账户重分配”阶段。在这个阶段中，worker 节点会调用 ./consensus_shard/pbft_all/accountTransfer_module.go 内的函数进行正确地处理重分配逻辑。

1. Worker 节点接收到 CLPA 重组消息 handlePartitionMsg，将在本轮共识结束后进入重分配阶段。该函数位于 ./consensus_shard/pbft_all/pbftOutside_moduleCLPA.go 文件中。

```
// the leader received the partition message
    from listener/decider,
// it init the local variant and send the
    accout message to other leaders.
func (crom *CLPARelayOutsideModule)
    handlePartitionMsg(content []byte)
```

2. 本轮共识结束后，worker 节点**进入重分配阶段**。该分片的 leader 节点**告知其他分片，它已经就绪**。该函数位于 `./consensus_shard/pbft_all/accountTransfer_module.go` 文件中。

```
// This function sends the ready messages to other shards.
// this message used in propose stage, so it will be invoked by InsidePBFT_Module
func (cphm *CLPAPbftInsideExtraHandleMod) sendPartitionReady()
```

3. 所有分片都已经就绪后，worker 分片根据 CLPA 重组消息 `handlePartitionMsg`，**挑选本分片内的账户状态和交易，将它们迁移到其他分片**。该函数位于 `./consensus_shard/pbft_all/accountTransfer_module.go` 文件中。

```
// send the transactions and the accountState to other leaders
func (cphm *CLPAPbftInsideExtraHandleMod) sendAccounts_and_Txs()
```

4. 每一个分片的 leader 节点接收来自于其他分片的**迁移数据**。该函数位于 `./consensus_shard/pbft_all/accountTransfer_module.go` 文件中。

```
// all nodes in a shard will do accout Transfer, to sync the state trie
func (cphm *CLPAPbftInsideExtraHandleMod) accountTransfer_do(atm *message.AccountTransferMsg)
```

5. **迁移完成后，leader 节点发起共识**，将账户状态重分配的信息打包为共识，广播给同分片内的其他节点。该函数位于 `./consensus_shard/pbft_all/accountTransfer_module.go` 文件中。

```
// propose a partition message
```

```
func (cphm *CLPAPbftInsideExtraHandleMod)
    proposePartition() (bool, *message.Request)
```

CLPA Blockchain 运行流程

1. **Supervisor 节点设置：**按照如下方法创建 supervisor 节点，即配置 committee 为 **CLPA committee**。

```
// 构建 Supervisor 对系统进行监听。
// 参数3 CLPA 指定了 Supervisor 具备 CLPA
    committee 的功能
var spv Supervisor
spv.NewSupervisor(supervisor_ip, chainConfig,
    "CLPA", testModules...)
```

2. **Worker 节点设置：**按照如下方法创建 worker 节点，即可以让 worker 节点能够接收、处理 CLPA 相关的消息。

```
// 创建 worker 节点。
// 参数 4 CLPA 指定了 worker 节点将采用 CLPA
    共识算法。
worker := pbft_all.NewPbftNode(sid, nid, pcc,
    "CLPA")
```

3. **重复第 2 步，启动多个 worker 节点**。

4. **等待运行结束，收集数据**。

第三章　BlockEmulator 技术论文

3.1　技术论文撰写背景

虽然我们已经在 BlockEmulator 的主页与 Github 代码仓库详细地介绍了该实验平台的设计理念与用户使用手册，但是仍然缺少一个更加严谨易懂的介绍文档。为此，我们在项目开源之后整理了一篇技术论文。论文题目为《BlockEmulator: An Emulator Enabling to Test Blockchain Sharding Protocols》[8]。

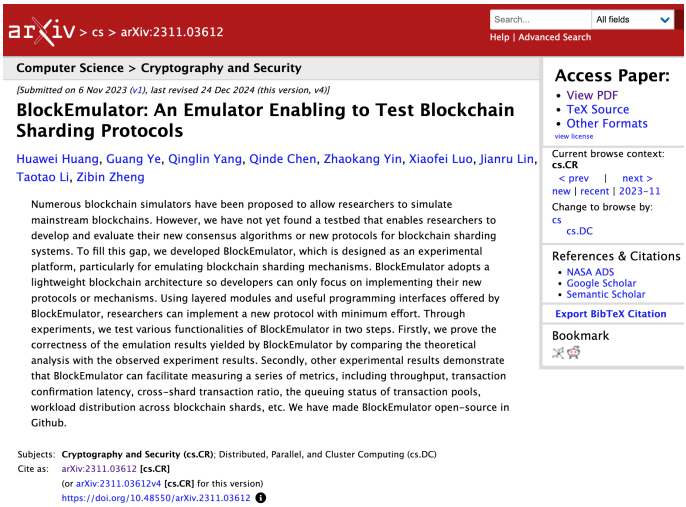


图 3.1: BlockEmulator 技术论文的 arXiv 页面。

2023 年 11 月，我们在将该技术论文投稿至 IEEE TSC 期刊后，同时

也将第一个版本上传至 arXiv 预印本网站。如图 3.1 所示，arXiv 页面访问链接为：https://arxiv.org/abs/2311.03612。目前，该论文已经发表在 IEEE TSC 期刊。如果您使用了 BlockEmulator 做实验，请别忘记引用我们这篇技术论文，bibTex 信息如下：

```
@article{huang2025blockemulator,
    title = {BlockEmulator: An Emulator Enabling to
        Test Blockchain Sharding Protocols},
    author = {Huang, Huawei and Ye, Guang and Yang,
        Qinglin and Chen, Qinde and Yin, Zhaokang and
        Luo, Xiaofei and Lin, Jianru and Zheng, Jian
        and Li, Taotao and Zheng, Zibin},
    journal = {IEEE Transactions on Services Computing
        (TSC)},
    year      = {2025},
    volume    = {18},
    number    = {2},
    pages     = {690--703}
}
```

3.2 技术论文简介

区块链仿真器对于区块链体系架构方向的研究者是必备的实验工具。目前常见的区块链仿真器通常被用于模拟主流区块链或运行区块链的各种协议机制。现有的区块链仿真器各有优点。但我们经过详细的调研，却没有发现有任何一个区块链仿真器能够支持区块链分片机制。为了填补这方面的空白，我们设计并开发了 BlockEmulator。它被设计为一个区块链新协议、新算法、新机制的实现与性能测试平台。特别地，研究者用户可以借助它来实现和验证区块链分片机制。

BlockEmulator 采用了轻量化的模块化架构设计。与复杂的现有区块链架构不同，它仅有数千行代码，且采用了多层级的架构设计。这种设计可以使研究者们将精力集中于自己需要二次开发的模块，而不用耗费大量时间精力在其他部分。通过 BlockEmulator 提供的现有机制和接口，研究者

们能够快捷且方便地设计新型区块链协议、网络消息分发机制、共识算法等等。更重要的是，BlockEmulator 提供了丰富的编程接口支持实验指标的测量以及实验数据的收集，可支持的常见实验指标包括交易吞吐量、交易确认时延、跨分片交易的比例、各个分片交易池内交易动态变化的情况等等。

为了验证 BlockEmulator 的有效性，我们在技术论文中展示了两组实验。第一组实验中，我们将理论推导结果与从 BlockEmulator 中观测到的实验结果相比较，验证了 BlockEmulator 产生实验数据的正确性。在第二组实验中，我们在 BlockEmulator 可扩展代码模块中实现了四种现有分片机制，并使用 BlockEmulator 对多种指标进行了测量；展示了 BlockEmulator 在收集各种丰富实验指标方面的实用功能。

研究者可根据自己的需求进行二次开发，只需对 BlockEmulator 的可扩展代码模块进行修改和重新配置，然后一个定制化的区块链系统得以实现。随着系统运行，实验结果也将会自动记录保存在本地日志文档中，方便研究者后续绘制实验图。

3.3 BlockEmulaotr 技术论文的引用

截至 2026 年 5 月，BlockEmulaotr 的技术论文已经得到 70 多次引用，如图 3.2 所示。

根据这些引用了 BlockEmulator 的论文，我们欣喜地发现一些同行已经发表的工作用 BlockEmulator 成功地进行了二次开发，实现了他们自己定制化开发的区块链协议、机制和算法。这些工作的作者在他们的实验中也使用了 BlockEmulator 对他们的设计进行了详细的实验评估。这正是我们研究组开源这个项目所期望看到的情景。

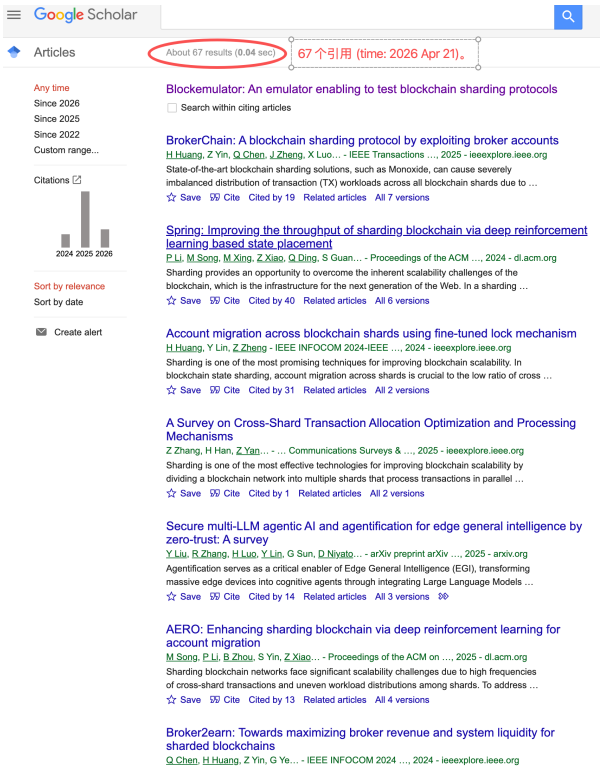
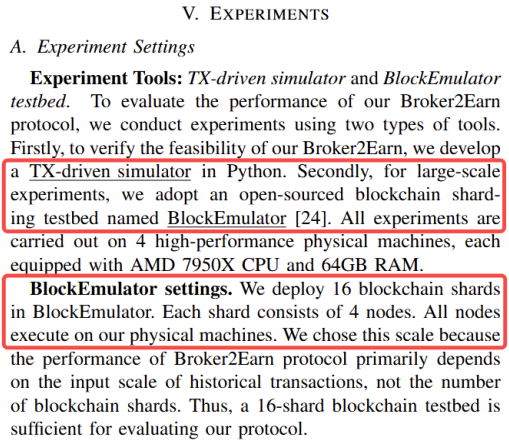


图 3.2: BlockEmulator 技术论文的部分引用情况。

接下来我们选择其中的几个案例作为示例，以展示用户如何使用 BlockEmulator 进行区块链二次开发与开展实验。

- 图 3.3 展示了 Chen 等作者使用 BlockEmulator 评估了 Broker2Earn (B2E) 的性能 [1]。作者通过修改 supervisor 模块，将提出的 B2E 协议嵌入到测试平台中。这个案例体现了 BlockEmulator 分层设计所带来的灵活性。

- 图 3.4 展示了 Ding 等作者如何使用 BlockEmulator 实现其提出的



V. EXPERIMENTS

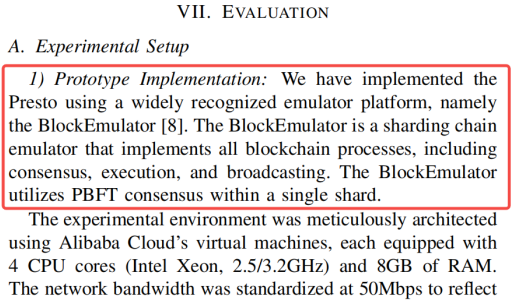
A. Experiment Settings

Experiment Tools: *TX-driven simulator* and *BlockEmulator testbed*. To evaluate the performance of our Broker2Earn protocol, we conduct experiments using two types of tools. Firstly, to verify the feasibility of our Broker2Earn, we develop a TX-driven simulator in Python. Secondly, for large-scale experiments, we adopt an open-sourced blockchain sharding testbed named BlockEmulator [24]. All experiments are carried out on 4 high-performance physical machines, each equipped with AMD 7950X CPU and 64GB RAM.

BlockEmulator settings. We deploy 16 blockchain shards in BlockEmulator. Each shard consists of 4 nodes. All nodes execute on our physical machines. We chose this scale because the performance of Broker2Earn protocol primarily depends on the input scale of historical transactions, not the number of blockchain shards. Thus, a 16-shard blockchain testbed is sufficient for evaluating our protocol.

图 3.3: Chen 等作者在 BlockEmulator 上评估了 B2E [1]。

Presto 协议 [3]。正如文中所说，由于 BlockEmulator 集成了区块链的完整流程，它有效减轻了作者在实现 Presto 协议时的开发负担。



VII. EVALUATION

A. Experimental Setup

1) Prototype Implementation: We have implemented the Presto using a widely recognized emulator platform, namely the BlockEmulator [8]. The BlockEmulator is a sharding chain emulator that implements all blockchain processes, including consensus, execution, and broadcasting. The BlockEmulator utilizes PBFT consensus within a single shard.

The experimental environment was meticulously architected using Alibaba Cloud's virtual machines, each equipped with 4 CPU cores (Intel Xeon, 2.5/3.2GHz) and 8GB of RAM. The network bandwidth was standardized at 50Mbps to reflect

图 3.4: Ding 等作者使用 BlockEmulator 实现了 Presto [3]。

- 在图 3.5，Li 等作者通过 BlockEmulator 评估了方案的效果。此外，作者还修改了 BlockEmulator 的 state placement 模块来满足他们的需求。这进一步彰显了 BlockEmulator 的灵活性和可扩展性。

5 EVALUATION

5.1 Experimental Settings

For the evaluation, the A-Shard is implemented via simulation in Python 3.9 and the T-Shard is implemented via a blockchain testbed. The agent is trained on a machine with an Intel(R) Core(TM) i7-9750H CPU @ 2.60 GHz, 64 GB RAM, and an NVIDIA GeForce RTX 2080. Additionally, we verify the trained result with an open-sourced sharding blockchain testbed, BlockEmulator [14]. BlockEmulator uses PBFT as its intra-shard consensus protocol and a relay-based approach for CSTs, which is consistent with our design described in Section 3. The T-Shard is deployed on 64 ecs.c7.large instances from the same zone in Alibaba Cloud with 16 shards and four consensus nodes in each shard. To demonstrate the adaptability of SPRING, we use real Ethereum TX data from 2015, 2019, and 2023 [52], encompassing up to six million TXs.

5.5.3 Throughput Exploration. The throughput is the number of TXs the sharding blockchain can process per second (TPS). We collect the TPS using BlockEmulator [14] and modify the state placement module of the BlockEmulator to verify the effectiveness of SPRING. As shown in Table 2, SPRING consistently outperforms the other algorithms. The performance of baselines is similar in all periods. Thus, we only present the result in 2023. Moreover, the TPS is improved up to 36.03% with 2023 TX data, which is higher than the reduction in CSTR, which is 20.99%. This indicates that CST's impact on the TPS is significant. Since the cross-shard needs an extra processing mechanism, it is more time-consuming. Consequently, reducing the CSTR does improve throughput, and SPRING outperforms other baselines.

图 3.5: Li 等作者使用 BlockEmulator 评估了方案的效果，并且修改了 BlockEmulator 内的模块 [14]。

- 图 3.6 展示了研究者如何使用 BlockEmulator 进行实验的另一个例子。在 TransShard 中，Wang 等作者将 BlockEmulator 作为一个交易驱动的模拟器，用它来重放以太坊的真实历史交易 [17]。

V. SIMULATION

A. EXPERIMENTAL SETTINGS

To demonstrate the effectiveness of TransShard, we implement an experimental prototype and a TX-driven simulator based on BlockEmulator [40] (an experimental blockchain platform using Golang), running on a machine with a Core i7 CPU running on a processor speed of 3.7 GHz and 32 GB RAM. BlockEmulator supports for common consensus protocols, such as PBFT [35]. And it also includes relay transaction mechanism [26] for cross-shard TXs. The TX-driven simulator is used to replay real Ethereum transaction data at a certain speed. Based on the data provided by XBlock [19], we randomly select three sets of one million consecutive transactions and show the average results of them.

图 3.6: 在 TransShard [17] 中，Wang 等作者使用 BlockEmulator 作为一个交易驱动的模拟器。

- 正如图 3.7 所示，Wu 等作者使用 BlockEmulator 模拟了交易的确认时延。通过 BlockEmulator，这些作者成功模拟了区块链分片操作，并在接近现实的环境中测量了交易确认的延迟。这为优化交易有效

In our simulate experiment, we utilize BlockEmulator [23] to simulate the real transaction confirmation latency in sharded blockchain systems. This decision is based on BlockEmulator's sophisticated ability to replicate the complex operations and network conditions of sharded blockchains accurately. Its detailed emulation of transaction processing, consensus mechanisms, and inter-shard communication provides a realistic environment in which to measure and analyze TX confirmation latency. Additionally, its capability to mimic real-world network scenarios, including packet transmission and bandwidth constraints, ensures that our latency measurements reflect practical blockchain conditions. This makes BlockEmulator an essential tool for our research, offering valuable insights into optimizing the transaction efficiency and scalability in sharded blockchain architectures.

In our study, we ensure that each shard queue remains stable, meaning that, for λ_{all}, the expected service time E[S] is less than the arrival rate λ. Figure 6 illustrates the transaction latency in a simulated sharded blockchain environment under real transaction conditions,

图 3.7: Wu 等作者使用 BlockEmulator 模拟了交易确认时延 [18].

性和系统可扩展性提供了有价值的启发。

- Huang 等作者使用 BlockEmulator 实现并评估了他们提出的账户迁移协议 [5]。如图 3.8 所示，作者提到 BlockEmulator 不仅提供了一系列内置机制，而且提供了丰富的试验指标测试功能。

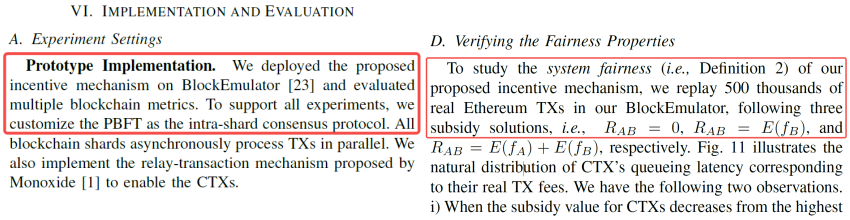
VI. Performance Evaluation

We implemented the proposed account migration protocol on top of an open-sourced blockchain testbed named *BlockEmulator* [35]. The fundamental functionalities of *BlockEmulator* include the following 4 points. i) It offers various customized consensus protocols. ii) It allows clients to import historical TXs into the transaction pool of the testbed. iii) It supports the built-in sharding mechanism. iv) It also provides multiple-metric measurement capabilities, such as recording transaction throughput, TX makespan, and other crucial metrics of a sharded blockchain. We then evaluate the proposed account migration protocol as well as two other locking mechanisms using *BlockEmulator*. The proposed protocol is implemented in GO with 5000+ lines of code.

图 3.8: Huang 等作者在 BlockEmulator 中实现和评估了他们提出的账户跨分片迁移协议 [5]。

- 如图 3.9 所示，Zheng 等作者在 BlockEmulator 上实现了他们所提出的 Justitia 机制 [22]，并基于以太坊的历史交易数据进行了广泛的性

能评估[1]。BlockEmulator 展示的实验结果表明，作者提出的 Justitia 激励机制不仅显著降低了跨分片交易在交易池中的排队时延，还避免了系统高额补贴导致的经济通胀问题。这个案例证明了 BlockEmulator 在实现分片区块链系统底层新机制方面的实用性。



VI. Implementation and Evaluation

A. Experiment Settings

Prototype Implementation. We deployed the proposed incentive mechanism on BlockEmulator [23] and evaluated multiple blockchain metrics. To support all experiments, we customize the PBFT as the intra-shard consensus protocol. All blockchain shards asynchronously process TXs in parallel. We also implement the relay-transaction mechanism proposed by Monoxide [1] to enable the CTXs.

D. Verifying the Fairness Properties

To study the *system fairness* (*i.e.,* Definition 2) of our proposed incentive mechanism, we replay 500 thousands of real Ethereum TXs in our BlockEmulator, following three subsidy solutions, *i.e.*, $R_{AB} = 0$, $R_{AB} = E(f_B)$, and $R_{AB} = E(f_A) + E(f_B)$, respectively. Fig. 11 illustrates the natural distribution of CTX's queueing latency corresponding to their real TX fees. We have the following two observations. i) When the subsidy value for CTXs decreases from the highest

图 3.9: Zheng 等作者使用 BlockEmulator 实现了 Justitia 机制 [22]，并且使用 BlockEmulator 重放了 50 万条以太坊的历史交易用来测试他们提出的激励机制。

- 如图 3.10 所示，来自北航的 Huang 等作者提出了 ContribChain [19] 并且在 BlockEmulator 上实现了他们提出的 P-Louvain 算法与 CobtribChain 区块链协议 [2]。为了评估 ContribChain，作者在 BlockEmulator 上进行了实验，并且使用了真实以太坊交易数据注入到 BlockEmulator 进行实验测试。作者的实验主要关注两个方面：i) 在账户分配算法方面，对比了他们提出的 P-Louvain 算法与 TxAllo [21] 和 CLPA 算法 [13] 的性能差异；ii) 在系统性能测试方面，他们将实现了自定义 NACV 算法和 P-Louvain 算法的区块链系统 ContribChain-1、只实现了 P-Louvain 的系统 ContribChain-2、实现了 CLPA 算法的系统与 Monoxide 分片区块链系统进行了实验对比。值得一提

[1] https://mp.weixin.qq.com/s/rU3zytC2xurI1S7VH1msZw，《两篇区块链论文被顶会 INFOCOM 2025 接收》

[2] https://mp.weixin.qq.com/s/0e5K_wNoLyZ7qiKmtylDOw，《压力均衡协议如何提升区块链性能与安全?》

V. EVALUATION

A. Experiment Settings

We implemented P-Louvain and ContribChain on an open-sourced blockchain testbed *BlockEmulator* [16].

Dataset: We used historical Ethereum transactions [31] from block height 10,000,000 to 10,999,999 (approximately from May 4, 2020, to October 6, 2020). The maximum number of transactions used in the experiments reached 30 million.

Transaction Processing: Each block is set to contain a maximum of 2000 TXs. The block interval is set to 5 seconds. Transactions are sent to the system at a specific injection rate.

Baselines: We compared P-Louvain with TxAllo [9] and CLPA [8], where TxAllo is also a modified account allocation algorithm based on *Louvain*. In the system experiments, we set ContribChain-1 to use NACV and P-Louvain, and ContribChain-2 to use only P-Louvain. Then we compared them with CLPA and Monoxide [5], where Monoxide allocates accounts based on the first few digits of the account address.

Other Settings: The parameters μ, θ, λ, α, and β were set to 0.9, 1.5, 2, 0.7, and 2, respectively.

图 3.10: 来自北航的 Huang 等作者基于 BlockEmulator 实现了新型区块链协议 P-Louvain 与 ContribChain [22]。

的是，CLPA 算法和 Monoxide 协议都已经内嵌到了 BlockEmulaotr 的算法/协议模块，作者们因此可以大大减轻复现这些经典算法的代码实现负担。

第四章　开始使用它做实验

本节内容主要来自于 GitHub 上的 BlockEmulator 用户手册[1]，并根据最新版本 BlockEmulator 做了些许修改。

4.1　启动方式

启动 BlockEmulator 有两种方式：

1. **使用提前编译好的可执行文件**。
   - 无需安装 BlockEmulator 所需要的 packages / libraries。
   - 方便快速体验。
2. **手动编译、运行 BlockEmulator 的源代码**。

   这是二次开发 BlockEmulator 的必经之路，可以大幅度地修改代码，设计自己的共识协议。

两种方式本质上相差不大，因为**方案 2 相比于方案 1 仅仅是多了一个手动编译的过程**。因此，接下来我们会直接介绍方案 1。方案 2 则会在 chapter4.6 中介绍。

[1] https://github.com/HuangLab-SYSU/block-emulator/blob/main/20240906-blockEmulator-UserManual-OpenSource.pdf

4.2 事前准备

4.2.1 拉取源代码

首先，为了运行 BlockEmulator，用户需要在 GitHub[1] 上拉取 BlockEmulator 最新的**源代码**。

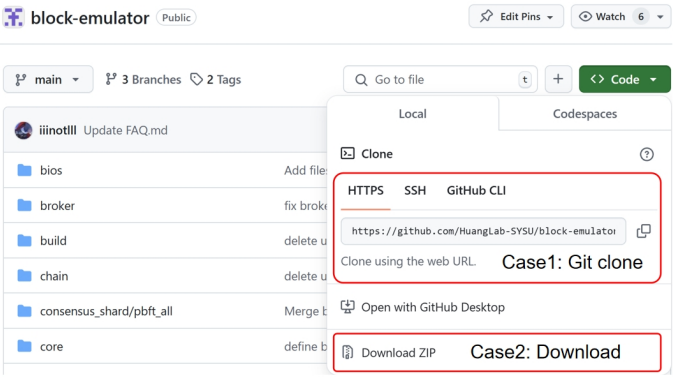


图 4.1: 下载 blockEmulator。

如图 4.1，拉取代码主要有两种方式：

1. 使用 `git clone` 命令。

   `git clone https://github.com/HuangLab-SYSU/block-emulator.git`。

2. 下载 ZIP 文件，然后解压缩到合适的文件夹。

完成拉取源码，所有 BlockEmulator 的源码将会被放在 `block-emulator` 文件夹内。

[1] `https://github.com/HuangLab-SYSU/block-emulator`

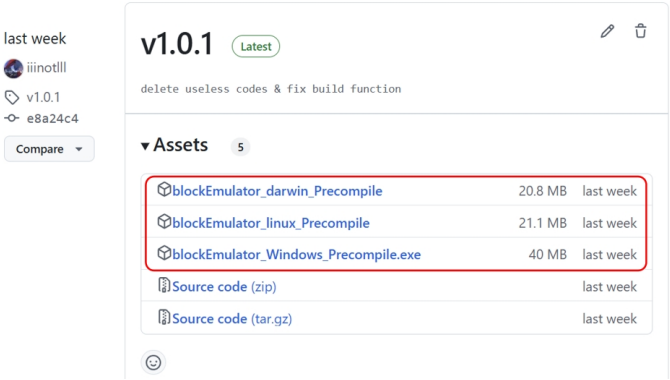


图 4.2: BlockEmulator 的 Release 界面，用户可以下载对应操作系统的，预先编译好的可执行文件。

4.2.2 获取预编译的可执行文件

为了帮助用户快速上手，免去下载 Go 语言和相关依赖的麻烦。我们将预先编译的可执行文件放在了 BlockEmulator-GitHub-Release [1]，作为一个体验版本。

进入上述 Release 页面后，可以下载图 4.2 中的 Releases 文件。

用户从中挑选自己操作系统适配的预编译可执行文件下载即可。**下载完成后，请将可执行文件放在 `block-emulator` 文件夹内，即放在 BlockEmulator 的根目录下**。

4.2.3 下载数据集

BlockEmulator 支持的数据集来自 xblock [2]。

为了便于用户运行，目前开源代码中附带了一个包含 300,000 条以太

[1] `https://github.com/HuangLab-SYSU/block-emulator/releases`
[2] `https://xblock.pro/#/dataset/14`, “Ethereum On-chain Data”

坊历史交易的数据集 selectedTxs_300K.csv。刚刚的“拉取源代码”操作已经获取了这个数据集，因此，小规模的实验并不需要前往 xblock 下载更多数据了。

如果用户追求大规模数据集下的实验，那么可以前往 xblock 网站下载。

4.3 启动 BlockEmulator

4.3.1 了解系统参数

BlockEmulator 中的系统参数可以分为两类：

1. 第一类系统参数。与**共识协议的具体设计细节**、**网络配置**、**网络规模**、**共识协议选择**、**实验结果输出目录**相关，可以在 paramsConfig.json 或 ipTable.json 文件中进行修改，修改后即可生效。

2. 第二类系统参数。与**指定节点身份**和**生成批处理文件**相关，可以在**命令行**中直接指定。

4.3.2 第一类系统参数

有关第一类参数的配置，其实我们在 2.2.6 节中就已经进行了阐述。在此，我们用图 4.3 和图 4.4 作为参数解释。

4.3.3 第二类系统参数

第二类系统参数通过命令行配置。命令行配置使用了 pflag 包的方法，用户可以在命令行中输入 --help 或者 -h 来获取帮助。

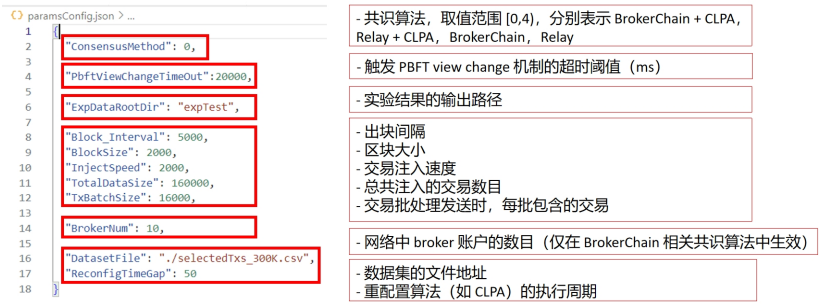


图 4.3: 在 ./paramsConfig.json 文件中可配置的系统参数。

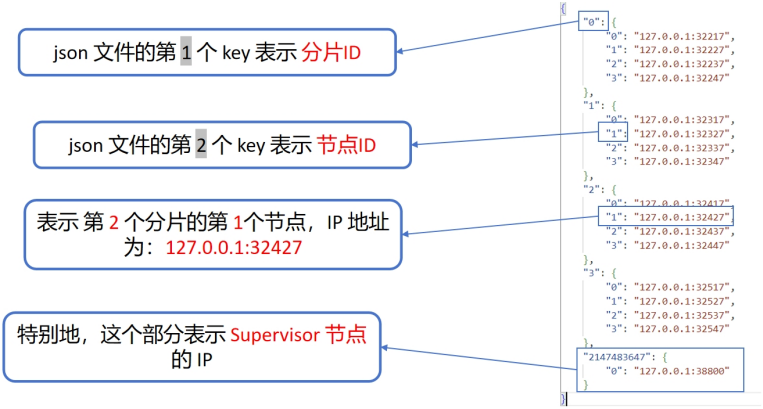


图 4.4: 在 ./ipTable.json 文件中可配置的系统参数。

在 block-emulator 文件夹的终端下输入指令：

./blockEmulator_Windows_Precompile.exe -h:

```
> .\blockEmulator_Windows_Precompile.exe -h
Usage of \blockEmulator_Windows_Precompile.exe:
  -g, --gen             isGen is a bool value, which
      indicates whether to generate a batch file
  -n, --nodeID int      nodeID is an Integer, which
      indicates the ID of this node. Value range: [0,
      nodeNum). (default -1)
  -N, --nodeNum int     nodeNum is an Integer, which
      indicates how many nodes of each shard are
      deployed.  (default 4)
  -s, --shardID int     shardID is an Integer, which
```

```
        indicates the ID of the shard to which this node
        belongs. Value range: [0, shardNum).  (default
        -1)
  -S, --shardNum int    shardNum is an Integer, which
      indicates that how many shards are deployed.  (
      default 4)
  -f, --shellForExe     isGenerateForExeFile is a bool
      value, which is effective only if 'isGen' is true
      ; True to generate for an executable, False for '
      go run'.
  -c, --supervisor      isSupervisor is a bool value,
      which indicates whether this node is a supervisor
      .
pflag: help requested
```

（**请注意：如果用户使用的是 Linux 或 MacOS，那么应该使用其对应的可执行文件，也就是 blockEmulator_linux_Precompile 或 blockEmulator_darwin_Precompile**。）

（如果使用可执行文件时，系统提示“权限不足”，那么则要通过命令 chmod +x {文件名} 来提高文件权限至可执行。）

以输出的 -S 这一行为例，pflag 包的 --help **输出**如下：

```
-S, --shardNum int shardNum is an Integer, which indicate
s that how many shards are deployed. (default 4)
```

从左到右依次是：**-S（命令行指令的简称）**，**--shardNum（命令行指令的全称）**，**int（命令行指令的类型）**，**shardNum is an Integer, which indicates that how many shards are deployed.（命令行指令的解释）**，**default 4（命令行指令的默认值）**。

通过 pflag 包的方法，用户便可以快捷地使用 第二类参数。我们将在下一节介绍如何**使用第二类参数来启动节点**。

4.3.4 启动单个节点

在 paramsConfig.json 和 ./ipTable.json 文件中设置了第一类参数之后，用户便可以使用第二类参数**启动节点并指定节点的身份**。

利用 pflag 包内 --help 的输出内容，我们可以在 BlockEmulator 中使用这样一条命令来启动一个 worker 节点：

```
blockEmulator_Windows_Precompile.exe -n 1 -N 4 -s 0 -S 4
```

根据 pflag 包输出的参数解释，可以判断出这条命令启动了这样的一个节点：

- 网络规模：分片总数为 4（-S 4），每个分片 4 个节点（-N 4）。
- 该节点是：序号为 0（-s 0）的分片内的序号为 1（-n 1）的节点。

重复地使用类似的命令启动节点，便可以构建一个分片区块链网络。

4.3.5 启动大量节点

上述的命令仅仅启动了一个节点，而一个分片区块链系统可能存在成千上百个节点，因此**手动地创建每一个节点是不切实际的**。**BlockEmulator 提供了一种批处理启动大量节点的方式**。回忆一下，使用 --help 时，有这样一条输出：

```
-g, --gen isGen is a bool value, which indicates whether
to generate a batch file.
```

所以，只需要在命令行中利用 -g，即可生成批处理脚本，**该批处理脚本能够批量地启动多个节点**。（如果是可执行文件执行该命令，需要再加上 **--shellForExe**，来生成针对可执行文件的批处理脚本）

比如，如果需要生成一个分片数目为 2，每个分片包含 4 个节点的区块链系统，我们可以使用如下命令生成批处理脚本：

```
blockEmulator_Windows_Precompile.exe -g -f -S 2 -N 4
```

运行了此命令之后，会在 block-emulator 根目录下生成一个 windows_exe_run_IpAddr=127_0_0_1.bat 文件，文件的内容如下：

```
<!-- 启动序号为 0 的分片中的非 leader 节点。 -->
start cmd /k blockEmulator_Windows_Precompile.exe -n 0 -N 4 -s 0 -S 2 &
start cmd /k blockEmulator_Windows_Precompile.exe -n 1 -N 4 -s 0 -S 2 &
start cmd /k blockEmulator_Windows_Precompile.exe -n 2 -N 4 -s 0 -S 2 &

<!-- 启动序号为 1 的分片中的非 leader 节点。 -->
start cmd /k blockEmulator_Windows_Precompile.exe -n 3 -N 4 -s 0 -S 2 &
start cmd /k blockEmulator_Windows_Precompile.exe -n 0 -N 4 -s 1 -S 2 &
start cmd /k blockEmulator_Windows_Precompile.exe -n 1 -N 4 -s 1 -S 2 &

<!-- 启动两个分片各自的 leader 节点 -->
start cmd /k blockEmulator_Windows_Precompile.exe -n 2 -N 4 -s 1 -S 2 &
start cmd /k blockEmulator_Windows_Precompile.exe -n 3 -N 4 -s 1 -S 2 &

<!-- 启动 supervisor 节点，由该节点注入交易。 -->
start cmd /k blockEmulator_Windows_Precompile.exe -c -N 4 -S 2 &
```

可见，-g 指令的逻辑很简单，就是**将启动该区块链系统的所有命令输入到了一个.bat 文件中**。用户仅需要双击运行这个文件即可。

Tips：

1. Linux 和 MacOS 系统如果提示“权限不足”，需要使用命令 **chmod +x executable_file 来提高文件权限至可执行**。

2. BlockEmulator 的程序会**自动检测用户的操作系统**。如果用户使用 Windows 的可执行文件在 Windows 系统上运行 -g 命令，那么最终

生成的批处理脚本是 .bat 文件。如果用户使用的是 Linux 和 MacOS 的可执行文件，那么最终生成的批处理脚本是 .sh 文件。

3. BlockEmulator 会根据 ipTable.json 文件中的内容输出若干个批处理脚本，简而言之，**BlockEmulator 将会根据节点的 IP 将命令划分到各个脚本中**。

比如，如果 ipTable.json 的格式如下：

```
{
    "0": {
        "0": "127.0.0.1:32217",
        "1": "127.0.0.1:32227"
    },
    "1": {
        "0": "127.0.0.1:32317",
        "1": "127.0.0.1:32327"
    },
    "2147483647": {
        "0": "10.0.0.7:38800"
    }
}
```

此时，运行 blockEmulator_Windows_Precompile.exe-g-f-S2-N2，则会出现两个.bat 文件：windows_exe_run_IpAddr=127_0_0_1.bat 和 windows_exe_run_IpAddr=10_0_0_7.bat，分别对应两个 IP 地址 127.0.0.1 和 10.0.0.7。

4.3.6 运行 BlockEmulator

尽管上文中，我们各种参数和启动方式做了很多介绍。但对于用户而言，仅需要两步即可启动分片区块链系统。1. 运行 blockEmulator_Windows_Precompile.exe-g-f-S2-N2 生成批处理文件 2. 启动生成的批处理文件（通过命令行或者双击（Windows 系统下））。

图 4.5 便是一个在运行中的 BlockEmulator 实验案例。

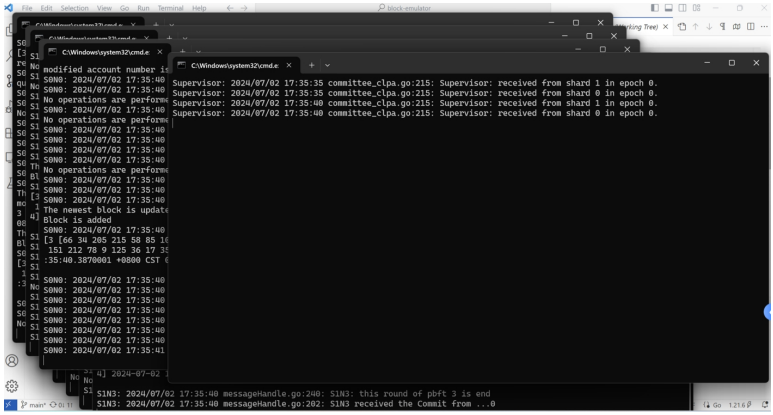


图 4.5: 运行 blockEmulator。

Tips:

1. 在 BlockEmulator 运行中，用户基本无需任何操作，只需等待结果被输入到文件夹即可。

2. 在 Linux 和 MacOS 中，如果运行突发意外，可能导致后台的 BlockEmulator 无法被正确关闭，那么 BlockEmulator 可能会一直运行而不停止。**此时，应该使用 `pkill -9 -f blockEmulator` 命令杀死所有带有 `blockEmulator` 名称的进程**。

4.4 运行结束后的数据收集

4.4.1 实验输出文件说明

BlockEmulator 在运行过程中，节点所输出的内容被统一地归放在 `ExpDataRootDir` （默认是 `expTest`）文件夹中：

```
/expTest
```

```
|-- database
|   |-- chainDB
|   \-- mptDB
|
|-- log
|   |-- S0
|   |-- S1
|   \-- Supervisor.log
|
|-- result
|   |-- pbft_shardNum=2
|   \-- supervisor_measureOutput
```

其中内容包括：

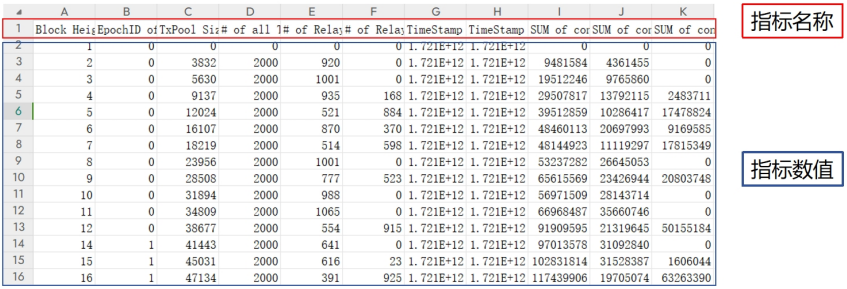
1. database 文件夹，包含了 chainDB（负责存储区块数据）和 mptDB（负责存储状态树数据）。
2. log 文件夹，包含了各个节点的 log 日志。
3. result/supervisor_measureOutput 文件夹，包含了 Supervisor 节点输出的实验指标。
4. result/pbft_shardNum=2 文件夹，包含了 Worker 节点输出的实验指标。

4.4.2 result 文件夹说明

一般来说，用户通常会比较关注 result 文件夹内的输出，因为它给出了较为常用的、有价值的统计数据。

每个分片 leader 节点输出的实验结果

比如图 4.6，其中的数据取自运行完成后的 expTest/result/pbft_shardNum=2/Shard02.csv 文件。

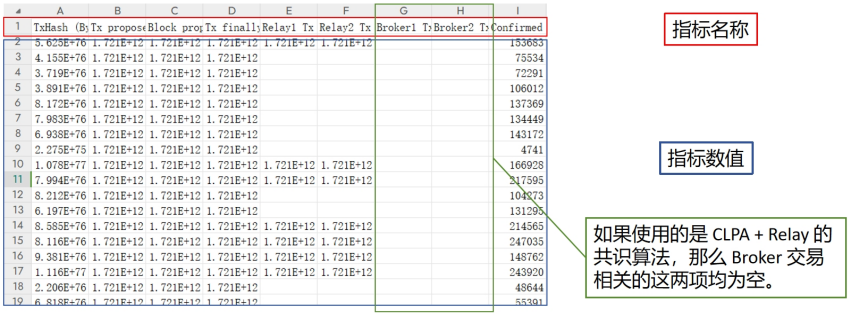


	A	B	C	D	E	F	G	H	I	J	K
1	Block Hei	EpochID of	TxPool Siz	# of all T	# of Relay	# of Relay	TimeStamp	TimeStamp	SUM of cor	SUM of cor	SUM of cor
2	1	0	0	0	0	0	1.721E+12	1.721E+12	0	0	0
3	2	0	3832	2000	920	0	1.721E+12	1.721E+12	9481584	4361455	0
4	3	0	5630	2000	1001	0	1.721E+12	1.721E+12	19512246	9765860	0
5	4	0	9137	2000	935	168	1.721E+12	1.721E+12	29507817	13792115	2483711
6	5	0	12024	2000	521	884	1.721E+12	1.721E+12	39512859	10286417	17478824
7	6	0	16107	2000	870	370	1.721E+12	1.721E+12	48460113	20697993	9169585
8	7	0	18219	2000	514	598	1.721E+12	1.721E+12	48144923	11119297	17815349
9	8	0	23956	2000	1001	0	1.721E+12	1.721E+12	53237282	26645053	0
10	9	0	28508	2000	777	523	1.721E+12	1.721E+12	65615569	23426944	20803748
11	10	0	31894	2000	988	0	1.721E+12	1.721E+12	56971509	28143714	0
12	11	0	34809	2000	1065	0	1.721E+12	1.721E+12	66968487	35660746	0
13	12	0	38677	2000	554	915	1.721E+12	1.721E+12	91909595	21319645	50155184
14	14	1	41443	2000	641	0	1.721E+12	1.721E+12	97013578	31092840	0
15	15	1	45031	2000	616	23	1.721E+12	1.721E+12	102831814	31528387	1606044
16	16	1	47134	2000	391	925	1.721E+12	1.721E+12	117439906	19705074	63263390

图 4.6: 实验结果：工作节点收集到的实验数据。

Supervisor 节点输出的实验结果

比如图 4.7，其中的数据取自程序运行完成后的 `expTest/result/supervisor_measureOutput/Tx_Details.csv` 文件。



	A	B	C	D	E	F	G	H	I
1	TxHash (By	Tx propose	Block proc	Tx finally	Relay1 Tx	Relay2 Tx	Broker1 Tx	Broker2 Tx	Confirmed
2	5.625E+76	1.721E+12	1.721E+12	1.721E+12	1.721E+12	1.721E+12			153683
3	4.155E+76	1.721E+12	1.721E+12	1.721E+12					75534
4	3.719E+76	1.721E+12	1.721E+12	1.721E+12					72291
5	3.891E+76	1.721E+12	1.721E+12	1.721E+12					106012
6	8.172E+76	1.721E+12	1.721E+12	1.721E+12					137369
7	7.983E+76	1.721E+12	1.721E+12	1.721E+12					134449
8	6.938E+76	1.721E+12	1.721E+12	1.721E+12					143172
9	2.275E+75	1.721E+12	1.721E+12	1.721E+12					4741
10	1.078E+77	1.721E+12	1.721E+12	1.721E+12	1.721E+12	1.721E+12			166928
11	7.994E+76	1.721E+12	1.721E+12	1.721E+12	1.721E+12	1.721E+12			217595
12	8.212E+76	1.721E+12	1.721E+12	1.721E+12					104273
13	6.197E+76	1.721E+12	1.721E+12	1.721E+12					131295
14	8.585E+76	1.721E+12	1.721E+12	1.721E+12	1.721E+12	1.721E+12			214568
15	8.116E+76	1.721E+12	1.721E+12	1.721E+12	1.721E+12	1.721E+12			247035
16	9.381E+76	1.721E+12	1.721E+12	1.721E+12	1.721E+12	1.721E+12			148762
17	1.116E+77	1.721E+12	1.721E+12	1.721E+12	1.721E+12	1.721E+12			243920
18	2.206E+76	1.721E+12	1.721E+12	1.721E+12					48644
19	6.818E+76	1.721E+12	1.721E+12	1.721E+12					55391

图 4.7: 实验结果：Supervisor 收集到的实验数据。

4.4.3 使用数据绘制实验图

通过使用 `./figurePlot` 内的画图示例代码，我们可以更好地分析 `./result` 文件夹内的数据：

- `python tx_distribution_relayMethod.py`。该代码可以绘制出不

同类型交易的时延分布图，图 4.8 展示了其效果。

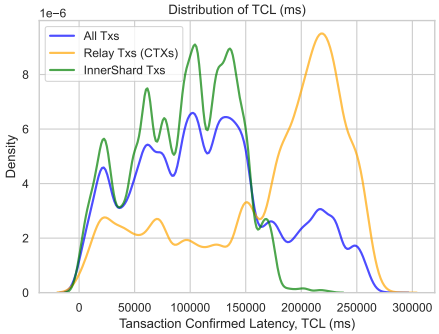


图 4.8: 实验图：不同类型的交易时延的统计。

- `python tx_pool_vary.py`。该代码可以绘制出每个分片交易池随时间的变化情况，图 4.9 展示了其效果。

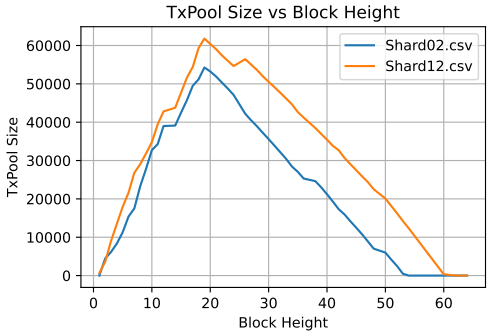


图 4.9: 实验图：交易池的动态变化。

4.5 设计用户自定义的协议

4.5.1 回顾：BE 系统运行机制

我们先回顾在2.2.3章节提到的 BlockEmulator 系统重要概念：

- **监管节点（Supervisor 节点）**：负责读取交易数据，并依据账户与分片的归属表将这些交易分配给对应的分片主节点。
- **分片结构**：每个分片由一个主节点和其他多个工作节点组成。主节点是分片内的中心角色，负责处理交易和运行协议；工作节点则辅助协议的执行。

接下来我们以 `moduleBroker` 机制为例，说明 BE 系统内交易的处理流程：

1. Supervisor 节点从数据集中读取交易，并通过 `MsgSendingControl()` 函数把交易发送给各个分片的主节点，对应代码在`./supervisor/committee_broker.go` 文件。
2. 各个分片的主节点接收到消息，并调用 `handleInjectTx()` 函数进行处理，在该函数中，在该函数中，主节点通过 `pbftNode.CurChain.Txpool.AddTxs2Pool()` 将交易放入交易池，对应代码在`./consensus_shard/pbft_all/pbftOutside_module_Broker` 文件。
3. 主节点在 PBFT 共识的 Propose 阶段从交易池中取出一部分交易打包为区块，并发给本分点的所有节点，对应代码在`./consensus_shard/pbft_all/pbftInside_moduleBroker.go` 文件。

4. 各个节点在 PrePrepare 阶段首先验证传入的预准备消息中的区块是否有效。如果有效，则将其放入请求池中；否则，记录一条无效区块的日志并返回错误，对应代码在`./consensus_shard/pbft_all/pbftInside_moduleBroker.go` 文件。

5. 在 `moduleBroker` 机制中的 Prepare 阶段，PBFT 协议本身就能支持不同节点间的投票结果广播，所以此时节点不需要额外的操作。

6. 节点从请求池中获取对应的区块请求，解码区块，并将其添加到当前链。此外主节点会将跨片交易中继到其他分片，对应代码在`./consensus_shard/pbft_all/pbftInside_moduleBroker.go` 文件。

4.5.2 创建自己的定制化协议

如果读者需要创建自己的协议，可以遵循以下步骤：

1. 实现 Supervisor 节点的四个函数：

   - `HandleBlockInfo()`：用来处理 Supervisor 节点接收到区块后的处理逻辑。
   - `MsgSendingControl()`：用来处理 Supervisor 节点发放交易的逻辑。
   - `HandleOtherMessage()`：用来处理其他消息。
   - Supervisor 的初始化函数。

2. 实现 PBFT 共识节点的七个函数：

   - `HandleinPropose()`：处理 Propose 阶段的逻辑，通常是将交易打包成区块并广播。

- `HandleinPrePrepare()`：处理 PrePrepare 阶段的逻辑，通常是验证区块的合法性。
- `HandleinPrepare()`、`HandleinCommit()`：处理 Prepare 和 Commit 阶段的逻辑。
- `HandleReqestforOldSeq()`、`HandleforSequentialRequest()`：处理旧区块的请求。
- `HandleMessageOutsidePBFT()`：处理非 PBFT 共识相关的消息，消息类型可由用户自定义。

3. 在`./params/static_config.go` 文件的 `CommitteeMethod` 数组中添加自定义协议的名称。
4. 在 `supervisor.go` 文件中添加 Supervisor 节点的初始化函数，并在 `NewPbftNode` 函数中添加自定义节点协议。
5. 在参数配置中将 `ConsensusMethod` 设置为自定义协议的编号。
6. 按照4.3.6 章节的方法运行系统。

BMDS-BPT 是实现了拜占庭容错的流水线出块协议，主要有以下三个特点：

- 选举和出块并行进行。
- 每隔一段时间进行选举，根据权重选举出下一个出块节点。
- 选举完成后，将未来的出块节点和任期先缓存起来。
- 节点如果是本轮出块，则每隔一定时间出块，无需等待广播结果。

- 节点如果是验证节点，则检查接收到的区块合法性并判断出块节点是否在任期内。

接下来，我们将以 BMDS-BPT 协议在 block-emulator 中的具体实现为例，详细介绍如何基于 block-emulator 开发和实现自定义的共识协议。

`HandleinPropose()`：主节点构造一个 `message.Request` 消息，记录当前时间，同时调用 `initWeightOfShard` 函数生成各个节点的权重信息，同时将权重广播给本分片的其他节点。

```
func InitWeightOfShard(nodeNum, shardID uint64) []Weight {
    weights := make([]Weight, nodeNum)
    for i := uint64(0); i < nodeNum; i++ {
        weights[i].Safety = randomFloat(0.5, 1.0)
        weights[i].Network = randomFloat(0.5, 1.0)
        weights[i].Performance = randomFloat(0.5, 1.0)
    }
    if params.DiffBandwidth {
        weights[0].Network = 100
    }
    return weights
}
func Encode(w []Weight) []byte {
    weights, _ := EncodeWeights(w)
    return weights
}
func (c *Committee) HandleinPropose() (bool, *message.Request) {
    r := &message.Request{
        RequestType: message.BlockRequest,
        ReqTime:     time.Now(),
    }
    weightOfShard := InitWeightOfShard(c.NodeNum, c.NodeID)
    r.Msg.Content = Encode(weightOfShard)
    return true, r
}
```

`HandleinPreprepare()`：节点接收到主节点的 Propose 消息，提取其中的权重信息并解码。调用 getChoice 根据权重选择下一个出块节点。

```
func (c *Committee) getChoice(weights []Weight) uint64 {
    if params.FixSelectedNode {
        return uint64(params.FixSelectedNodeID)
```

```
    }
    weigh := make([]Info, 0)
    for i := 0; i < len(weights); i++ {
        w := float64(0)
        w = weights[i].Safety*safety_coefficient +
            weights[i].Network*network_coefficient +
            weights[i].Performance*
            performance_coefficient
        weigh = append(weigh, Info{weight: w, id:
            uint64(i)})
    }
    sort.Slice(weigh, func(i, j int) bool { return
        weigh[i].weight-weigh[j].weight > 0 })
    return weigh[0].id
}
func DecodeB(b []byte) []Weight {
    weights, _ := DecodeWeights(b)
    return weights
}
func (c *Committee) HandleinPrePrepare(ppmsg *message.
    PrePrepare) (bool, []byte) {
    fmt.Println("reveive weight len", len(ppmsg.
        RequestMsg.Msg.Content))
    w := DecodeB(ppmsg.RequestMsg.Msg.Content)
    choice := c.getChoice(w)
    return true, []byte{byte(choice)}
}
```

HandleinPrepare(): 无需操作。

```
func (c *Committee) HandleinPrepare(pmsg
    *message.Prepare) bool {
    return true
}
```

HandleinCommit():

1. 将最终选定的节点 ID 写入 SelectedNodeEachHeight。
2. 判断本节点是否是本轮出块节点。如果不是，直接返回。
3. 如果是本轮出块节点，生成新区块并把交易打包进区块中。
4. 将新区块广播给同分片其他节点。
5. 记录区块生成时间，保存到本地区块池，并将区块加入本地链。

6. 统计并分类区块内的跨分片交易，生成 Snap 消息并广播给对应分片的主节点。

7. 将区块相关信息发送给 Supervisor 节点。

```
func (w *Work) Start(selectNodes []uint64) {
    w.pl.Plog.Printf("choice node %v \n", selectNodes)
    w.BlockPoolLock.Lock()
    defer w.BlockPoolLock.Unlock()
    for {
        b := w.CurChain.CurrentBlock
        hID := int(b.Header.Number)
        selected := -1
        if len(selectNodes)-1 > hID {
            selected = int(selectNodes[hID+1])
        }
        if uint64(selected) != w.NodeID {
            return
        }
        block :=
            w.CurChain.GenerateBlock(int32(w.NodeID))
        if !bytes.Equal(block.Header.ParentBlockHash,
            w.CurChain.CurrentBlock.Hash) {
            w.pl.Plog.Panic("err:", "Err when
                generating block")
        }
        blockId := block.Header.Number
        for _, tx := range block.Body {
            if tx.IsBaojiaoTX{
                tx.BlockId = blockId
                global.TxChanMapLock.Lock()
                cc := global.TxChanMap[tx.TXId]
                global.TxChanMapLock.Unlock()
                if cc != nil {
                    cc<-tx.BlockId
                    close(cc)
                }
            }
            if tx.IsBaojiaoTX2{
                tx.BlockId = blockId
                global.TxChanMapLock.Lock()
                cc := global.TxChanMap[tx.TXId]
                global.TxChanMapLock.Unlock()
                if cc != nil {
                    cc<-tx.BlockId
                    close(cc)
                }
            }
        }
        go w.BroadcastBlock(block)
        w.blockArriveTS[block.Header.Number] =
            time.Now()
```

```
44          w.BlockPool[block.Header.Number] = block
45          w.CurChain.AddBlock(block)
46          interShardTxs := make([]*core.Transaction, 0)
47          relay1Txs := make([]*core.Transaction, 0)
48          relay2Txs := make([]*core.Transaction, 0)
49          RelayPool :=
                make(map[uint64][]*core.Transaction)
50          for _, tx := range block.Body {
51              rsid:=uint64(0)
52              if tx.IsBaojiaoTX {
53                  rsid = w.RunningNode.ShardID
54              }else if tx.IsBaojiaoTX2{
55                  rsid =
                        w.CurChain.Get_PartitionMap(tx.Recipient)
56              }else {
57                  rsid =
                        w.CurChain.Get_PartitionMap(tx.Recipient)
58              }
59              if rsid != w.RunningNode.ShardID {
60                  tx.Relayed = true
61                  _, ok := RelayPool[rsid]
62                  if !ok {
63                      RelayPool[rsid] =
                            make([]*core.Transaction, 0)
64                  }
65                  RelayPool[rsid] =
                        append(RelayPool[rsid], tx)
66                  relay1Txs = append(relay1Txs, tx)
67              } else {
68                  if tx.Relayed {
69                      relay2Txs = append(relay2Txs, tx)
70                  } else {
71                      interShardTxs =
                            append(interShardTxs, tx)
72                  }
73              }
74          }
75          snap := &Snap{
76              ShardID:     w.ShardID,
77              Block:       block,
78              ConfirmTime: time.Now(),
79          }
80          snapByte, _ := json.Marshal(snap)
81          snapMsg := message.MergeMessage(CSnap,
                snapByte)
82          for sid := uint64(0); sid <
                uint64(params.ShardNum); sid++ {
83              go networks.TcpDial(snapMsg,
                    w.ip_nodeTable_original[sid][0])
84          }
85          bim := message.BlockInfoMsg{
86              BlockBodyLength: len(block.Body),
87              InnerShardTxs:   interShardTxs,
88              Epoch:           0,
```

```
            Relay1Txs: relay1Txs,
            Relay2Txs: relay2Txs,
            SenderShardID: w.RunningNode.ShardID,
            ProposeTime:   b.Header.Time,
            CommitTime:    time.Now(),
        }
        bByte, _ := json.Marshal(bim)
        msg_send := message.MergeMessage(message.CBlockInfo, bByte)
        go networks.TcpDial(msg_send, w.ip_nodeTable[params.SupervisorShard][0])
        w.Record(block)
    }
}
func (c *Committee) HandleinCommit(id uint64) bool {
    c.SelectLock.Lock()
    defer c.SelectLock.Unlock()
    for i := 0; i < c.blockPerRound; i++ {
        c.SelectedNodeEachHeight = append(c.SelectedNodeEachHeight, id)
    }
    go c.Worker.Start(c.SelectedNodeEachHeight)
    return true
}
```

4.5.3 消息发送和处理

节点会根据共识机制的规则进行消息的发送和处理。换言之，只需要定义好节点发送和处理消息的逻辑，那么就能让它们按照共识机制运行。为了方便理解，我们给出 block-emulator 中的消息发送示例代码。这段代码展示了如何构造并发送信息给指定节点：

```
// 创造发送消息的结构体实例
sii := message.SeqIDinfo{
    SenderShardID: rbhm.pbftNode.ShardID,
    SenderSeq:     rbhm.pbftNode.sequenceID,
}

// 将实例编码为[]byte数组
sByte, err := json.Marshal(sii)
if err != nil {
    log.Panic(err)
}

// 往消息头中加入消息类型，
```

```
// 并通过TCP发送给i号分片的j号节点
msg_send := message.MergeMessage(message.CSeqIDinfo,
    sByte)
go networks.TcpDial(msg_send,
    rbhm.pbftNode.ip_nodeTable[i][j])
```

接下来，我们给出以下的示例代码来说明如何处理消息：

```
// 假设此时从网络中监听到一条发送给本服务的消息
// 消息数据存储在字节切片 msg 中

// 使用 SplitMessage 方法解析出消息类型和消息内容
msgType, content := message.SplitMessage(msg)

// 根据不同的消息类型进行处理
switch msgType {
case message.CSeqIDinfo:
    // 创建一个结构体实例，用于存储解析后的消息内容
    sii := new(message.SeqIDinfo)

    // 将 JSON 格式的消息内容反序列化到结构体中
    err := json.Unmarshal(content, sii)
    if err != nil {
        log.Panic(err)
    }

    // 此处可以使用 sii 变量继续后续的逻辑处理
    // 例如：更新状态、触发其他操作等
}
```

4.6 手动编译源码运行 BlockEmulator

我们已经在前文介绍了如何使用**预编译的可执行文件**来运行 BlockEmulator。如果用户想采用方式 2（即，**手动编译、运行 BlockEmulator 的源代码**），其实只需要将自己的代码编译为可执行文件，就可以按照上文所述的方式运行。

为了将源代码编译为可执行文件，需要如下操作。

4.6.1 下载和配置 Go 环境

Go 的官方下载网址为：`https://go.dev/dl/` 。进入上面的网站后，选择适合机器操作系统与架构的版本，下载安装即可。

安装完 Go 之后，在 `block-emulator` 目录下，执行 `go get .` 或者 `go mod tidy`，下载运行代码所依赖的包（具体需要哪些包，可以查看 `./go.mod` 文件）。

4.6.2 修改 BlockEmulator 进行二次开发

在安装了 Go 之后，用户便可以开始对 BlockEmulator 进行二次开发了。此时，用户可以通过修改 `.go` 里面的某些文件，按照自己的设计来实现基于 BlockEmulator 的二次开发。

4.6.3 编译 BlockEmulator

仅仅修改 `.go` 里面的文件是无法对 BlockEmulator 的运行结果产生影响的，用户**需要对修改后的代码进行再次编译**。

编译方式如下：

1. 用户可以采用以下命令来生成可执行文件 blockEmulator_Windows_Precompile.exe：

   ```
   go build -o blockEmulator_Windows_Precompile.exe main.go
   ```

   其中 go build 是 Go 语言的编译指令，-o blockEmulator_Windows_Precompile.exe 指定了将代码编译为可执行文件 blockEmulator_Windows_Precompile.exe，main.go 表明该 Go 语言的程序入口为 main.go。

2. 用户还可以使用我们提供的 ./zPrecompileScripts 文件内的编译脚本。它做的事情和上述命令一样，用户只需要选择对应的脚本，然后执行即可。

编译完成后，用户只需要按照**使用提前编译好的可执行文件**的步骤进行执行即可。

Tips: 用户也可以使用类似 go run main.go 的命令来直接编译并启动多个节点。但是 go run 操作进行了编译和运行两个操作，所以，当启动多个节点时，同一份代码需要被编译多次，这是冗余且浪费性能的。我们并**不推荐**这样做。

第五章　BlockEmulator v1.0 的其他分支

本章节主要展示使用了 BlockEmulator 的早期版本做实验的研究组前期两个工作。我们将它们开源在 BlockEmulator 的其他分支中供大家参考。由于 BlockEmulator 的早期版本并不完善，所以不建议用户在此基础上直接使用这两个项目的源码，但是可以借鉴、迁移其中的代码。此外，在章节 6 还介绍了 BlockEmulator 的另一个分支：Broker2Earn [1] 算法的调用方法。

5.1　Fine-tune-lock 分支

5.1.1　分支介绍

Fine-tune-lock 分支[1]来自论文 *Account Migration across Blockchain Shards using Fine-tuned-Lock Mechanism* [5] (INFOCOM'24)。

5.1.2　论文内容

基本思路

在一个分片区块链中（如 BrokerChain），分片负载不均衡会对区块链的性能造成影响。当各分片处理交易的速度一致时，分片负载不均衡会导

[1] `https://github.com/HuangLab-SYSU/block-emulator/tree/Fine-tune-lock`

致一些拥挤的分片中有大量交易在等待被处理，而其他分片则处于空闲状态，这样就没有充分地利用分片技术带来的好处。

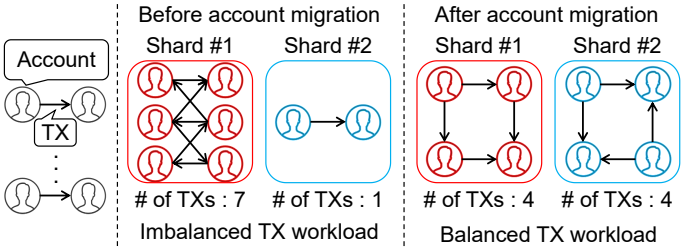


图 5.1: 不同的账户划分方案会在各个分片内产生不同的交易负载。

进行账户重划分是一个较为可行的解决上述挑战的手段。账户重划分是通过图分割算法或社区检测算法，将具有某些社交特征的一批账户迁移到特定分片，目的是降低跨分片交易的比例，同时力求分片内交易负载均衡。通过调研相关工作，我们发现对于分片区块链的账户重划分的研究尚处于初始阶段，目前只发现有一篇论文提出了可行的账户迁移的实现机制。然而在这个机制（论文中称为 SOTA-Lock）中，账户在被迁移的过程中，账户的状态数据会被锁住，涉及到的交易将无法被系统及时处理，因此这些交易的确认时延相对较长。Fine-tuned Lock 是一种新的账户迁移机制，旨在减小账户迁移对关联交易的影响。在 Fine-tuned Lock 机制中，对于正在迁移中的账户，执行扣钱操作的交易（i.e., Payer TX）仍然会被锁住，不能被执行；但收钱方的交易（i.e., Payee TX）可以被执行。采用这种细粒度的锁机制可以减轻账户迁移对其关联交易的影响。

主要贡献

1. 这篇工作提出了一种使用细粒度锁的账户迁移协议(Fine-tuned Lock)，它可以减轻账户迁移对其关联交易的影响。

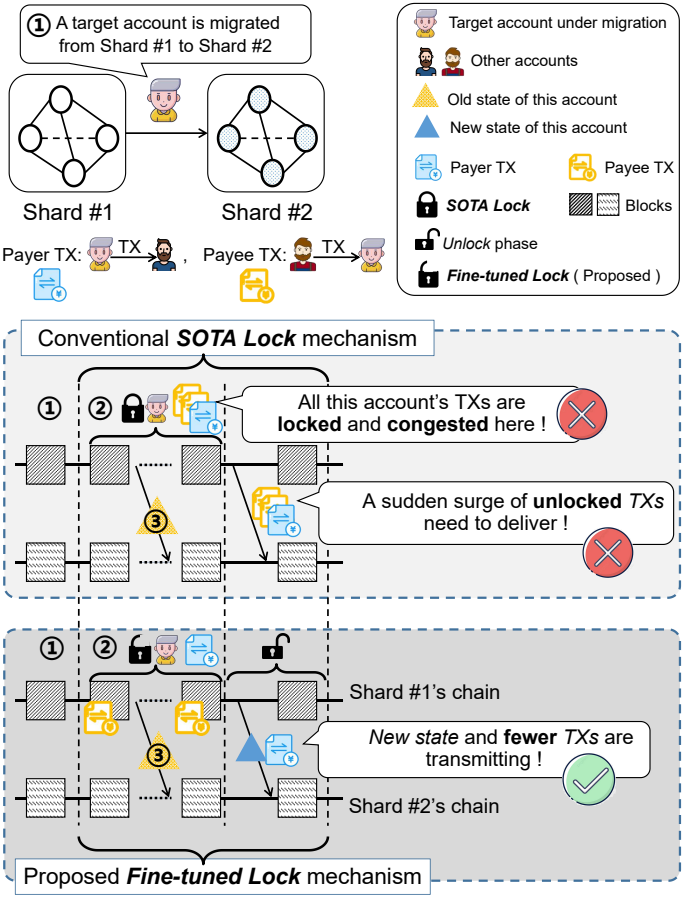


图 5.2: SOTA Lock 与 Fine-tuned Lock 机制之间的对比。①：触发账户迁移，一个目标账户从 shard #1 迁移至 shard #2。②：锁操作开始。③：将被迁移账户的旧状态数据发送到 shard #2。

2. 为了实现账户迁移，这篇工作设计了新的账户状态和区块的数据结构。并且对传统的 relay transaction 机制进行了修改，而且需要解决账户迁移过程中可能出现的安全漏洞，例如针对分片区块链发起的“重放攻击（即论文中的 replay attack）”。

3. 作者在自行开发的区块链实验平台 blockEmulator 上对账户迁移机制 Fine-tuned Lock 做了原型系统的实现，并将其部署在腾讯云服务器中。实验结果表明，在被迁移账户的关联交易的确认时延方面，提出的方法优于 SOTA Lock 方法。

5.1.3 启动此分支

如果想要了解这段 Go 代码，您可以从 `./text/test_shard.go` 这个文件内的函数开始查看。

下面是一些有助于用户启动的提示：

- **数据集**。我们在这个分支中提供了一个数据集（即 `./20W.csv`），用户可以通过命令行的 `-t` 命令指定测试数据集。
- **网络配置**。在运行源码之前，用户可以在 `./params/config.go` 中修改下面这些参数来进行网络配置：
  1. `ClientAddr`。它代表了 *Client* 节点的监听套接字（IP 地址和端口）。*Client* 节点在功能上类似于主分支的 *supervisor* 节点，它主要负责注入交易。
  2. `NodeTable`。这个数据结构定义了各个 *worker* 节点的监听套接字(IP 地址和端口)。它在功能上类似于主分支的 `ipTable.json` 文件。
- **启动节点**。本分支中，启动一个节点的操作与主分支类似，比如：

  ```
  go run main.go -S 4 -f 1 -s S1 -n N1 -t 20W.csv
  ```

  您可以在 `./test/test_shard.go` 中找到它的解释，并且可以通过类似的命令启动其他节点。

5.1.4 其他信息

早期版本。本分支是在早期版本的 BlockEmulator 上建立的，所以代码结构会和主分支内的代码有些差异。而且，早期版本的 BlockEmulator 的功能并不完备，所以本分支并不支持批处理启动节点。在完成这份工作时，我们是通过 Python 脚本和手动部署这两种方式来进行多节点实验的。

省略了某些安全性相关的实现。一些与安全性或验证相关的功能，例如 Merkle 证明和验证，并未在此代码中实现。这是因为，这些功能在其他论文中已有讨论，并非本工作的核心内容。

5.2 tMPT 分支

5.2.1 分支介绍

tMPT 来自论文 *tMPT: Reconfiguration across Blockchain Shards via Trimmed Merkle Patricia Trie* [10]（IWQoS'23）。其源码目前位于 BlockEmulator 的 tMPT 分支[1] 中。

5.2.2 论文内容

在 tMPT 这篇论文中，作者提出了一个分片重组协议（协议的代号是 tMPT），旨在保证分片区块链系统安全性的同时，还可以提高分片重组的效率。本文中，“分片重组 (shard shuffle)” 有时也被成为 “分片重配置 (shard reconfiguration)”。作者为提出的分片重组协议设计了 trimmed Merkle Patricia Trie (tMPT) 数据结构，并运用 tMPT 对分片内的状态树进行压缩，为的是提高分片重组的效率。为了进一步减少分片重组过程

[1] https://github.com/HuangLab-SYSU/block-emulator/tree/tMPT

对区块链系统在 liveness 方面的影响，还进一步提出了一种分片间“部分重组”的方案。作者在 BlockEmulator 上对分片重组方案进行了原型系统实现，并将其部署在云服务器中做实验。实验表明，tMPT 在分片重组效率上优于 SOTA 的数据同步方法；tMPT 的吞吐量比以太坊的“完全同步”（Full Synchronization）的方法高 198%。

论文背景

分片技术是提高区块链可扩展性的一种可行的技术路线 [12, 15, 16, 20]。通过将所有共识节点划分至多个分片中，分片技术可以帮助区块链实现对交易的并行处理。因此，分片技术可以大大提高区块链网络的吞吐率，适用于交易到达速率高的区块链平台。

然而，分片技术的引入将区块链系统的安全性从整个网络分摊至单个分片，因此需要一定的保护机制来保证每个分片的安全性。而分片重组是一个较为可行的增强分片区块链系统安全的手段。如图 5.3 所示，在分片重组的过程中，当共识节点迁移到一个新的分片时，该节点需要同步新分片中的交易或账户的状态等信息，以便能够允许共识节点在新分片中可以开始“干活”：对新分片内部的交易进行验证、参与片内共识等等操作。Elastico [15] 提出定期对各个分片的节点进行定期洗牌，然后将共识节点随机分配给各个分片。RapidChain [20]、Omniledger [12] 等论文则设计了允许分片节点部分同步的方法，以便减少分片重组过程对整个区块链系统的影响。

通过调研现有的区块链分片相关的工作，我们发现对于分片区块链的分片重组的研究尚处于很初始的阶段，尚且缺少一个对分片进行重组的实现方案。本文展示我们提出的一种分片重组方案，旨在减少分片重组所需的时间，同时确保分片区块链系统的安全性。

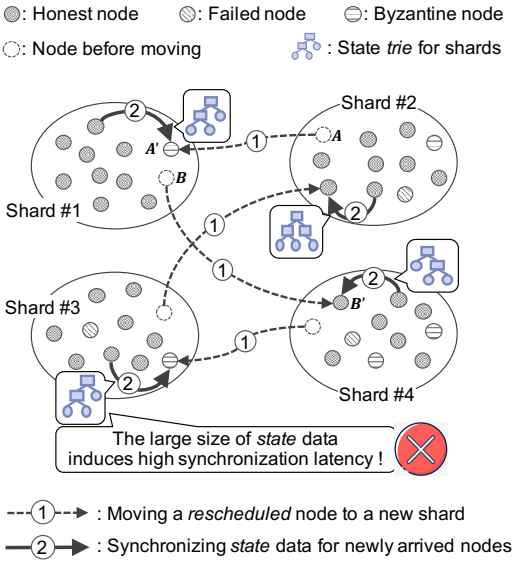


图 5.3: 系统进行分片重组操作时，每个分片内一部分共识节点会被迁移到其他分片。问题是：在对某个节点进行跨分片迁移时，如何支持对新加入节点本地维护的状态数据进行更新。当需要被同步的数据量太大时，数据同步操作会导致分片重组耗时太久。

基本思路

根据 Ethanos [15] 的调查结果，以太坊上的交易存在着“时间局部性”，即部分账户在一周内会进行多次交易。这些活跃账户的状态数据也会在短时间内经历多次更新。受此启发，本文提出的方案通过仅在分片重组时才为共识节点同步活跃账户的状态数据，这样可大大减少分片重组时传输的数据量，从而可提高分片重组的效率。

协议简介

所提出的协议包括两种类型的节点和两种对应类型的分片。

- 验证节点 (Validator node)。验证节点保存其曾参与验证的历史区块的数据和对应的账户状态。通过存储活跃账户状态，验证节点可以进行新区块的交易验证；通过存储历史区块数据，他们可以为用户提供历史区块查询服务。验证者节点组成多个验证分片。

- 见证节点 (Witness node)。见证节点保存全网的账户状态和节点数据，负责生成重组方案并帮助验证节点进行交易验证。见证节点组成见证分片。

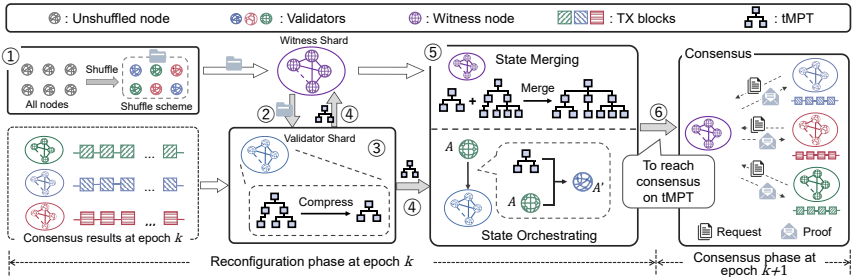


图 5.4: tMPT 协议的运行流程。

如图 5.4 所示，作者将系统运行过程划分为共识阶段与重组阶段。共识阶段中，验证分片执行片内共识，处理交易并出块。重组阶段中，见证分片生成重组方案，协助验证分片进行分片重组，并更新账户状态信息。具体步骤如下。

- 步骤 1：每隔一定的出块间隔，见证分片利用 VRF 随机函数生成一份重组方案，并在分片内对该重组方案进行共识。

- 步骤 2：重组方案被广播至全网，各分片在当前区块共识完毕后进入重组阶段。

- 步骤 3：重组阶段开始。各工作分片分片遍历当前状态树，删去状态树中最近访问周期小于 k 的节点，得到 epoch k 的部分状态树。
- 步骤 4：Epoch k 的部分状态树被发送至重组后该分片的对应的验证节点和见证分片。
- 步骤 5：新收到状态树信息。验证分片收到各分片发来的状态树，并与该分片的历史状态信息进行合并，得到各分片的全局状态信息。
- 步骤 6：验证分片和见证分片均对本分片内更新后的状态信息进行共识。共识完成后，各分片进入共识阶段执行交易验证和出块。

5.2.3 启动此分支

准备工作

安装 Go 及获取 Go 包，在前文已经描述，不在过多赘述。

数据集

我们已经提供了部分针对的以太坊交易数据集 `./len3.csv`，因此无需从其他网站下载数据集。此数据集包含三列，分别为 `from`、`to` 和 `value`。`from` 代表交易的 *付款方*。`to` 代表交易的 *收款方*。`value` 代表交易的 *转账金额*。

网络参数配置

网络参数配置可在 ./params/config.go 文件中找到：

```
var (
    ClientAddr = "127.0.0.1:8200"
    NodeTable  = map[string]map[string]string{
        "S0": {
            "N0": "127.0.0.1:8010",
        },
        // ......
    }
    // ......
)
```

分片重组操作的参数配置

tMPT **分片重组频率**相关的参数，被定义在 pbft/config.go 中：

```
num_block_per_epoch = 100
```

参数 num_block_per_epoch 控制分片重配置的频率。具体而言，当每个分片的区块数量达到 num_block_per_epoch 时，会触发一次分片重配置。完成重配置后，分片区块链系统将进入下一个 Epoch。

提示：如果 num_block_per_epoch 的值较大，重配置的时间间隔将会变长。

常规参数设置

与 *PBFT* 共识相关的一些参数可在 `./params/config.go` 文件中找到。当前分支内置了运行脚本，你可以运行以下脚本启动 21 个节点：

```
# Windows:
run4shard.bat
# Linux/MacOS:
./run4.sh
```

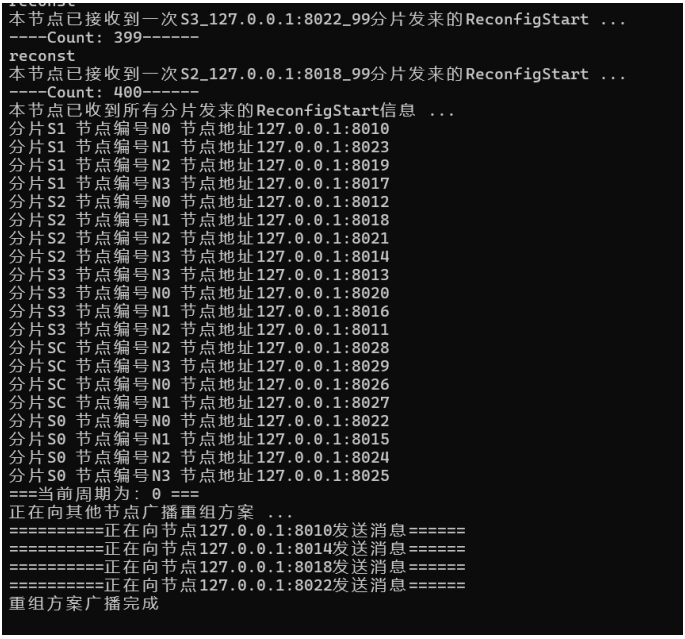


图 5.5: tMPT 运行期间的日志输出截图，该节点启动指令为 `go run main.go -S 4 -f 1 -s SC -n N0`。

如图 5.5 所示，当 SC 节点从工作分片收集到 `shardNumber * num_block_per_epoch` 条消息时，将触发重配置。在此示例中，`shardNumber=4`，`num_block_per_epoch=100`。当收集到 400 条 `ReconfigStart` 消息时，广播重配置消息。

结果收集

重配置的时间和数据成本记录在以下文件中：

- 时间成本：`./log/reconfig_time_per_epoch.csv`
- 数据成本：`./log/S0_reconfig_data.csv`

所有结果均存储在 `./log` 文件夹中。

5.2.4 其他信息

- **早期版本**：本工作基于较早版本的 BlockEmulator 开发，因此代码结构与当前 `main` 分支不一致。
- **省略了一些安全性保障的实现**：一些与安全性或验证相关的功能（如 Merkle 证明和验证）未在此代码中实现。这些功能已在其他论文中讨论，且非本研究的核心内容。

第六章　Broker2Earn 分支

本章节介绍 BlockEmulator 的另一个分支：Broker2Earn [1] 算法说明与调用方法。

6.1　Broker2Earn 工作简介

Broker2Earn（以下简称 B2E）[1] 是发表在 INFOCOM 2024 的一项研究成果。Broker2Earn 协议构建在 BrokerChain [7] 分片区块链之上，旨在为 BrokerChain 区块链提供一个招募 broker 角色的激励机制，同时可以为底层分片区块链内部提供充足的代币流动性。

6.1.1　B2E 协议的研究背景与动机

分片技术是保持区块链去中心化特性同时提高区块链可扩展性的一种可行技术路线。其基本思想是将所有区块链节点划分为若干分片，每个分片并行处理交易，从而大幅提升整个区块链网络的吞吐量，并减少交易确认时延。然而，分片技术也为区块链引入了新的技术挑战：

- **跨分片交易会影响分片链的可扩展性：**在状态分片中（即每个分片维护不同账户的状态），一笔交易所涉及的账户可能分属不同分片，这种交易被称为跨分片交易（cross-shard transactions, CTXs）。跨分片交易的处理往往比片内交易更耗时，因此高比例的跨分片交易会对区块链可扩展性造成影响。

- **激励机制缺失导致分片链性能不高：** BrokerChain 协议 [7] 通过引入“做市商账户（broker account）”来解决跨分片交易的原子性难以保障的难题——Broker 账户通过质押 Token 为分片区块链提供流动性。Broker 账户质押的 Token 可用于将一笔“跨分片交易”巧妙地转化为两笔易于上链的“片内交易”。然而，BrokerChain 协议并未解决一个关键的问题：**分片区块链如何吸引足够多的 volunteer 账户自愿担当 broker 的角色，并主动为分片区块链内部提供代币流动性**。在没有激励机制的情况下，普通用户没有意愿自愿成为 broker 角色。此外，即便用户可以提供代币流动性，另一个技术挑战是：如何有效利用这些流动性来最大化地帮助分片区块链内部处理跨分片交易。

6.1.2 Broker2Earn 与 BrokerChain 的关系

Broker2Earn 协议为 BrokerChain 协议解决了如何招募“中间人（broker）”账户的问题。

- **BrokerChain**（INFOCOM'22）[7]：提出了 broker 机制的基础框架，实现了通过 broker 账户将单笔跨分片交易转化为两笔片内交易的基本功能。

- **Broker2Earn**（INFOCOM'24）[1]：在 BrokerChain 的基础上进一步引入了 broker 角色的激励机制，具体包括 broker 的招募与质押机制、佣金分配机制，以及基于“松弛与舍入”（Relax-and-Rounding）方法的 broker 分配优化算法。

6.1.3 Broker2Earn 协议的主要贡献

- **激励机制设计：**本项工作为分片区块链提出了一种激励机制 Broker2Earn 协议，旨在吸引普通钱包用户成为“做市商”账户。用户可以通过质押 tokens 赚取稳定收益，同时，用户质押到 Broker2Earn 协议的 tokens 可为分片区块链内部提供代币流动性，用于减少分片间的跨分片交易。

- **优化问题建模与算法：**本项工作将 BrokerChain 区块链招募“做市商账户”的问题描述为一个 utility 最大化的问题。优化目标是最大化做市商收益的同时尽量减少分片链内跨分片交易的数量。本项工作证明该最大化问题是 NP-hard 的，并设计了一个基于 Relax-and-Rounding（松弛-舍入）方法的“在线近似算法”，该算法能够在线实时运行。

- **协议实现与实验验证：**我们在开源区块链实验平台 BlockEmulator 中实现了 Broker2Earn 协议，并使用了以太坊的历史交易进行了广泛的性能评估。

6.2 Broker2Earn 协议设计说明

6.2.1 Broker2Earn 协议概述

针对上述挑战，Broker2Earn 协议为 BrokerChain 提供了一个可以吸引普通账户成为“broker 角色”（即，做市商账户）的激励方案。

如图 6.1 所示，Broker2Earn 协议的工作原理流程阐述如下：

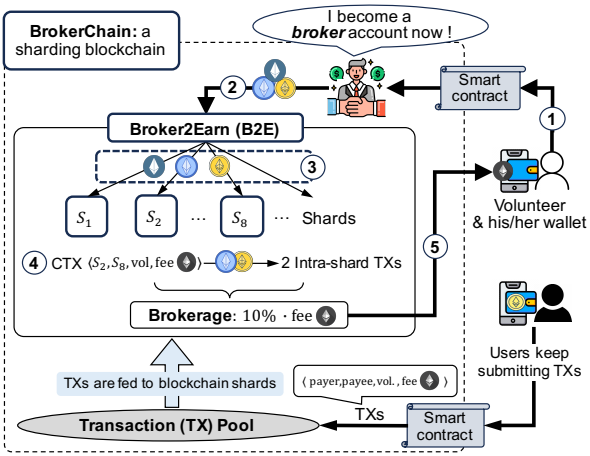


图 6.1: Broker2Earn 协议在 BrokerChain 分片区块链中的工作原理。

1. **注册成为 Broker：**志愿者（Volunteer）通过智能合约（Smart Contract）注册，宣布自己希望成为 broker 账户。

2. **质押 Token：**Broker 账户将一定数量的 Token 质押到智能合约中，质押的 Token 为分片区块链提供流动性（即系统的”运转资本”）。

3. **B2E 算法分配 Broker：**Broker2Earn 算法负责将各 broker 账户分配到不同分片（$S_1, S_2, \ldots, S_n$），以实现最优的流动性分配。

4. **处理跨分片交易：**当一笔跨分片交易 CTX$\langle S_2, S_8, \text{vol}, \text{fee}\rangle$ 到来时，由所在分片的 broker 账户进行中转，将其分解为 2 笔片内交易（intra-shard transactions）。

5. **Broker 获得收益：**成功处理跨分片交易后，broker 账户从交易手续费中获得一定比例的佣金（Brokerage，例如 10% × fee），作为提供流动性的激励回报，并由智能合约自动结算至 broker 账户的钱包。

6.2.2 问题建模

Broker2Earn 所求解的核心问题可形式化描述如下：给定一组志愿者（Volunteers），每个志愿者均可通过质押一定数量的 Token 成为 broker 账户。系统需就以下三方面作出决策：(i) 哪些志愿者被选作 broker 账户、(ii) 各 broker 账户的质押数量、(iii) 各 broker 账户分配至哪些分片。其优化目标包含以下两个方面：

1. **最大化所有 broker 账户的累计收益：**即所有 broker 账户处理跨分片交易后所获佣金之和达到最大；
2. **最大化系统处理跨分片交易的能力：**即尽可能多地将跨分片交易经由 broker 转化为处理效率更高的片内交易，从而降低系统整体的跨分片交易比例。

文献 [1] 已证明该优化问题属于 **NP-hard**，因此无法在多项式时间内获得其精确最优解。

6.2.3 设计“在线近似算法”

为在可接受的计算开销下求解前述 NP-hard 问题，我们为 Broker2Earn 协议设计了一个基于“松弛与舍入”（Relax-and-Rounding）方法的**在线近似算法（Online Approximation Algorithm）**。该算法包含以下三个核心步骤：

1. **松弛（Relax）：**将原整数规划中的 0/1 整数变量松弛为连续变量，从而将原问题转化为一个线性规划（Linear Programming, LP）问题。线性规划可在多项式时间内精确求解，进而得到连续松弛域上的最优解。

2. **舍入（Rounding）：**将线性规划获得的分数解通过随机舍入（Randomized Rounding）方法映射至整数解，并在理论上保证所得整数解满足设定的近似比（Approximation Ratio）界。

3. **在线运行（Online Running）：**算法可随交易流的到达进行实时决策，无需预先获取系统中所有交易的全局信息。

该在线算法在理论上保证了近似最优性，并经过了严格的近似比证明（详见论文 [1]）。

6.3 Broker2Earn 算法的解释

我们上传到 blockEmulator 代码仓库的代码分支命名为 **blockEmulator/broker2earn**，它是由 BrokerChain 的 INFOCOM 2022 会议版代码修改而来，在保留原 broker 机制的基础上，新增了以下四项功能：

1. **阻塞式注入机制（Blocking Injection）：**每批交易注入后，Supervisor 将等待该批次的全部交易在区块链上完成确认，之后再注入下一批次。链上确认计数以 `params.InjectSpeed` 作为阈值：当累计确认数（片内交易数 + 跨分片交易对数）达到此阈值时，当前 epoch 终止并触发下一批次的注入。由此每个 epoch 的边界明确且可控，B2E 算法始终在完整批次的交易范围内进行分配决策，与论文中的理论模型严格一致。

2. **未分配 CTX 重试机制（Retry Pool）：**B2E 算法首次运行时，若某些跨分片交易因 broker 账户余额不足而未能完成分配，这些交易将进入重试池（`restBrokerRawMegPool`）。随着对应 Tx1 完成上链

确认、broker 账户余额逐步恢复，系统会自动对重试池中的 CTX 重新执行 B2E 算法，直至重试池清空或连续 5 秒无新分配为止。

3. **交易金额过滤（Transaction Filtering）：**在读取交易数据集时，对于任一跨分片交易，若其转账金额超过所有 broker 账户的最大可用余额上限（`Init_broker_Balance` × `ShardNum`，约为 2 ETH × 分片数），系统将直接过滤该交易。此类交易超出了任何 broker 的承接能力，若不予过滤将长期占据重试池，进而导致实验无法正常终止。

4. **逐 Epoch 统计记录（Epoch Stats）：**每个 epoch 结束后，系统自动将该 epoch 的 B2E 分配结果、交易统计与 broker 累计收益等数据写入 CSV 文件，以便后续的数据分析与可视化。

6.4 调用 Broker2Earn 算法做实验

本小节介绍如何获取 Broker2Earn（B2E）算法的源代码、如何配置实验参数、以及如何启动实验的具体方法。

6.4.1 B2E 算法的获取方式

获取 B2E 的代码，请访问：`https://github.com/HuangLab-SYSU/block-emulator/tree/broker2earn`

6.4.2 使用 B2E 算法做实验的流程概述

Broker2Earn 实验的完整流程涵盖从源代码获取到结果可视化的全部环节，可划分为**实验前**（搭好环境、写好参数）、**开始实验**（启动节点、自动批处理）、**实验后**（收集 CSV、画图）三个阶段共 7 个步骤，整体流程

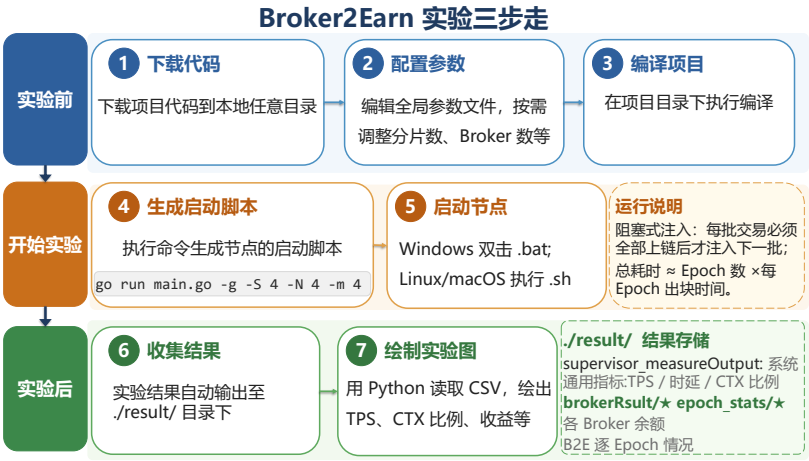


图 6.2: Broker2Earn 实验完整操作流程概览。

如图 6.2 所示。本节将介绍前两个阶段（步骤 1~6），第 6.5 节将以一个使用实验数据绘图的案例来描述实验后阶段。

6.4.3 B2E 代码结构说明

获取并解压源代码后，项目根目录结构如下：

```
blockEmulator-Broker2Earn/
|-- block-emulator/
|   |-- params/
|   |   \-- global_config.go        # 全局参数配置文件
|   |-- supervisor/
|   |   |-- committee/
|   |   |   \-- committee_broker_b2e.go   # B2E Supervisor 核心逻辑
|   |   \-- Broker2Earn/
|   |       |-- B2E.go                    # Relax-and-Rounding 主流程
|   |       \-- URFA.go                   # URFA 线性松弛分配算法
|   |-- data/
|   |   \-- selectedTxs_300K.csv    # 内置以太坊历史交易数据集
|   \-- ipTable.json                # 节点 IP 配置文件
```

6.4.4 B2E 实验参数的配置

Broker2Earn 的系统参数集中定义于 `params/global_config.go` 文件中，以 Go 变量形式直接声明，无需借助额外的 JSON 配置文件。用户在运行实验前，课按需在该配置文件中修改相应变量的值即可。

```
var (
    Block_Interval       = 5000     // 出块间隔（毫秒）
    MaxBlockSize_global  = 2000     //
        每个区块最多包含的交易数
    InjectSpeed          = 5000     // Supervisor
        每轮注入的交易条数（= 每个 epoch 的交易数）
    TotalDataSize        = 500000   //
        实验总交易数（默认 50 万笔）
    BatchSize            = 5000     //
        每批发送量，建议与 InjectSpeed 保持一致
    BrokerNum            = 50       // 系统中 broker
        账户数量（默认 50 个）
    NodesInShard         = 4        //
        每个分片的共识节点数
    ShardNum             = 4        // 分片总数
    IterNum_B2E          = 5        // B2E
        算法每轮迭代次数
    Brokerage            = 0.1      // broker
        佣金比例（0.1 即 10%）
    DataWrite_path       = "./result/"
    FileInput            = "./data/selectedTxs_300K.csv"
)
```

B2E 算法的各参数解释

- **`ShardNum`**：分片总数。
- **`NodesInShard`**：每个分片内的共识节点数。
- **`BrokerNum`**：系统初始参与的 broker 账户数量。程序启动时从 `broker/` 目录下按顺序读取前 `BrokerNum` 个地址作为 broker 账户。
- **`InjectSpeed`**：Supervisor 每轮向系统注入的交易数量，同时作为每个 epoch 的结束阈值。当系统在链上累计确认的交易当量（片内交

易数 + 跨分片交易对数）达到 `InjectSpeed` 时，触发下一批次的注入并进入新的 epoch。

- **`IterNum_B2E`**：B2E 算法在每个 epoch 内执行 URFA 贪心分配的迭代次数。取值越大，分配结果越精细，但相应的计算开销也越高。
- **`Brokerage`**：broker 的佣金比例。例如取值 `0.1` 表示每笔跨分片交易手续费的 10% 归 broker 账户所有。
- **`TotalDataSize`**：参与实验的交易总数。总 epoch 数 = `TotalDataSize` ÷ `InjectSpeed`。
- **`MaxBlockSize_global`**：每个区块所容纳的最大交易数。
- **`Block_Interval`**：出块时间间隔（单位：毫秒）。
- **`FileInput`**：输入交易数据集的文件路径。

用命令行参数 `-m 4` 指定 Broker2Earn 算法

BlockEmulator 内置了多种交易处理算法，包括 CLPA、原版 Broker 与 Relay 等。在终端中通过 `go run main.go` 启动程序时，通过命令行参数 `-m < 编号 >` 指定本次实验所采用的算法；`-m` 参数所有可选取值与对应的算法模式见表 6.1。本节所采用的 Broker2Earn 算法对应编号 `4`，因此后续第 6.4.6 节”编译与启动实验”中用于生成启动脚本及运行节点的命令均会附加 `-m 4` 参数。

6.4.5 节点 IP 配置

若修改了分片总数（`ShardNum`）或每分片节点数（`NodesInShard`），需同步更新根目录下的 `ipTable.json` 文件。以 2 分片、每分片 2 节点的部

表 6.1: 命令行参数 -m 取值与对应算法模式

-m 值	算法模式
0	CLPA_Broker
1	CLPA
2	Broker（原始 BrokerChain，无激励）
3	Relay（默认值）
4	**Broker_b2e (Broker2Earn)**

署为例，文件格式如下：

```
{
    "0": {"0": "127.0.0.1:32217", "1": "127.0.0.1:32227"},
    "1": {"0": "127.0.0.1:32317", "1": "127.0.0.1:32327"},
    "2147483647": {"0": "127.0.0.1:38800"}
}
```

其中，Key "2147483647" 代表 Supervisor 节点的 ID（该数字对应的十六进制为 0x7fffffff）。

6.4.6 编译与启动实验

完成参数配置后，在 block-emulator/ 根目录下按下述步骤启动实验（以 4 分片、每分片 4 节点的部署为例）：

1. **生成批处理启动脚本**。执行如下命令，系统将基于 main.go 即时编译并生成用于启动所有节点的批处理脚本：

   ```
   go run main.go -g -S 4 -N 4 -m 4
   ```

   其中，-S 4 指定分片总数为 4，-N 4 指定每分片节点数为 4，-m 4 选定 Broker2Earn 算法。命令执行完毕后，终端将依次打印"[BlockEmulator] Generating launch scripts: shardNum=4, nodesInShard=4, mode=4

(Broker_b2e)”以及”[BlockEmulator] Done. Batch files…have been written…”两条提示信息（如图 6.3 所示），同时在当前目录下生成两份批处理文件（Windows 平台对应 .bat，Linux/macOS 平台对应 .sh），由用户根据所在平台选择执行。

2. **运行批处理文件以启动所有节点**。

   - Windows 用户：双击或者命令行执行生成的 .bat 文件；
   - Linux/macOS 用户：

   ```
   1 bash run_IpAddr=127_0_0_1.sh
   ```

   批处理脚本会针对每个节点依次执行 go run main.go -n <node-id> -N < 每片节点数 > -s <shard-id> -S < 分片总数 > -m 4，从而拉起所有共识节点与 Supervisor 节点。

3. **等待实验自动结束**。系统将依次注入交易、调用 B2E 算法分配 broker，待当前批次完成链上确认后再注入下一批次；直至 TotalDataSize 笔交易全部处理完毕，自动将实验结果写入 ./result/ 目录。运行过程中 Supervisor 节点的实时日志输出如图 6.4 所示，从中可观察到 PBFT 共识各阶段（pre-prepare、prepare、commit）的消息流转、新区块上链记录，以及主节点向 Supervisor 汇报区块信息的提示行。

```
D:\blockEmulator-Broker2Earn\block-emulator>go run main.go -g -S 4 -N 4 -m 4
[BlockEmulator] Generating launch scripts: shardNum=4, nodesInShard=4, mode=4 (Broker_b2e)
[BlockEmulator] Done. Batch files (.bat for Windows, .sh for Linux/macOS) have been written to the current directory.
```

图 6.3: 生成批处理启动脚本时的终端输出。

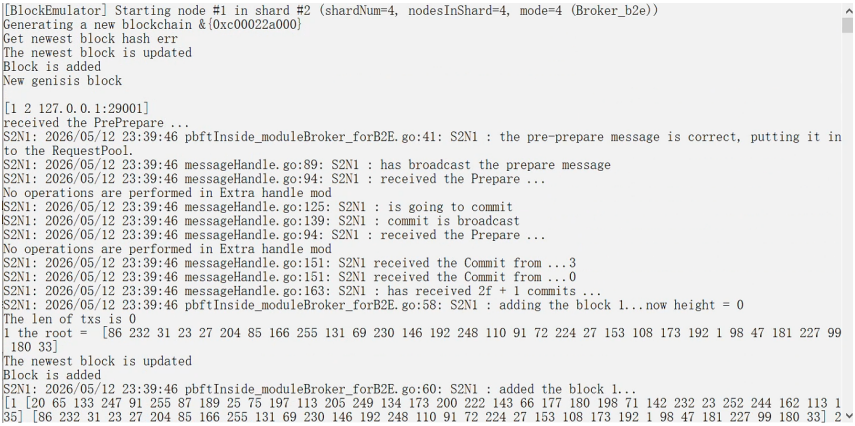


图 6.4: 实验运行过程中节点终端的日志输出。

注意事项：

- 若 Linux/macOS 系统在执行 `.sh` 脚本时提示“权限不足”，可改用 `bash run_*.sh` 直接调用解释器执行，无需为脚本本身赋予可执行权限。

- 若实验异常中断但后台节点进程未自动终止，可在 Linux/macOS 下执行 `pkill -9 -f "go run main.go"` 强制终止所有相关进程；Windows 下可在任务管理器中结束相关 `go.exe` 与节点进程。

- 阻塞式注入机制要求每批交易**全部上链**后方可继续注入，因此实验总耗时与区块链出块速度（`Block_Interval`）直接相关。若希望加快实验进度，可适当减小 `TotalDataSize` 或缩短 `Block_Interval`。

- 程序在读取交易数据时将自动过滤掉转账金额超过 `Init_broker_Balance`× `ShardNum`（即约 2 ETH× 分片数）的跨片交易。这是因为此类交易超出了所有 broker 的承接能力，被过滤后不计入 `TotalDataSize`。

6.5 Broker2Earn 实验案例展示

6.5.1 数据集的说明

Broker2Earn 实验所用数据集来自以太坊主网区块高度 19000000 到 19499999 区间的历史交易记录。该数据集托管于百度网盘[1]。在此基础上，按照本章所采用的分片划分方案进行预处理：剔除片内交易、仅保留跨分片交易，主要关注 broker 机制对跨分片交易的处理效果；剔除智能合约调用类交易（即接收方和发放方为合约地址的交易），仅保留普通账户之间的转账记录；并去除自转账与字段异常等其他无效记录。经上述处理后最终保留 600,000 笔交易，与 Broker2Earn 原始论文 [1] 所采用的实验数据规模保持一致。每条记录包含付款方账户地址（`from`）、收款方账户地址（`to`）以及转账金额（`value`，单位为 `Wei`）三个字段。下载后将 `params/global_config.go` 中的 `FileInput` 指向该文件，并将 `TotalDataSize` 设为 600000 即可。

6.5.2 设置运行 BlockEmulator 的相关系统参数

下面给出一组典型的 Broker2Earn 实验配置，部署规模为 $4 \times 4 + 1$，即 4 个分片、每分片 4 个共识节点，外加 1 个 Supervisor 节点：

```
var (
    Block_Interval       = 5000    // 出块间隔 5 秒
    MaxBlockSize_global  = 2000    // 每块最多 2000
        笔交易
    InjectSpeed          = 5000    // 每 epoch 注入
        5000 笔交易
    TotalDataSize        = 300000  // 共注入 30
        万笔交易（60 个 epoch）
    BatchSize            = 5000
```

[1] Tx-dataset_forB2E_600K-TXs-CHEN-Qinde-2026May12.csv 下载链接：`https://pan.baidu.com/s/1Ap8S2njayTOqTj9lB6yBzw?pwd=1234`

```
    BrokerNum            = 10       // 10 个 broker 账户
    NodesInShard         = 4
    ShardNum             = 4
    IterNum_B2E          = 5
    Brokerage            = 0.1
    DataWrite_path       = "./result/"
    FileInput            = "./selectedTxs_300K.csv"
)
```

在该参数配置下，实验共运行 60 个 epoch（300000 ÷ 5000），系统初始设置 10 个 broker 账户，佣金比例（`Brokerage`）为 10%。此处 `TotalDataSize` 与 `BrokerNum` 为便于演示而采用的缩减值；若采用代码默认参数（`TotalDataSize = 500000`、`BrokerNum = 50`），则实验共运行 100 个 epoch，可更充分地评估激励机制的长期表现。

6.5.3 收集实验结果数据、绘制实验图

实验运行结束后，全部输出的实验结果数据存放于 `./result/` 目录中，主要包括以下三类 CSV 文件：

- **`supervisor_measureOutput/`**：通用性能指标，包括 `Average_TPS.csv`（每 epoch 平均吞吐量）、`Transaction_Confirm_Latency.csv`（交易平均确认时延）以及 `CrossTransaction_ratio.csv`（跨片交易比例）。
- **`brokerResult/`**：每个 broker 账户对应一组文件，分别记录其在各分片中的自由余额（`_brokerBalance.csv`）、锁定余额（`_lockBalance.csv`）以及累计佣金收益（`_Profit.csv`）。
- **`epoch_stats/`**：本版本新增的算法（如：B2E）逐 epoch 的统计文件，文件名形如 `epoch_stats_Broker2Earn_inject5000_shard4.csv`，各列含义见表 6.2。

表 6.2: epoch_stats 文件各列含义

实验结果文档中的每一列	含义
Epoch	epoch 序号（从 0 开始计数）
Tx_injection_speed	本 epoch 内的交易注入速度
CTXs_served_by_Alg	由算法（如 B2E）成功分配给 broker 的跨分片交易数
CTXs_unserved_by_Alg	由算法（如 B2E）未能够成功分配给 broker 的跨分片交易数
CTXs_served_by_Broker	本 epoch 内实际上链的使用 broker 的 CTX 数量（用于校验算法分配的结果 CTXs_served_by_Alg 是否正确上链）
ITX_count_handled_thisEpoch	本 epoch 内上链确认的片内交易数
Total_Profit_ETH	截至本 epoch 全部 broker 的累计收益（单位：ETH）
Active_Broker_Count	当前活跃 broker 账户的数量
Block_Count_thisEpoch	本 epoch 内产生的区块总数

以 epoch_stats/ 文档为例，其中每行记录了当前 epoch 内的实验观测数据。最终这些数据可以被绘制成箱线图，用以呈现各个观测指标数据的分布特征。实验数据主要记录了 3 个指标：1）被算法服务的 CTXs（Cross-shard TXs）；2）未被算法服务的 CTXs；3）以及被算法服务的 ITXs（Intra-shard TXs）的数量。

绘制实验图的 Python 代码如下：

```
import pandas as pd
import matplotlib.pyplot as plt

# 两份 epoch_stats CSV 路径
B2E_CSV = './epoch_stats_Broker2Earn_inject5000_shard4.csv'
BC_CSV  = './epoch_stats_Brokerchain_inject5000_shard4.csv'

df_b2e = pd.read_csv(B2E_CSV)
df_bc  = pd.read_csv(BC_CSV)

# 三个对比指标：列名 -> 显示标题
metrics = [
  ('CTXs_served_by_Alg',   'CTXs served'),
  ('CTXs_unserved_by_Alg', 'CTXs unserved'),
```

```
    ('ITX_count_handled_thisEpoch', 'ITXs served'),
]
# 配色：B2E 浅蓝 + 深蓝边；BrokerChain 浅橙 + 深橙边
palette = [('#A8C8E1', '#2E5F9C'), ('#F2C28C', '#C9701B')]

n = len(metrics)
fig, axes = plt.subplots(1, n, figsize=(3.4 * n, 5.6))

for ax, (col, label) in zip(axes, metrics):
    data = [df_b2e[col].dropna().values, df_bc[col].dropna().values]
    bp = ax.boxplot(
        data, patch_artist=True, widths=0.6,
        medianprops=dict(color='#C8423A', linewidth=2.4),
        flierprops=dict(marker='o', markersize=6, alpha=0.6,
                        markerfacecolor='#888888',
                        markeredgecolor='#888888'),
    )
    for i, (face, edge) in enumerate(palette):
        bp['boxes'][i].set_facecolor(face)
        bp['boxes'][i].set_edgecolor(edge)
        bp['boxes'][i].set_linewidth(2.0)
        for line in (bp['whiskers'][2*i], bp['whiskers'][2*i+1],
                     bp['caps'][2*i],     bp['caps'][2*i+1]):
            line.set_color(edge)
            line.set_linewidth(1.8)
    ax.set_xticks([1, 2])
    ax.set_xticklabels(['B2E', 'BrokerChain'], fontsize=17)
    ax.set_title(label, fontsize=17, fontweight='bold')
    ax.tick_params(axis='y', labelsize=17)
    ax.grid(True, axis='y', linestyle='--', alpha=0.45)

fig.suptitle('B2E vs BrokerChain · Per-Epoch Distribution',
             fontsize=20, fontweight='bold')
plt.tight_layout(rect=[0, 0, 1, 0.91])
plt.savefig('B2E_vs_BrokerChain_boxplot.pdf',
            dpi=300, bbox_inches='tight')
plt.show()
```

B2E 分配算法在各 epoch 上稳定地将大量跨分片交易分配至 broker 账户进行中转处理。由图 6.5 可观察到：其中，如左图所示，B2E 服务的跨分片交易的数目比 BrokerChain 协议服务的跨分片交易高很多；并且，中图所示，B2E 算法把注入的跨分片交易都执行完成了；最后，右图展示

了两个算法服务的片内交易（ITXs，intra-shard TXs）的数量，因为它们使用的是同一份数据，所以两者几乎没有差别。

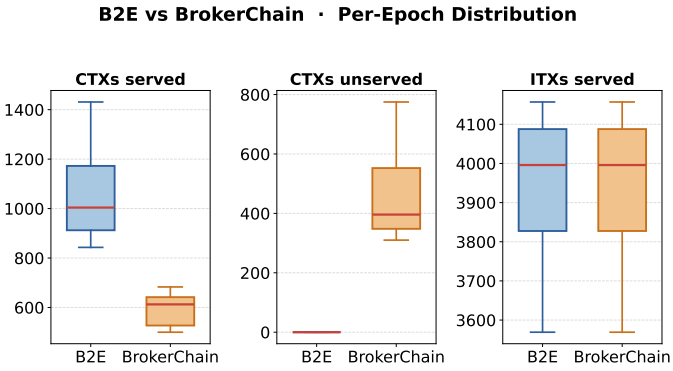


图 6.5: B2E 以及 BrokerChain 的性能表现。其中，CTXs: cross-shard TXs; ITXs: intra-shard TXs。

第七章　BlockEmulator-X：更好用的版本

BlockEmulator-X 封装了区块链系统底层复杂的技术细节；让研究者告别低效、低质且难以获得认可的“自建实验环境”，转而利用被上百篇论文验证过的成熟框架，产出具备国际学术公信力的科研成果。

7.1　BlockEmulator-X 诞生的背景

2025 年下半年到 2026 年初，黄华威教授研究组的叶光同学重构了 BlockEmulator v1.0 的代码，推出了一个进化的版本：BlockEmulator-X [1]（后文某些地方简称为 BE-X）。

比起 BlockEmulator v1.0，进化后的 BlockEmulator-X （即 BlockEmulator v2.0）有如下改进之处：

1. **全新的日志记录：**让用户能清晰追踪程序执行链路，便于调试代码；
2. **更简易的配置方式：**通过配置文件内的注释可直观理解各配置项的定义与功能；
3. **更轻量的程序：**减少了单节点内处理消息的线程数，简化了程序执行逻辑并提升性能；
4. **新功能：**研究人员可以用它执行分片内**智能合约**的测试。

[1] BE-X 的 GitHub 页面：`https://github.com/HuangLab-SYSU/block-emulator-x`

7.2 BlockEmulator-X 的定位与价值

7.2.1 学术定位：区块链实验标准基础设施

- **身份定义：**它是一个专为区块链研究者打造的“一站式”仿真实验工具，目标是成为区块链研究领域事实上的标准实验环境。
- **专注领域：**解决分片区块链中跨分片智能合约调用及复杂交互逻辑的模拟与验证。

7.2.2 BE-X 解决的痛点

当缺乏标准实验工具时，区块链研究人员普遍面临两个层面的挑战：

- **技术复杂度层面：**需手动架设多条仿真链，编写复杂的中继协议，开发门槛极高，且系统极不稳定，容易因底层故障导致实验中断。
- **学术认可层面的“说服力”危机：**用户自建的实验环境往往缺乏公信力。由于实验环境是“私有定制”的，外界难以验证其严谨性，导致实验数据的真实性与结论的说服力经常受到同行评审的质疑。

7.2.3 BE-X 的优势：从“自证清白”到“学界公认”

- **配置替代开发：**研究者无需再做底层开发，只需通过配置文件即可灵活调整区块链参数、完全聚焦于核心算法与逻辑层面的创新。
- **来自学界共同体的权威背书：**BlockEmulator 是一个经过学界检验的成熟实验工作工具，已被上百篇高水平学术论文采用并引用。

- **实验数据的公信力：**使用 BE-X，相当于在“标准实验室”中进行实验。产生的实验结果具有学术透明度和可重复性，能提升研究论文实验结果的权威性和说服力。

7.3 启动使用 BlockEmulator-X

7.3.1 运行一个内置小规模的实验

BE-X 内置了一个默认配置的小规模实验（4 个分片，每个分片 4 个节点，外加一个 Supervisor）。它的启动脚本是 `example_run.sh`。该脚本可以自动下载 package 依赖、删除历史实验数据、并编译代码，示例如下：

```
#!/bin/bash

SHARD_NUM=4
NODE_NUM=4

# Delete the old experiment directory.
rm -rf ./exp/
mkdir -p ./exp/

set -ex

# Download modules and pre-compile.
go mod download
go build ./...

# Start consensus nodes.
for ((i=0; i<SHARD_NUM; i++)); do
  for ((j=0; j<NODE_NUM; j++)); do
    go run cmd/consensusnode/main.go -shard_id="${i}"
        -node_id="${j}" &
  done
done

# Start the supervisor.
go run cmd/supervisor/main.go -shard_id=0x7fffffff
    -node_id=0 &

wait
```

用户可以用 bash 运行此脚本：

```
bash example_run.sh
```

7.3.2 运行用户自定义规模的实验

上述脚本只支持启动默认配置的小规模区块链实验。如果需要运行规模更大或有其他新需求的实验，则需要用户对 BE-X 进行重新配置。

配置 config.yaml

用户可以根据 `config.yaml` 进行配置，每个配置项在该文件中均有详细阐述。

可配置内容包括但不限于：

- 分片数目（`system.shard_num`）
- 每个分片的节点数目（`system.node_num`）
- 共识协议（`system.consensus_type`）
- 注入交易总量（`supervisor.tx_number`）
- 交易注入速度（`supervisor.tx_injection_speed`）

用户修改配置后，再次启动 BE 时，最新配置便能被应用。

配置 IP 表

如果用户修改了默认的分片数目或每分片内的共识节点数目，则需要确保根目录下的 `ip_table.json` 中定义了各个共识节点的 IP 地址。

设置节点 IP 地址的示例（共 2 个分片，每分片内有 2 个共识节点，还有一个 supervisor 节点）：

```
{
  "0": {
    "0": "127.0.0.1:32217",
    "1": "127.0.0.1:32227"
  },
  "1": {
    "0": "127.0.0.1:32317",
    "1": "127.0.0.1:32327"
  },
  "2147483647": {
    "0": "127.0.0.1:38800"
  }
}
```

修改 example_run.sh 脚本

如果修改了 `system.shard_num` 或 `system.node_num`，用户需要在 `example_run.sh` 脚本中修改变量 `SHARD_NUM` 和 `NODE_NUM`，然后重新运行脚本即可。用户如果需要基于 BlockEmulator 进行二次开发，可根据自身需要撰写定制化的脚本。

7.4 BE-X 的系统架构

在 BE-X 中，节点类型包括两种：Supervisor 节点和 ConsensusNode 节点（BE v1.0 中被称为 Worker 节点）。

1. **Supervisor 节点**：负责发送交易、承担委员会（committee）功能和统计整合系统指标的节点。在一组 BlockEmulator 实验中，系统中仅有一个 Supervisor 节点。

2. **ConsensusNode 节点**：负责进行出块的共识节点。分片区块链中有多个分片，每个分片由若干个 ConsensusNode 节点组成。

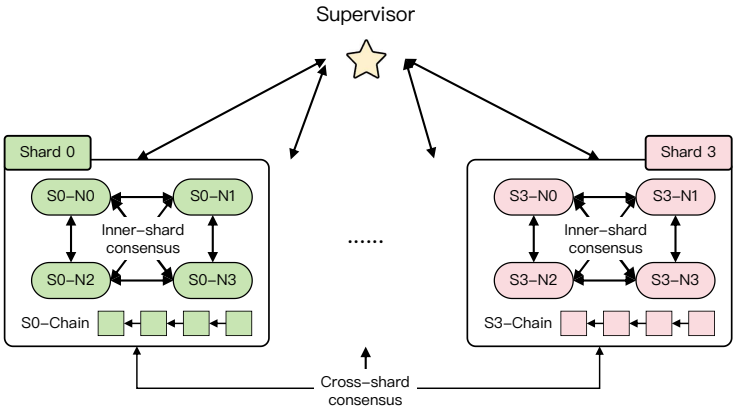


图 7.1: BE-X 的系统架构，图中的部署规模为 $4 \times 4 + 1$（共 4 个分片，每个分片 4 个共识节点，还有一个 supervisor 节点）。

- 同一个分片内的所有 ConsensusNode 节点通过片内共识(inner-shard consensus）对出块操作达成共识，来共同维护着一条片内区块链；
- 不同分片内的 ConsensusNode 节点之间通过跨片共识（cross-shard consensus）传递跨分片消息，可实现“交易中继（transaction relay)”或“账户迁移（account migration)”等功能。

7.5 两类节点的工作流程

7.5.1 Supervisor 节点的工作流程

启动 `cmd/supervisor/main.go` 时，Supervisor 节点的工作流程：

1. **初始化 (Initialization)**：读取命令行参数（加载 `LocalParams`）；读取配置文件（加载 `Config`）。

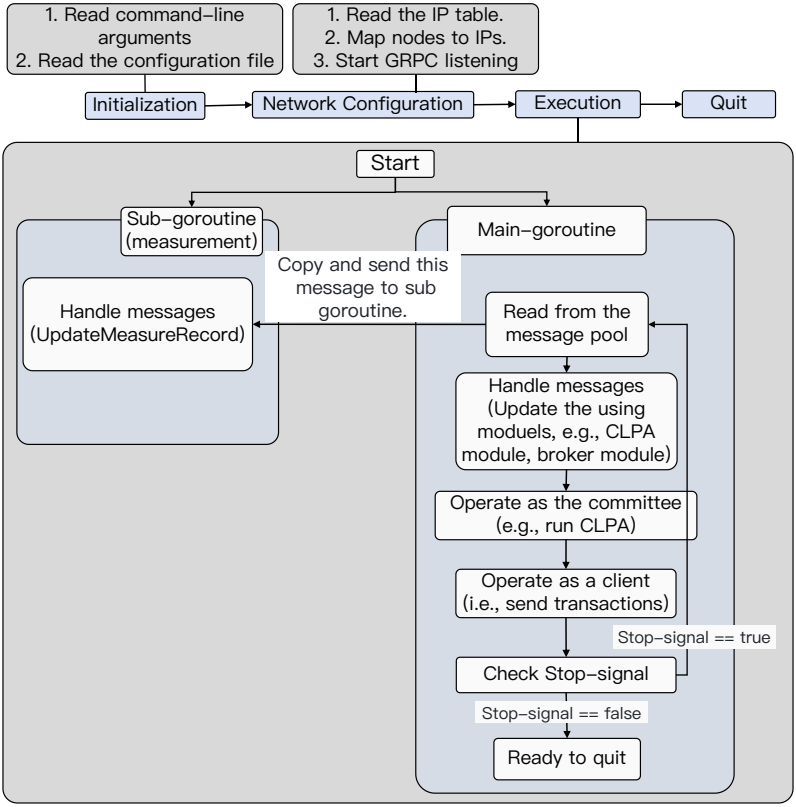


图 7.2: Supervisor 节点在 BE-X 中的工作流程。

2. **网络配置（Network Configuration）**：读取 IP 表；建立节点到 IP 的映射；GRPC 监听，并建立消息池。

3. **执行（Execution）**：Supervisor 执行 `Start()` 函数：

   (a) 启动两个线程，分别是主线程和子线程，子线程用于统计指标，可以与主线程同时执行而无数据冲突；

   (b) 主线程从 GRPC 消息池中读出消息，一份消息会同时被主线程

和子线程消费：i）**主线程**根据使用的组件执行对应的更新操作（如 CLPA 模块、broker 模块）；ii）**子线程**根据采用的 Measure 接口，调用 UpdateMeasureRecord() 函数。

(c) 主线程执行 Committee 相关的操作（如运行 CLPA）；

(d) 主线程执行 Client 操作（即分发交易）；

(e) 检查停止信号（Stop-signal）：

- false：主线程进入下一轮循环；
- true：等待资源关闭，退出；

4. **退出（Quit）**：关闭剩余资源（统计结果文件、网络等）。

7.5.2 ConsensusNode 节点的工作流程

启动 cmd/consensusnode/main.go 时，ConsensusNode 节点的工作流程：

1. **初始化（Initialization）**：读取命令行参数（加载 LocalParams）；读取配置文件（加载 Config）。

2. **网络配置（Network Configuration）**：读取 IP 表；建立节点到 IP 的映射；GRPC 监听，并建立消息池。

3. **执行（Execution）**：ConsensusNode 执行 Start() 函数：

(a) 通过 RegisterHandleFunc() 函数来注册处理函数；

(b) 从 GRPC 消息池中读取消息；

(c) 接口 ShardInsideOp 和 ShardOutsideMsgHandler 处理消息；

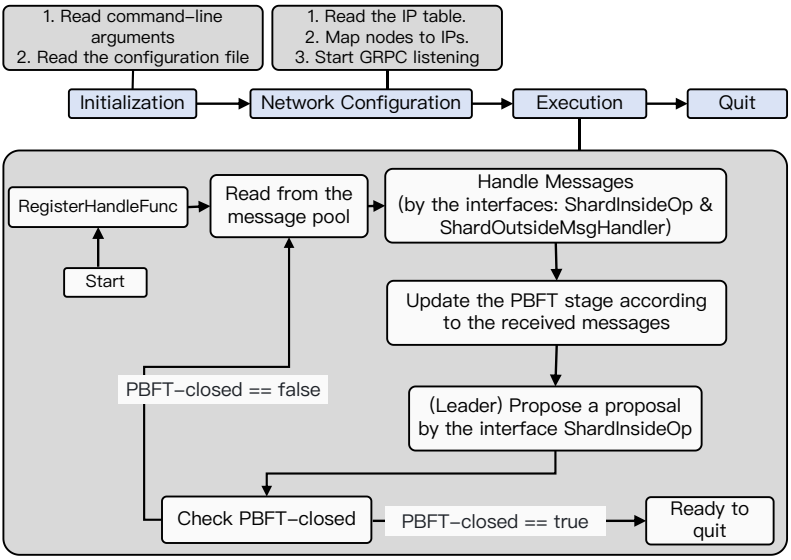


图 7.3: Consensus 节点在 BE-X 中的工作流程。

(d) 根据收到的消息，更新 PBFT 状态；

(e) `Leader` 检查是否达到提出 `Propose` 的条件，根据接口 `ShardInsideOp` 的 `BuildProposal()` 函数创建 `proposal` 并且广播。

4. **退出（Quit）**：关闭剩余资源（区块链、网络等）。

7.6 BlockEmulator v2.0 (BE-X) 的更新内容

7.6.1 参数配置

在 BlockEmulator (简称为 BE) v1.0 和 v2.0 中，系统内的配置项被分为全局配置（global configuration）和本地参数（local parameters）：

- **全局配置**：区块链系统中的所有节点共享的同一份配置，包括分片数目，区块大小等。

- **本地参数**：区块链系统中各节点内不同的配置，包括分片 ID，节点 ID，矿工地址等。

相比于 BE v1.0，BE v2.0 对这些配置项的使用会更加规范：如果某个数据结构需要用到某个配置项，它会在被创建时就获得这些配置参数，而非在运行时从全局变量获取。这样可减少一些意义重复的参数，降低代码的维护成本。

7.6.2 从 yaml 文件中读取全局配置

BE v1.0 读取 json 文件获得全局配置，而 json 文件不支持添加注释。这导致用户在使用 BlockEmulator 时，往往不知道配置项的具体含义。

在 BE v2.0 中，BlockEmulator 在启动时会从 yaml 文件中读取全局配置。因为 yaml 文件支持添加注释，用户就可以便捷地根据自己的需求，修改对应的配置项。BE v2.0 的配置文件是 `config.yaml`，其中包含了所有全局配置项的注解和它们的取值空间。该 `config.yaml` 配置文件示例如下。

```
system:
  ### ----------------------------------------
  ### `shard_num` is the number of shards. It should
      be a positive integer.
  shard_num: 4
  ### ----------------------------------------

  ### ----------------------------------------
  ### `node_num` is the number of nodes per shard. It
      should be a positive integer.
  node_num: 4
  ### ----------------------------------------
```

7.6.3 通过命令行输入本地配置参数

BE v2.0 沿用了 BE v1.0 通过命令行（command-line arguments）的方式来输入系统本地配置参数。用户还可以通过 -h 命令查看所有支持的命令行参数，示例如下。

```
# 查看 ConsensusNode 支持的命令行参数
go run cmd/consensusnode/main.go -h
# 输出结果
#  -account_addr string
#        miner address
#  -config string
#        path to config file (default "config.yaml")
#  -ip_table string
#        path to ip_table.json (default "ip_table.json")
#  -node_id int
#        local node id (default -1)
#  -pprof-port int
#        port to serve pprof; the port should be larger than 5000
#  -shard_id int
#        local shard id, 0x7fffffff denotes the supervisor shard (default -1)

# 查看 Supervisor 支持的命令行参数
go run cmd/supervisor/main.go -h
# 输出结果同上
```

7.7 存储机制

BE v1.0 的存储模式被设计为：区块链上的存储内容包括对“区块数据”的存储和“账户状态”的存储：

- **区块数据的存储**：存储本分片区块链的所有“区块”，并且提供根据区块哈希查找区块的功能；
- **账户状态的存储**：将“账户状态”组织成 MPT 进行存储，并且保留历史快照，可以支持快速回滚。

为了更好地支持 EVM 在 BlockEmulator 上的运行，BE v2.0 对 v1.0 的存储设计模式进行了修改：在保留对“区块数据”存储方式的前提下，将“账户状态的存储”拆分为两棵 MPT，分别用来存储“账户基本状态”和“账户所属分片信息”：

- **区块数据**：本分片区块链的所有区块，并且提供根据区块哈希查找区块的功能。
- **账户基本状态**：账户的基本状态（`Balance`、`Nonce`、`Code` 和 `StorageRoot`），与 ETH 一致。此部分底层使用了 geth 的 `StateDB`，从而提供对 EVM 的支持。`StateDB` 底层将数据组织成 MPT，支持基本的历史快照和回滚功能。
- **账户所属分片信息**：账户所属分片（`ShardLocation`），用于标识账户位于哪个分片。此部分被存储在一个专用的 MPT，以保留历史快照，支持对账户状态的快速“回滚”操作。

7.7.1 对“区块数据”的存储

区块数据的存储在 `pkg/storage/block` 实现，它默认使用的底层数据库是 BoltDB（沿用自 BE v1.0）。其接口实现如下：

```
1  package block
2
3  import (
4      "context"
5  )
6
7  // Store should be a key-value database that stores the
8  // information of blocks.
9  type Store interface {
10     // AddBlock adds a block into the database. It
           contains the operations of
11     // (1) updating the newest blockHash,
12     // (2) adding the header of this block into the
           storage,
```

```
13      // (3) adding the block into the storage.
14      // These 3 operations must be atomic.
15      AddBlock(ctx context.Context, blockHash,
            encodedBlock, encodedBlockHeader []byte) error
16
17      // GetBlockByHash returns the block with the given
            Hash.
18      GetBlockByHash(ctx context.Context, blockHash
            []byte) ([]byte, error)
19
20      // AddBlockHeader adds the header of a block into
            the database. It contains the operations of
21      // (1) updating the newest blockHash,
22      // (2) adding the header of this block into the
            storage.
23      // These 2 operations must be atomic. Please
            distinguish it from AddBlock.
24      // If your storage is limited, AddBlockHeader
            helps you catch up with other nodes quickly
25      // because it reduces the storage required for a
            Block.
26      AddBlockHeader(ctx context.Context, blockHash,
            encodedBlockHeader []byte) error
27
28      // GetBlockHeaderByHash returns the block header
            for the given blockHash.
29      GetBlockHeaderByHash(ctx context.Context,
            blockHash []byte) ([]byte, error)
30
31      // UpdateNewestBlockHash updates the newest
            blockHash.
32      // This function should be called when the
            blockchain wants to rollback.
33      UpdateNewestBlockHash(ctx context.Context,
            newBlockHash []byte) error
34
35      // GetNewestBlockHash returns the latest block
            hash.
36      // Blockchain can quickly find the tail of a chain.
37      GetNewestBlockHash(ctx context.Context) ([]byte,
            error)
38
39      Close() error
40  }
```

7.7.2 对“账户基本状态”的存储

账户“基本状态”的存储复用了 geth 的 StateDB 实现[1]。在 BlockEmulator 中，pkg/storage/vmstate 和 pkg/vm 分别完成了对 StateDB 底层数据库和 StateDB 的包装创建。其相应的代码如下：

```go
// pkg/storage/vmstate/vmstate.go
// NewStateStore creates state.Database for stateDB.
func NewStateStore(cfg config.StorageCfg, lp
    config.LocalParams) (state.Database, error)

// pkg/vm/vm.go
// NewExecutor creates a new executor with given
    parameters.
func NewExecutor(stateStore state.Database, root
    common.Hash, vmChainCfg *params.ChainConfig)
    (*Executor, error) {
    // Init state db.
    stateDB, err := state.New(root, stateStore)
    if err != nil {
        return nil, fmt.Errorf("failed to new a state
            database: %w", err)
    }

    // Set the evmCfg config for evmCfg.
    evmCfg := gethvm.Config{
        ExtraEips: []int{EIP3855},
    }

    return &Executor{
        stateDB:    stateDB,
        vmChainCfg: vmChainCfg,
        evmCfg:     evmCfg,
    }, nil
}
```

需要注意的是：State.Database 仅创建一次，它对应着底层数据库和快照；而 StateDB 则是每次 Commit 之后便不可继续使用，需要重新创建。一般来说，处理每个 Block 时会创建一个 StateDB，当处理完成后 StateDB 会被 Commit；处理下一个 Block 时便需要重新创建一个 StateDB。

[1]geth 的 StateDB 实现：github.com/ethereum/go-ethereum/core/state

7.7.3 对“账户所属分片信息”的存储

对“账户所属分片信息”的存储在 `pkg/storage/trie` 中实现。BlockEmulator 基于 go-ethereum（版本 v1.16.7）的代码实现了此接口。为了确保该接口的通用性，它被设计为 Key-Value 的存储方式。

BE v2.0 将添加账户状态的操作设计为两类操作：

- `MAddKeyValuesAndCommit()`：添加多个 {Key, Value}，返回更新后的状态，并更新数据库；
- `MAddKeyValuesPreview()`：添加多个 {Key, Value}，返回更新后的状态，但不更新数据库，通常用于“验证”/“生成”新区块的阶段。

7.7.4 账户状态存储的接口

```
// Store is an MPT-based, append-only structure whose
    leaf nodes should be considered as accounts.
// The upstream layer of storage not only stores
    nodes, but also provides proofs for the nodes.
type Store interface {
    // GetCurrentRoot returns the root of the trie.
    GetCurrentRoot(ctx context.Context) ([]byte, error)

    // MGetValsByKeys returns the values corresponding
        to the given keys.
    MGetValsByKeys(ctx context.Context, keys [][]byte)
        ([][]byte, error)

    // MAddKeyValuesAndCommit adds the given key-value
        pairs into the trie and commits them into the
        database.
    MAddKeyValuesAndCommit(ctx context.Context, keys,
        values [][]byte) ([]byte, error)

    // MAddKeyValuesPreview adds the given key-value
        pairs into the trie but does not commit them.
    MAddKeyValuesPreview(ctx context.Context, keys,
        values [][]byte) ([]byte, error)

    // SetStateRoot sets the root of the trie.
    SetStateRoot(ctx context.Context, root []byte)
        error
```

```
18
19      // Close closes the database
20      Close() error
21  }
```

7.8 数据结构

BlockEmulator 实现了区块链系统中的基础数据结构（pkg/core），并以此搭建了区块链（pkg/chain）。

7.8.1 账户状态

BE v2.0 的账户状态数据结构需要适配分片区块链系统，它相比于以太坊的数据机构多了一个新字段 ShardLocation，用于标识某个账户所在的分片。

Question：如何确定一个账户所在的分片？

Answer：当账户不存在于数据库中时，可通过**对账户地址取模**的方式，将一个给定的账户归属到某个分片；而当数据库中有这个账户的状态时，以此账户状态的 ShardLocation 字段决定所属分片。

7.8.2 交易

为了适配分片区块链系统中的“跨分片交易”，BE v2.0 对交易的数据结构做了修改，添加了两个 optional settings 参数。交易的新数据结构为：

```
1  type Transaction struct {
2      Sender      account.Address
3      Recipient   account.Address
4      Value       *big.Int
5      Nonce       uint64
6      Signature   Signature
7      CreateTime  time.Time
8      Data         []byte
```

```
9
10      GasLimit uint64
11
12      RelayTxOpt  // the optional setting only for relay
            transactions.
13      BrokerTxOpt // the optional setting only for
            broker transactions.
14  }
```

其中，参数 **RelayTxOpt** 与 **BrokerTxOpt** 的解释如下：

- **RelayTxOpt**：当系统使用 Relay 方式处理跨分片交易时，该结构体会生效，它包含：
  - `RelayStage`：“Relay 交易”的阶段，细分为“未确定”、“正在处理前半部分”和“正在处理后半部分”子阶段；
  - `ROriginalHash`：“Relay 交易”的原始哈希值。如果当前目标交易不是跨分片交易，则为空。
- **BrokerTxOpt**：当系统使用“Broker 账户”的方式处理跨分片交易时，该结构体会生效，它包含：
  - `BrokerStage`：“Broker 交易”的阶段，细分为“非 broker 交易”、“broker1 交易”和“broker2 交易”子阶段；
  - `Broker`：“Broker 账户”的地址；
  - `BOriginalHash`：“Broker 交易”的原始哈希值。如果当前目标交易不是跨分片交易，则为空；
  - 其他论文提及但未具体实现的字段。

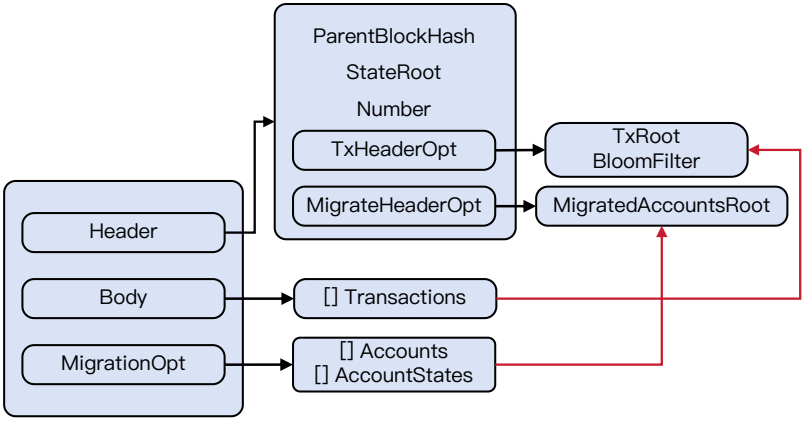


图 7.4: 区块的新数据结构。

7.8.3 区块 (Block)

为了支持分片区块链系统中的账户迁移(account-migration,即将账户由一个分片迁移到另一个分片), BlockEmulator v2.0 对主流区块结构进行了修改, 将区块分为了交易区块 (TxBlock) 和迁移区块 (MigrationBlock):

- **交易区块**: 区块内包含交易, 用于处理交易, 与寻常区块功能一致;
- **迁移区块**: 区块内包含账户状态信息, 专门用于账户迁移阶段。

此外, 为了支持定位账户所属分片, BlockEmulator v2.0 引入了账户所属分片存储。为了记录、查询和回滚此部分存储, Header 中新增了 `LocationRoot` 作为账户所属分片存储的树根。

区块有关的新数据结构:

```
type Header struct {
    ParentBlockHash []byte
    StateRoot       []byte
    Number          uint64
    Miner           account.Address
```

```
6       CreateTime      time.Time
7
8       // LocationRoot is only used in a sharded
            blockchain system to denotes the
9       // root of location trie.
10      // This trie is used to store the account location.
11      LocationRoot []byte
12
13      TxHeaderOpt
14      MigrationHeaderOpt
15  }
16
17  // TxHeaderOpt is the struct for transaction handling.
18  // This struct should be used when this block is a
        normal one (not a block for
19  // account migration).
20  type TxHeaderOpt struct {
21      TxRoot []byte
22      Bloom  bloom.Filter
23  }
24
25  // MigrationHeaderOpt is the struct for the account
        migration.
26  // This struct should be used when this block is an
        account migration one.
27  type MigrationHeaderOpt struct {
28      MigratedAccountsRoot []byte //
            MigratedAccountsRoot is the merkle root of
29      // MigratedAccounts in MigrationOpt.
30  }
31
32  // Body is the struct for transaction handling.
33  // Note that either MigrationOpt or Body is nil.
34  type Body struct {
35      TxList []transaction.Transaction
36  }
37
38  // MigrationOpt is the struct for account migration.
39  // It stores information about accounts to be migrated
        to this shard.
40  // Note that either MigrationOpt or Body is nil.
41  type MigrationOpt struct {
42      MigratedAccounts []account.Address //
            MigratedAccounts is the list of
43      // accounts to be migrated in this stage.
44      MigratedStates   []account.State   //
            MigratedStates is the list of
45      // account states corresponding to accounts in
            MigratedAccounts.
46  }
47
48  type Block struct {
49      Header
50      Body
```

```
51      MigrationOpt
52  }
```

TxHeaderOpt 的内容由 Body 计算得出，MigrationHeaderOpt 由 MigrationOpt 计算得出。

7.8.4 交易池

交易池 (TxPool) 主要提供如下接口：

```
1  // TxPool is a pool that buffers transactions.
2  type TxPool interface {
3      // AddTxs adds the given transactions into the
           pool.
4      AddTxs(txs []transaction.Transaction) error
5
6      // PackTxs pops transactions from the pool.
7      // The size of transactions will be limited by the
           given parameter 'limit'.
8      PackTxs(limit int) ([]transaction.Transaction,
           error)
9
10     // GetTxListSize returns the size of the given tx
           list.
11     GetTxListSize(txs []transaction.Transaction) (int,
           error)
12 }
```

7.8.5 链 (Chain)

前文介绍了 BlockEmulator-X 的存储层及一些基础的数据结构设计。在此基础上，Chain 的新数据结构设计如下：

```
1  // Chain describes a blockchain.
2  type Chain struct {
3      s          *storage.Storage // the storage for both
           block-storage and trie-storage.
4      curHeader block.Header      // the current header
           in this blockchain.
5      shardID    int64
6      epochID    int64
7
8      cfg config.BlockchainCfg
```

```
9
10      mux sync.Mutex
11  }
```

Chain 结构的公开函数较多，在此不多赘述。需要注意的是：访问公开函数需要加锁，访问内部函数一般不加锁。

7.9 网络与通信

BlockEmulator v1.0 在进行消息发送时，先从全局 IP 表中读取指定节点的 IP 地址，再调用函数 `TCPDial` 发送消息。这种形式限制了消息发送的二次开发（强制需要 IP 地址），且缺乏消息缓存能力。

而 BlockEmulator v2.0 对网络模块进行了改造，为此定义了节点信息数据结构 `NodeInfo`（`pkg/nodetopo`）：

```
1  type NodeInfo struct {
2      NodeID, ShardID int64
3  }
```

通过网络层的封装，程序可以通过节点的 {`NodeID`, `ShardID`} 而非直接通过 IP 来发送消息。

7.9.1 连接与通信

BlockEmulator v2.0 的网络模块在发送消息时，仅需提供目标节点的 `NodeInfo`，而无需显式指定具体的网络地址。因此，接口中的消息发送函数 `SendMsg2Dest` 以 `NodeInfo` 作为其中的参数来实现消息投递。负责管理连接与通信的网络连接接口 `P2PConn` 定义如下。

```
1  // P2PConn is a peer-to-peer connection that should
       contain a message buffer.
2  type P2PConn interface {
```

```
    // ListenStart starts listening for messages from
        other nodes as a server.
    ListenStart() error

    // DrainMsgBuffer drains (reads all and pops)
        messages in the buffer.
    DrainMsgBuffer() []*rpcserver.WrappedMsg

    // SendMsg2Dest sends the given message to the
        given dest node.
    SendMsg2Dest(ctx context.Context, dest
        nodetopo.NodeInfo, msg *rpcserver.WrappedMsg)

    Close()
}
```

P2PConn 需要在数据结构内部实现一个消息缓冲区（buffer），用于暂存消息。其执行流程如下：

1. 程序用一个子线程启动 ListenStart() 监听来自其他节点的消息，并将收到的消息加入消息缓冲区；
2. 上层代码需要读取消息缓冲区时，调用 DrainMsgBuffer() 读取缓冲区中的所有消息；
3. 需要对某个节点发送消息时，调用 SendMsg2Dest()，根据给定的 NodeInfo 将消息发送到指定节点。

由于区块链中广播等操作较为常用，BlockEmulator v2.0 将这些操作封装在结构体 ConnHandler 中：

- **MSendDifferentMessages()**：根据给定参数 node2Msg，发送不同消息给不同节点；
- **GroupBroadcastMessage()**：广播同样消息给 group 数组内的节点。

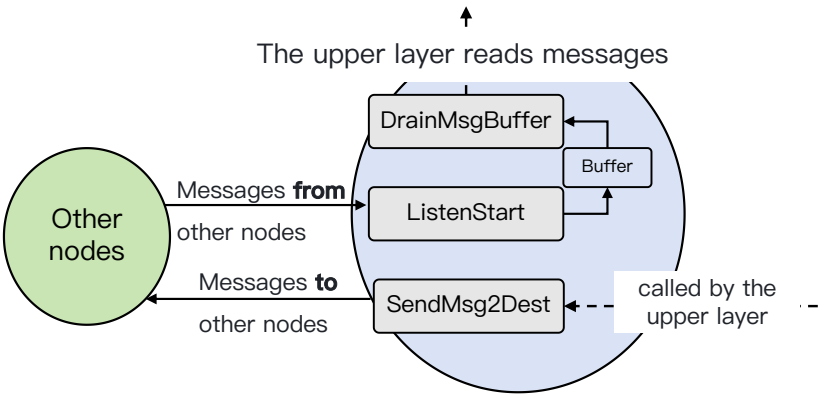


图 7.5: 消息缓冲区的设计。

另一方面，结构体 ConnHandler 实现广播等操作的定义如下：

```
type ConnHandler struct {
    P2PConn
}

func (p *ConnHandler) MSendDifferentMessages(ctx
    context.Context, node2Msg
    map[nodetopo.NodeInfo]*rpcserver.WrappedMsg)
func (p *ConnHandler) GroupBroadcastMessage(ctx
    context.Context, group []nodetopo.NodeInfo, msg
    *rpcserver.WrappedMsg)
```

7.9.2 消息

BE v2.0 使用 protobuf（https://protobuf.dev/）生成 RPC 连接相关代码，并以生成的 WrappedMsg 结构体作为所有消息传播载体。具体解释如下。

WrappedMsg 数据结构

```
type WrappedMsg struct {
    state         protoimpl.MessageState
        `protogen:"open.v1"`
    MsgType       string
        `protobuf:"bytes,1,opt,name=msgType,proto3"
        json:"msgType,omitempty"`
    Payload       []byte
        `protobuf:"bytes,2,opt,name=payload,proto3"
        json:"msgType,omitempty"`
    unknownFields protoimpl.UnknownFields
    sizeCache     protoimpl.SizeCache
}
```

主要关注两个字段：

- **MsgType**：消息类型，用于告诉上层程序应该将此 WrappedMsg 解码为何种类型；
- **Payload**：消息负载，将各种类型的消息编码得到的字节序列。

节点处理消息时，如图 7.6 所示，需要先根据 MsgType 将 Payload 解码为正确类型，再进行处理；节点发送消息时，需要将消息编码为字节序列 Payload，并填写对应 MsgType，再发送。

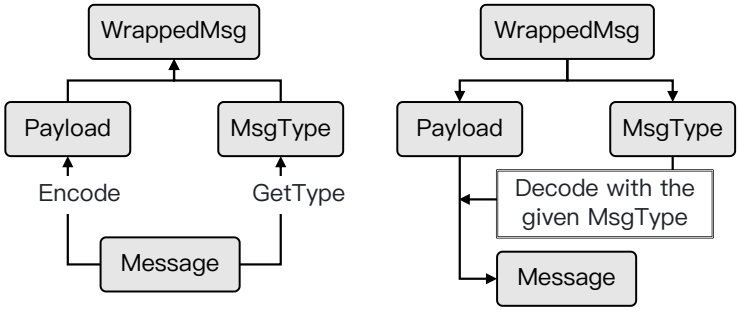


图 7.6: WrapMsg 的编码与解码逻辑。

在 BlockEmulator v2.0 中，将 `WrappedMsg` 解码为对应消息的操作暂不支持，因为 Go 语言实现该接口需要使用断言，不够优美。但 `pkg/message` 中包含了将消息包装为 `WrappedMsg` 的操作，阐述如下：

将消息包装为 `WrappedMsg` 的操作

```go
// WrapMsg encodes different types of messages.
func WrapMsg(msg any) (*rpcserver.WrappedMsg, error) {
    msgType, err := getMsgType(msg)
    if err != nil {
        return nil, fmt.Errorf("getMsgType failed: %w",
            err)
    }

    var buf bytes.Buffer
    encoder := gob.NewEncoder(&buf)

    if err = encoder.Encode(msg); err != nil {
        return nil, fmt.Errorf("encoder failed: %w",
            err)
    }

    return &rpcserver.WrappedMsg{
        MsgType: msgType,
        Payload: buf.Bytes(),
    }, nil
}

func getMsgType(msg any) (string, error)
```

7.9.3 对 libp2p 网络协议的支持

在 BlockEmulator v1.0 中，所有节点均维护一份相同的 IP 表（即 `ip_table.json`）。节点通过该表获取自身监听端口，并据此确定其他节点的网络地址。

这种基于 IP 表的网络模式要求所有节点能够通过 `IP:PORT` 彼此直连。然而在真实复杂的网络环境中，部分节点可能处于 NAT 之后，通过内网地址访问外部网络，并不具备可被公网直接访问的 IP 地址，因此其

网络地址无法被写入 IP 表，从而导致节点间无法建立连接。

因此，BlockEmulator v2.0 在全新的网络层设计中引入并实现了 libp2p 网络协议。该协议无需依赖预先配置的 IP 表即可完成节点发现与通信，并能够支持位于 NAT 之后的节点加入 BlockEmulator 实验环境。

若要启用 libp2p 网络协议，只需要将 `config.yaml` 中的 `network.communication_mode` 设置为 `libp2p` 即可。

启用 libp2p 时，BlockEmulator 实验不再依赖 IP 表，此时 Supervisor 节点将兼任 libp2p 中的 bootstrap 节点，负责为新加入的节点提供入网入口，帮助其快速发现并连接到网络中的其他节点。

libp2p 模式下 ConsensusNode 节点将依次执行如下操作：

1. **启动监听：**在本地监听某个端口，并生成自己的 PeerID（身份标识）；
2. **发现并连接其他节点：**通过 `bootstrap` 节点或 DHT 等机制获取其他 peer 信息，然后主动建立连接；
3. **通过连接进行协议通信：**连接建立后，通过 stream，按 protocol ID 发送和接收消息。

7.10 节点执行

如图 7.7 所示，BlockEmulator v1.0 使用消息驱动模式进行共识：当节点收到消息时，会马上启动一个线程（Go routine）对其进行处理。线程启动后，如果程序判断此时不是处理此消息的时机，则该线程会睡眠并等待被唤醒。

在 BlockEmulator v2.0 中，当节点收到消息时，会先将消息放入消息缓冲池（`Message buffer pool`），节点循环地从缓冲池中取出消息，进行

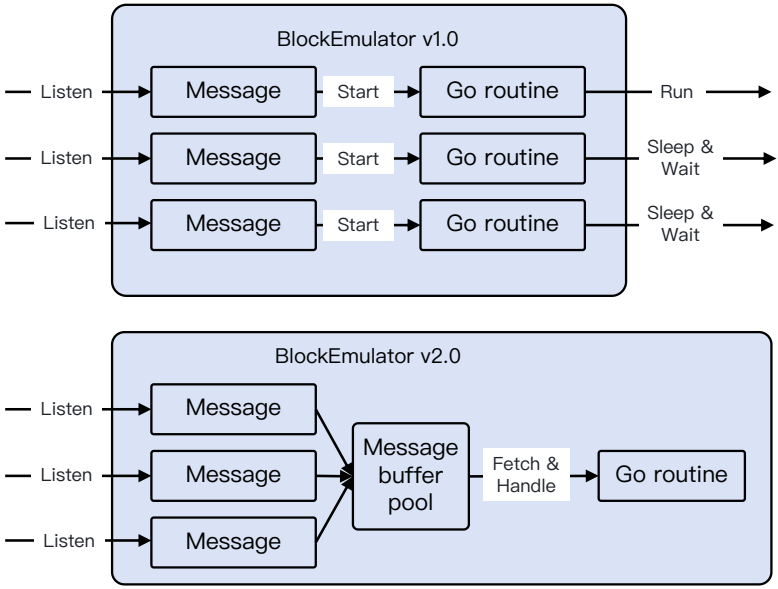


图 7.7: BE v1.0 与 BE v2.0 对节点执行中的消息处理逻辑对比。

有序处理。相比于 v1.0，BE v2.0 仅有一个线程（Go routine）将消息从消息缓冲池中取出，而后依次处理消息，使得消息串行处理，避免多线程竞争导致的数据并发问题。

7.11 日志功能

BlockEmulator v1.0 中，日志的输出内容混杂了 `fmt.Print` 和 `consensus_shard/pbft_all/pbft_log`（或 `supervisor/supervisor_log`）。这导致了日志的输出比较混乱，并且仅有共识层的日志才能输出到文件。

同时，BE v1.0 对于错误信息简单粗暴地使用 `log.Panic()` 直接退出程序。这可能会导致一些无关紧要的异常造成实验被终止。

BE v2.0 使用 Go 官方标准库 log/slog 打印日志，其中规定了 4 个日志等级 (Debug, Info, Warn, Error)。日志的输出方式定义在 pkg/logger 中。用户可以根据错误或异常的具体情况进行不同级别的打印。设置较高等级时，等级较低的日志输出会被屏蔽，例如设置 Level = Info 时，Debug 类型的日志不会被输出，但 Info，Warn，Error 都会被输出。

```
1 func main() {
2     slog.Debug("", "", xxx)    //
          基本上仅在开发阶段输出这些内容，用于 Debug
3     slog.Info("", "", xxx)     //
          一些基本日志信息，不是错误，用 Info
4     slog.Warn("", "", xxx)     // 使用 Warn
          输出一些无伤大雅的异常
5     slog.Error("", "", xxx)    // 发生了严重错误时，用
          Error 输出
6 }
```

第八章　用 BE-X 定制化测试智能合约

8.1　测试智能合约的使用场景

用户（测试者）使用 BlockEmulator-X 测试智能合约执行情况的目的可能包括但不限于：**智能合约的功能性测试**与 **交易顺序敏感性测试**。

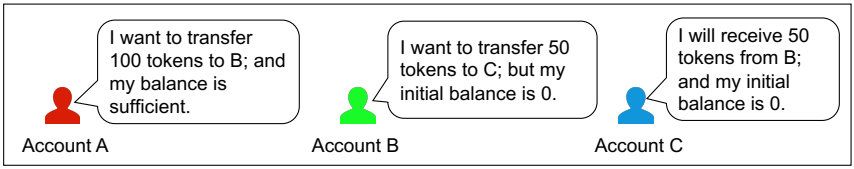
8.1.1　智能合约功能性测试

1. **智能合约中单个函数的正确性测试：**针对单个函数进行验证，确保其逻辑正确。例如，用户可测试调用函数 `transfer(address to, uint256 amount)` 后，对应账户的余额是否增减。

2. **智能合约状态读写一致性测试：**验证智能合约的状态在多次读写后的正确性。例如：同一账户连续多次调用 `set(uint x)` 函数后，旧值是否被覆盖；以及不同账户调用智能合约对同一个账户的操作是否影响该账户的状态等。

8.1.2　交易顺序敏感性测试

所谓“交易顺序敏感性测试”是指：用户在给定不同的交易顺序的前提下，可以测试交易顺序对最终执行结果的影响。以测试符合以太坊 ERC20 标准的智能合约代码为例，图8.1 展示了一个测试 “交易顺序敏感性” 的场景。用户 B 想调用合约中的 `transfer()` 函数向用户 C 转 50 个代币。但是用户 B 初始的余额为 0，无法支持这笔转账。那么，用户 B 必须等待用



Tx ID	SC caller	Function called	**Result**
tx1	A	transfer(B,100)	Sucess
tx2	B	transfer(C,50)	Sucess
tx3	C	balanceof()	50

Tx ID	SC caller	Function called	**Result**
tx2	B	transfer(C,50)	Revert
tx1	A	transfer(B,100)	Sucess
tx3	C	balanceof()	0

图 8.1: “交易顺序敏感性”的案例示例。图中 SC 代表 Smart Contract。

户 A 调用合约中的 `transfer()` 函数向用户 B 转 100 个代币之后，才能执行向用户 C 的转账操作。图8.1 中表格中展示的是智能合约调用顺序以及智能合约执行后得到的结果。左下方的表格展示的是正确的交易执行顺序的案例：最终用户 B 成功地调用了智能合约中的 `transfer()` 函数向用户 C 转了 50 个代币；并且用户 C 通过调用智能合约中的 `balanceof()` 函数获取到了账户中的余额。而右下方表格展示的是一个错误的交易执行顺序造成的结果：首先执行 tx2，则会由于用户 B 的余额不足无法执行该交易而触发以太坊的 `Revert` 机制，从而导致该交易无法被执行；而后续的交易（tx1 与 tx3）将被正常地执行，但最终用户 C 的余额并不是预期中的 50，而是 0。

以上案例是为了说明 BlockEmulator-X 是如何帮助用户测试指定交易顺序的场景。用户完全可以自己提供交易数据集、并指定交易执行的顺序，并最终把交易集传入交易池后，强制这些交易按照指定的顺序来执行。

由于 BlockEmulator-X 的这个特性，用户可以测试交易顺序的敏感性，从而可推断出针对某个目标智能合约被交易调用的较优顺序。

相较于 BlockEmulator-X，其他的区块链实验平台（例如 Anvil, Ganache

等）则是通过脚本事先将交易集传到交易池中，然后矿工再从交易池中按照一定的规则，从交易池中选取一些交易并打包成区块。这会导致用户并不清楚自己调用某个智能合约的交易，真正被执行的顺序：即，用户对调用智能合约的交易最终执行结果是不确定的。除此之外，还可能会出现用户进行了多次实验后，却每次得到的结果均不相同的情况。

8.2 使用 BE-X 测试智能合约的优势

- **交易按照确定性顺序执行：** BE-X 会按照用户输入数据集中指定的交易顺序来执行。这可以保证最终执行结果的确定性。即，在相同的配置和相同输入数据集的情况下，多次实验得出的结果是一致的。
- **参数配置可控：** BE-X 会根据用户设置的各种系统参数（例如，出块间隔、区块大小等信息）来运行实验。
- **实验需求自定义：** BE-X 允许用户灵活更改数据集的输入方式、智能合约类型、以及测试逻辑，以适配用户自定义的实验需求。

8.3 测试步骤概述

本小节先概述用户（实验者）使用 BlockEmulator-X 测试智能合约的实验流程。后续章节将详细展开实验流程的每一个步骤。图8.2展示了用户使用 BE-X 测试智能合约的主要流程。

8.3.1 Step1：编写并编译智能合约

智能合约的编写需要用户使用 Solidity 语言进行编写，用户可以使用以下两种方式进行智能合约的编写：

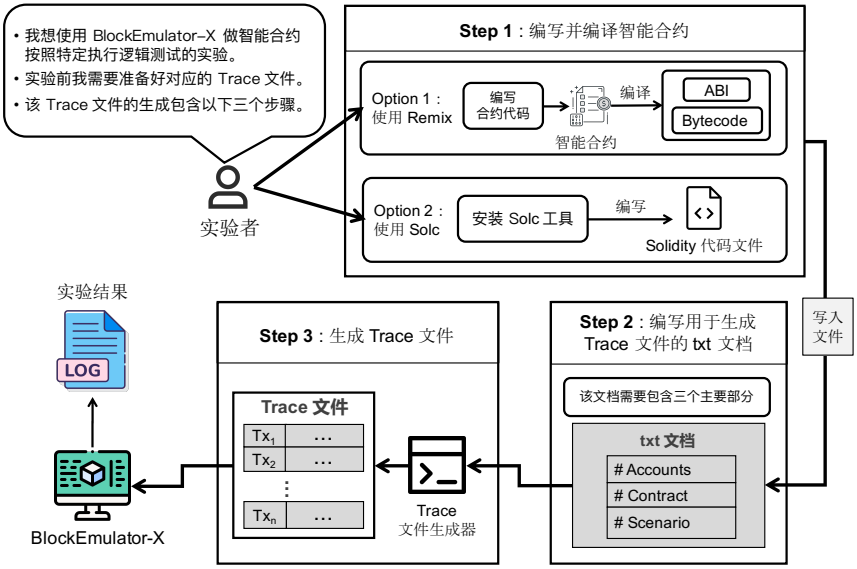


图 8.2: 使用 BlockEmulator-X 进行智能合约测试的流程概览图。

1. **Option 1: 使用 Remix IDE：** Remix 是以太坊官方，一个基于浏览器的 Solidity 集成开发环境，无需本地安装即可使用，适合进行合约的快速编写、编译与调试。

2. **Option 2: 使用 Solc 工具：** Solc 是 Solidity 官方提供的智能合约编译器，用于将 Solidity 源代码编译成 EVM 可执行的字节码和 ABI 等输出文件。使用该方式需要用户提前在自己机器中安装好 Solc。

8.3.2 Step2 & Step3：生成 “Trace 文件”

由于 BlockEmulator-X 是由交易驱动的，用户测试自定义智能合约时需要事先准备好相关交易数据集，该交易数据集中需要包含部署以及调用智能合约时的交易，但是用户手动填写该交易数据集中的相关字段的操作

比较复杂。为此，我们在 BlockEmulator-X 中集成了一个通过用户自定义的 txt 文档和 sol 代码生成相应 Trace 文件的工具，有关该 txt 文档的详细说明将在章节8.5.1展开；而 Trace 文件生成器的使用方法也将在章节8.5详细说明。

8.3.3 最终：运行实验、查看实验结果

使用 BlockEmulator-X 进行智能合约相关实验的操作与章节7.3中提到的启动 BlockEmulator-X 进行实验的操作是相同的。需要用户首先在配置文件中设置好相关参数并准备好相应的交易数据集文件，然后启动 BlockEmulator-X 即可等待实验完成。不过在结果查看的步骤中，使用 BlockEmulator-X 完成智能合约相关实验的实验结果，即智能合约部署与调用的结果需要用户在 `./exp/logs` 包中产生的 log 文件中查看。而其余实验结果（例如 TPS，latency 等）的查看与使用普通的转账交易进行实验相同。

8.4 Step1：编写并编译智能合约

本节分别介绍前述两种编写并编译智能合约的方式：Option 1（使用 Remix）与 Option 2（使用 Solc 工具）。

8.4.1 Option 1: 使用 Remix

1. **打开 Remix：**用户可通过浏览器访问并使用 Remix[1]。

2. **在 Remix 中新建 sol 文件并编写智能合约代码：**如图 8.3 所示，用

[1]Remix 官网：`https://remix.ethereum.org`

图 8.3: 使用 Remix 编写智能合约。

户首先需要创建以.sol 结尾的文件，然后在编辑区中输入自己的智能合约代码。

3. **在 Remix 中编译智能合约并获取对应的 ABI 与字节码：**如图 8.4 所示，在用户编写完成智能合约代码之后，可以点击 Remix 中自带的编译器完成智能合约的编译，编译完成之后即可复制该智能合约的 ABI 和字节码。不过需要注意的是，为了防止 Solidity 编译器版本不一致可能带来的问题，这里我们统一使用版本号为 0.8.34+commit.80d5c536 的编译器，即图 8.4 中所示。

8.4.2 Option 2: 使用 Solc 工具

1. **在项目中新建 `.sol` 文件并编写智能合约代码：**在项目中新建文件与编写智能合约代码的操作与使用 Remix 进行编写类似，这里就不赘述。

2. **使用 Solc 工具进行编译：** Solc [1] 是 Solidity 官方提供的智能合约

[1]Solc 工具的安装方式可参考: `https://docs.soliditylang.org/zh-cn/latest/installing-solidity.html`

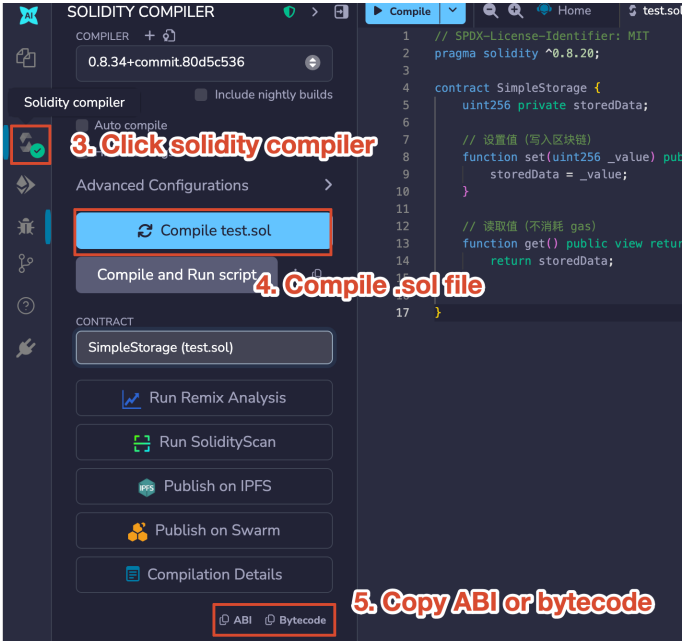


图 8.4: 使用 Remix 编译智能合约。

编译器，为了方便后续实验数据集的生成，BlockEmulator-X 内置了一个 “Trace 文件生成器”。该 Trace 文件生成器会调用 Solc 编译器对用户编写的智能合约进行编译并自动获取到该合约的 ABI 和字节码，用户无需去 Rexim 中复制智能合约编译后产生的 ABI 和字节码。但调用 Solc 需要用户事先在机器中安装好 Solc，为防止 Solc 版本不一致带来的问题，推荐大家安装版本号为 0.8.34 的版本。安装完成后，可通过以下命令查看已安装的 solc 版本：

```
1    solc --version
```

8.5 Steps 2&3：生成“Trace 文件”

由于 BlockEmulator-X 是由交易驱动的，用户在进行智能合约测试前需要事先准备好相应的交易数据集，包括调用地址、交易参数、`gasLimit`、`callingFunction` 等信息。当测试场景中包含大量部署或调用智能合约的操作时，手动构造完整的交易数据集不仅过程繁琐，而且耗时较长，还容易出现字段填写错误或编码不一致的问题。为简化智能合约测试流程，BlockEmulator-X 开发团队在项目中内置了一个通过特定格式的“txt 文档”来描述特定的测试场景，并结合智能合约的 ABI 和 Bytecode 直接生成 CSV 格式的交易数据集文件。我们将该工具命名为“Trace 文件生成器”。借助该工具，用户无需手动构造复杂的 `calldata` 与交易中的其他字段，即可快速完成智能合约测试数据集的生成。以下是该“Trace 文件生成器”的具体使用方式。

8.5.1 测试者自定义的“txt 文档”的编写方法

用户在使用“Trace 文件生成器”生成 Trace 文件之前，除需要准备好智能合约之外，还需要先准备一份包含特定格式的用户自定义“txt 文档”。以下是一份该“txt 文档”的详细说明与示例：

```
#====================================================
# Scenario txt Template
#====================================================
# Rules:
# 1) One command per line.
# 2) Lines starting with '#' are comments.
# 3) Keep key=value pairs separated by spaces.
# 4) For CALL params, use '|' as separator, ordered by
    ABI.
#====================================================


# ----------------------------
# [A] Account Definitions
```

```
# ----------------------------
# Format:
# ACCOUNT <alias> <0xAddress>
#
# alias: custom name used later in from=/params=
# 0xAddress: EVM address (20-byte hex, starts with 0x)

ACCOUNT deployer 0x5B38Da6a701c568545dCfcB03FcB875f56beddC4
ACCOUNT alice 0xAb8483F64d9C6d1EcF9b849Ae677dD3315835cb2
ACCOUNT bob 0x4B20993Bc481177ec7E8f571ceCaE8A9e22C02db
ACCOUNT spender 0x17F6AD8Ef982297579C203069C1DbfFE4348c372


# ----------------------------
# [B] Contract Definitions
# ----------------------------

# Option 1: use-solc=true (compiler mode)
# Format:
# CONTRACT <contract_alias> sol=<path_to_sol> [name=<contract_name_in_sol>]
#
# contract_alias: custom name referenced by DEPLOY/CALL
# sol: path to .sol file
# name: optional but recommended if one sol file has multiple contracts
#
# Use this line when running with: -use-solc=true

CONTRACT Token sol=./contracts/Token.sol name=Token


# Option 2: use-solc=false (artifact mode)
# Format:
# CONTRACT <contract_alias> abi=<path_to_abi_json> bytecode=<path_to_bytecode_file>
#
# abi: JSON ABI file path
# bytecode: deployment bytecode file path (creation bytecode)
#
# Uncomment and use this instead of Option 1 when running with: -use-solc=false
#
# CONTRACT Token abi=./abi/Token.json bytecode=./bin/Token.bin


# ----------------------------
# [C] Scenario Group
```

```
# ----------------------------
# Format:
# SCENARIO <scenario_name>
#
# This is only a logical group label.

SCENARIO initial_distribution


# ----------------------------
# [D] Deploy Step
# ----------------------------
# Format:
# DEPLOY id=<deploy_id> contract=<contract_alias>
    from=<account_alias_or_0x> [gas=<int>]
    [value=<wei_int>]
#
# id: deployment identifier; later CALL can reference
    it via contract_ref=
# contract: must match CONTRACT alias above
# from: account alias or 0x address
# gas: optional, default 2000000
# value: optional, default 0 (wei)

DEPLOY id=deploy_token contract=Token from=deployer
    gas=2000000 value=0


# ----------------------------
# [E] Call Steps
# ----------------------------
# Format:
# CALL contract_ref=<deploy_id>
    from=<account_alias_or_0x>
    function=<abi_method_name> [params=<p1|p2|...>]
    [gas=<int>] [value=<wei_int>]
#
# contract_ref: points to DEPLOY id, ensures calling
    the intended instance
# from: caller alias or 0x address
# function: ABI method name
# params: required only if function has inputs; use
    '|' separator in ABI order
# gas: optional, default 100000
# value: optional, default 0 (wei)

CALL contract_ref=deploy_token from=deployer
    function=transfer
    params=alice|1000000000000000000 gas=120000
    value=0
CALL contract_ref=deploy_token from=deployer
    function=transfer params=bob|5000000000000000000
    gas=120000 value=0

```

```
98
99  SCENARIO transfer_then_check_balance
100
101 CALL contract_ref=deploy_token from=alice
        function=transfer params=bob|1000000000000000000
        gas=120000 value=0
102 CALL contract_ref=deploy_token from=alice
        function=balanceOf params=alice gas=70000 value=0
103 CALL contract_ref=deploy_token from=alice
        function=balanceOf params=bob gas=70000 value=0
104
105
106
107 #====================================================
108 # Parameter Type Tips
109 #====================================================
110 # address: use alias defined by ACCOUNT, or full 0x
        address
111 # uint/int: decimal string (no scientific notation)
112 # bool: true/false or 1/0
113 # bytes: hex string like 0x1234abcd
114 #
115 # Functions with no parameters:
116 # - You can omit params=
117 # Example:
118 # CALL contract_ref=deploy_token from=alice
        function=totalSupply gas=70000 value=0
119 #
120 #====================================================
121 # Run Command Examples
122 #====================================================
123 # use solc:
124 # go run ./cmd/generatecontracttxfile/usetxt/main.go
        -in ./scenario.txt -out ./dataset.csv
        -use-solc=true
125 #
126 # no solc:
127 # go run ./cmd/generatecontracttxfile/usetxt/main.go
        -in ./scenario.txt -out ./dataset.csv
        -use-solc=false
128 #====================================================
```

用户可以在项目中的以下位置找到该 txt 文件：cmd/generatecontract-txfile/usetxt/example/example.txt。

以下是解释其中几个关键部分：

- **Accounts 部分：**该部分主要用于事先声明生成的交易数据集文件中将使用到的账户名称及地址。请注意：该部分在填写时每一行的开头

都应该以 ACCOUNT 开始，然后填入该账户的别名和以 0x 为开头的十六进制账户地址。

```
ACCOUNT AccountName AccountAddress
```

- **Contract 部分：**该部分根据用户上一节中使用的智能合约编译方式的不同而填写不同的内容。二者均需要以 CONTRACT 开始，并在后面填写上智能合约的名称（请注意是用 solidity 编写智能合约代码时命名的智能合约名，并不是文件名）。而两种不同的智能合约编写和编译方式在该文件中需要填写的文件路径中所包含的文件内容是不同的。对于使用 Remix 进行智能合约编写和编译的用户，需要将上一节中提到的复制的智能合约的 ABI 和 Bytecode 分别复制到项目目录中两个新的文件中。其中 ABI 为 json 文件；而 Bytecode 是一串十六进制数，建议复制到以.bin 结尾的空文件中。完成以上步骤后需要在 CONTRACT 这一行中分别填写 ABI 和 Bytecode 的文件路径，具体填写方式如下：

```
// 分别在 abi= 和 bytecode= 之后填写保存的
    ABI 和 Bytecode 的路径
CONTRACT ContractName abi=./xxx.json
    bytecode=./xxx.bin
```

而对于使用 Solc 进行编译的用户则需要填写智能合约的路径和名称，具体填写方式如下：

```
// 分别在 sol= 和 name=
    之后填写合约文件的路径和合约名
CONTRACT ContractName sol=./xxx.sol
    name=ContractName
```

- **Scenario 部分：**该部分是 Trace 文件生成的关键部分，主要由两部分组成。第一部分是场景名称描述，该部分的内容主要是为了方便用

户理解第二部分是测试智能合约的哪一项功能，具体填写方法如下：

```
// 以 SCENARIO 开始，并在后面填写场景的名称
SCENARIO ScenarioName
```

而第二部分则是生成的交易数据集文件中每条交易相关信息的来源。该部分分别包含部署和调用智能合约两种类型；对于部署智能合约的交易，可参考以下方式填写：

```
DEPLOY id=xxx contract=ContractName
    from=DeployerAddress
```

该部分需要用户在填写时以 DEPLOY 作为开头，id= 后填写智能合约的简称或别名。这里主要是为了方便用户调用。contract= 后填写需要部署的智能合约名。需要注意的是，这里填写的智能合约名需要在 Contract 部分中出现过的智能合约名称；否则会导致生成 Trace 文件失败。而 from= 后填写的是部署智能合约的账户地址，此处可以填写在 Accounts 部分出现的账户名称，也可以直接使用以 0x 开头的十六进制地址。

而对于调用智能合约的交易的填写方式如下所示：

```
CALL contract_ref=ContractId
    from=AccountAddress
    function=CalledfunctionName
    params=param1|param2|... gas=GasNum
```

与其他部分类似，该部分需要以 CALL 作为开头，contract_ref= 后填写的是在部署智能合约阶段出现过的被调用智能合约的别名。from= 后填写调用智能合约的账户地址。同样，这里的账户地址可以填写在 Accounts 部分出现的账户名称，也可以直接使用以 0x 开头的十六进制地址。function= 后填写的是被调用的函数名。params= 后需要填写的是被调用函数所需要的参数。如果含有多个参数中间使用 “|”

分开，如果被调用函数无需参数则不填写，即只需要 `params=` 即可；`gas=` 后填写调用智能合约支付的 gas 数量。不过值得注意的是，BE-X 目前暂未实现与测试 gas 消耗相关的功能，这是 BlockEmulator 开发团队的未来工作。

8.5.2 “Trace 文件生成器”的使用方法

BlockEmulator-X 项目中的 Trace 文件生成器的使用是一个 go 代码文件，用户可在项目中的以下位置找到并运行该文件：cmd/generatecontracttxfile/usetxt/main.go。

用户可以直接通过以下命令运行该文件并生成相应的 Trace 文件：

```
go run ./cmd/generatecontracttxfile/usetxt/main.go -in
    ./scenario.txt -out ./dataset.csv
    -use-solc=true/false
```

用户需要在 `-in` 后填写准备好的 txt 文档的路径，而 `-out` 后填的是生成的 Trace 文件的名称和存储的路径。最后的 `-use-solc` 有两个参数可以选择，分别为 `true` 和 `false`。`true` 表示使用 Solc 编译器进行编译，该命令适用于已经在用户自身的机器中已经安装好了 Solc 并且 txt 文档中给出的是用户自己编写的智能合约代码文件的路径的用户。而 `false` 表示不使用 Solc 进行编译，该命令适合使用 Remix 进行智能合约编写与编译的用户。

8.6 Finally：开始运行实验并查看实验结果

8.6.1 运行实验

用户启动 BlockEmulator-X 进行智能合约相关实验的流程与前面章节提到的启动流程是一样的，详细操作方式可参考章节 7.3。需要注意的是，目前 BlockEmulator-X 只支持单分片情况下的智能合约的部署与调用，即无法跨分片调用智能合约。另外，在进行配置项选择时，config.yaml 文件中新增了一个额外的字段，如下：

```
### `exclude_contract_txs` indicates whether to exclude smart contract-related transactions when reading transactions from a CSV file.
### true:  Exclude smart contract-related transactions
### false: Include all transactions (no exclusion)
exclude_contract_txs: false # It is a bool value (["true", "false"]).
```

在进行智能合约相关实验时需要将该字段的值设置为 `false`。

表 8.1: 使用 BlockEmulator-X 进行智能合约实验示例。

交易序号	部署/调用智能合约的账户地址	被调用函数与相关参数	合约部署/调用结果
1	0x5B38...ddC4	Deploy ERC20 contract	Contract Address
2	0x5B38...ddC4	`transfer(0xAb84...5cb2, 10 ETH)`	`true`
3	0x5B38...ddC4	`transfer(0x4B20...02db, 5 ETH)`	`true`
4	0xAb84...5cb2	`transfer(0x4B20...02db, 1 ETH)`	`true`
5	0xAb84...5cb2	`balanceOf(0xAb84...5cb2)`	9 ETH
6	0xAb84...5cb2	`balanceOf(0x4B20...02db)`	6 ETH
7	0xAb84...5cb2	`approve(0x17F6...c372, 5 ETH)`	`true`
8	0x17F6...c372	`transferFrom(0xAb84...5cb2, 0x4B20...02db, 2 ETH)`	`true`
9	0xAb84...5cb2	`balanceOf(0xAb84...5cb2)`	7 ETH
10	0xAb84...5cb2	`balanceOf(0x4B20...02db)`	8 ETH
11	0xAb84...5cb2	`allowance(0xAb84...5cb2, 0x17F6...c372)`	3 ETH

8.6.2 查看实验结果

对于不涉及智能合约的实验，可以在 exp 目录下的文件中查看实验的结果，主要包括 TPS、延迟等信息。而对于涉及智能合约的交易，查看智能合约的部署与调用结果目前需要用户在 exp/logs 目录下的 log 文件中查看。表 8.1 展示了使用 BlockEmulator-X 进行 ERC20 标准的智能合约部署与调用的结果，其中的部署/调用智能合约的账户地址列和被调用函数与相关参数列是事先通过章节8.5.1 中提到的 txt 文档设定好的测试场景，并通过 Trace 文件生成器生成的相应的 Trace 文件。而结果列则是通过运行 BlockEmulator-X 后得到的对应的结果。

图 8.5: 实验后，日志文件内容示例。

图8.5展示的是用户使用 BlockEmulator-X 进行智能合约实验后可能在日志文件中看到的一些相关信息，下面将详细解释。

```
"deploy contract succeed" NodeInfo=S{0}_N{0} "contract
    addr"=0xd9145cce52d386f254917e481eb44e9943f39138
```

以上展示的是部署智能合约时日志文件中记录的信息，其中 contract addr 显示的是成功部署智能合约后得到的智能合约地址。

```
"before call" NodeInfo=S{0}_N{0}
    data_hex=0xa9059cbb00...8ac7230489e80000
```

以上信息展示的是在智能合约被调用之前输出的被调用函数的字节码，此信息的输出是为了方便用户了解到当前被调用的智能合约及函数是什么。

```
result="\x00\x00\x00\x00\...\x00\x01"
```

以上展示的是调用智能合约中的函数并成功执行之后所返回的结果。

需要声明的一点是：*BE-X* 对智能合约执行情况的测试**并不能保证结果的逻辑正确性**，但会帮助用户观察给定的交易集合按照自定义顺序执行后的**诚实结果**。这个观察到的结果恰好可以用来帮助测试者来推断用户输入到 *BE-X* 自定义交易顺序的**合理**或者**不合理**之处。

8.7 可能会遇到的技术问题与解决方案

该部分将持续记录用户在使用 BlockEmulator-X 进行智能合约相关实验时可能遇到的技术问题与对应的解决方案，欢迎大家在 BlockEmulator-X 的 Github Issues 界面[1]与 BlockEmulator 用户交流群积极提出相关问题，BlockEmulator-X 开发团队会第一时间解决。

- **Issue:** “在进行交易顺序敏感性测试时，编写的交易顺序导致触发了 revert 的错误，为什么该区块中后续正常的交易就不执行了?”

  回答: 这是由于在目前版本的 BlockEmulator-X 的实现中，当处理区块中的交易出现 error 时，会直接退出对该区块的处理，即该区块中后续的交易将不会被执行。目前 BlockEmulator-X 开发团队正在尝试解决该问题。

- **Issue:** ... (BlockEmulator-X 开发团队将持续追踪与更新用户使用时碰到的各项技术问题)。

[1] https://github.com/HuangLab-SYSU/block-emulator-x/issues

第九章　用 BlockEmulator-Web 启动 BE-X

9.1　BlockEmulator-Web 是什么？

BlockEmulator-Web（简称为 BE-Web）是一个运行在用户本地的 web 端控制台页面，主要是为 BlockEmulator-X 提供一个便捷、可视化的方式来配置实验参数、运行实验，以及查看实验结果。

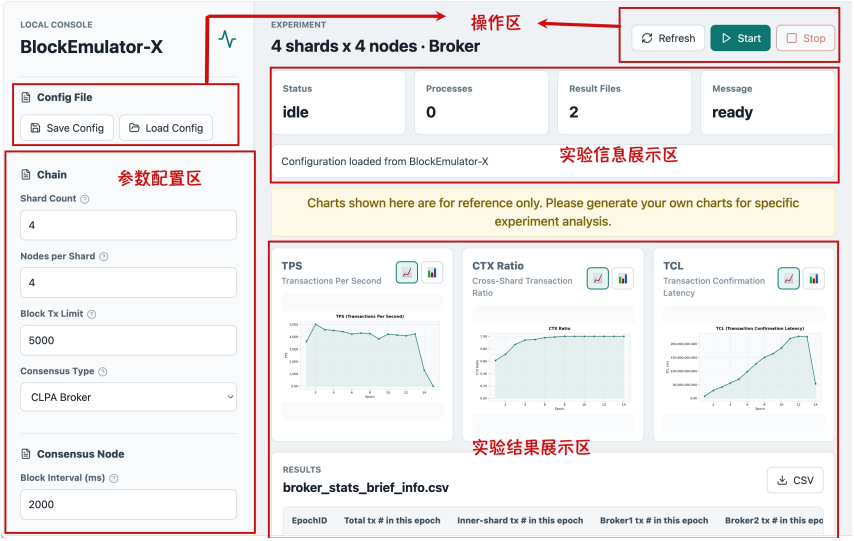


图 9.1: BE-Web 界面，大致可分为四个区域：操作区、参数配置区、实验信息展示区、与实验结果展示区。

9.1.1 BlockEmulator-Web 的功能

- **可视化配置编辑器：**用户可在浏览器页面中输入/修改部分重要参数，将鼠标悬停至每个参数旁的 “?”图标上方即可查看该参数的说明。
- **一键启动/停止实验：**用户点击绿色 `Start` 按钮后，后端将自动验证配置并生成对应的配置文件，然后启动所有节点进程运行实验。
- **实时监控状态：**`Status` / `Processes` / `Result Files` 三张卡片中的内容每隔 3 秒会自动刷新。
- **展示统计图表：**在一组实验运行完成后，BE-Web 会自动生成 TPS、CTX Ratio、TCL 三张图并展示在页面中。实验图展示支持 “折线图/柱状图” 一键切换。
- **实时查看日志：** BE-Web 支持用户按共识节点筛选日志，帮助用户更好地定位可能遇到的问题。实验日志每隔 3 秒会自动刷新末尾 50 KB 的数据。
- **参数配置可本地保存与加载：**用户可将当前实验的参数配置保存为 JSON 文件到本地。后续进行重复实验时，可随时加载复用已保存的配置文件。

9.1.2 BlockEmulator-Web 的项目架构

```
BlockEmulator-web
|-- backend                    # Go API 服务源码
|   |-- main.go                # HTTP 路由入口
|   \-- internal
|       \-- emulator           # 核心业务逻辑
|
|-- frontend                   # React + Vite 前端源码
|   \-- src
|       |-- main.jsx           # 主组件
|       \-- styles.css         # 全局样式
|
|-- python-backend             # Flask 图表服务源码
|   |-- app.py                 # Flask 路由
|   |-- charts.py              # matplotlib 绘图逻辑
|   \-- requirements.txt       # Python 依赖
|
|-- start.sh                   # macOS/Linux 一键启动脚本
|-- start.bat                  # Windows 一键启动脚本
\-- README.md                  # 项目说明文档
```

9.2 使用 BE-Web 前的准备工作

用户在使用 BE-Web 进行实验之前，还需要做好如下一些准备工作。

9.2.1 环境准备

- **Go 环境：**安装方式可参考章节 4.6.1。
- **安装 Node.js + npm：**可通过 Node.js 官网[1] 进行安装。安装完成后可通过以下命令测试安装是否成功：

```
node --version
npm --version
```

若成功安装则会显示对应的版本号，建议用户下载最新版本。

- **安装 Python 3：**用户可通过 Python 官网[2]下载最新版的 Python 3，可通过以下命令测试 Python 的版本：

```
python --version
```

9.2.2 克隆 BE-X/BE-Web 仓库到本地

- **克隆 BE-X：**从 Github 仓库[3] 克隆 BE-X 到本地的操作可参考章节 4.2.1。
- **克隆 BE-Web：**用户可通过 Github 仓库[4] 将 BE-Web 克隆到本地，具体操作可参考章节 4.2.1。

[1] https://nodejs.org/zh-cn/download

[2] https://www.python.org/downloads/

[3] BE-X 的仓库地址为：https://github.com/HuangLab-SYSU/block-emulator-x

[4] BE-Web 的仓库地址为：https://github.com/HuangLab-SYSU/BlockEmulator-Web

9.2.3 设置 BE-X 项目的本地路径

需要用户注意的是，BlockEmulator-Web 并不包含 BlockEmulator-X 本身。它是 BlockEmulator-X 的 Web 端控制台。用户在使用 BE-Web 前，需要确保本地已经存在 BlockEmulator-X 的源码。

在默认情况下，Web 后端会在以下路径中寻找 config.yaml：

```
1. ../block-emulator-x/config.yaml（即 BE-Web 项目的同级目录）
2. ../../block-emulator-x/config.yaml（即 BE-Web
    项目的上级目录）
```

因此，我们建议将两个项目放在同一父目录下。

如果你的 BlockEmulator-X 在其他位置，有两种方式来指定：

```
# 方式一：设置环境变量（推荐）
export BLOCK_EMULATOR_X_ROOT=/{你的 block-emulator-x 路径}

# 方式二：在启动脚本前设置（单次生效）
BLOCK_EMULATOR_X_ROOT=/{你的 block-emulator-x 路径}./start.sh
```

用户可通过执行以下命令验证 BlockEmulator-X 位置是否正确：

```
ls "$BLOCK_EMULATOR_X_ROOT/config.yaml"
# 或用默认路径：
ls ../block-emulator-x/config.yaml
```

如果显示 `No such file or directory`，则说明路径设置不正确，需要用户再次检查 BE-X 的项目路径是否设置正确。

9.3 启动 BE-Web 中的所有服务

9.3.1 通过脚本一键启动所有服务

在 BE-Web 项目中内置了两个一键启动所有服务的脚本：`start.sh` (MacOS/Linux) 和 `start.bat` (Windows)。用户可根据自己的操作系统选择对应的脚本，一键启动所有服务。

在启动服务时，脚本会依次做以下事情，并且终端会有明确输出：

```
+--------------------------------------------------+
|        BlockEmulator-Web Console Startup         |
+--------------------------------------------------+

[backend] Starting Go backend...
       PID: 12345  ->  http://localhost:9091

[frontend] Starting React + Vite dev server...
       PID: 12346  ->  http://localhost:5173

[charts] Starting Python chart server...
       PID: 12347  ->  http://localhost:5001

--------------------------------------------------
All requested services are running.
Press Ctrl+C to stop all services.
--------------------------------------------------
```

不过需要注意的是，首次启动时会执行如下额外操作:

- **前端首次启动：**脚本会自动运行 `npm install`，下载 React、Vite 等依赖。用户将看到 `node_modules not found, running npm install...` 的提示，以及 npm 的安装进度条。
- **Python 首次启动：**脚本会自动创建虚拟环境并执行 `pip install -r requirements.txt`，安装 `Flask` 和 `matplotlib`。用户此时能看到 `Creating Python virtual environment...` 和 pip 的安装进度。

首次运行 `npm install` 可能需要几分钟，请耐心等待，之后再启动 BE-Web 时将会跳过这些步骤。

9.3.2 按需启动某个服务

用户如果只需要启动某个服务，可分别通过以下命令运行脚本启动：

```
# 只启动 Go 后端
./start.sh backend

# 只启动 React 前端
./start.sh frontend

# 只启动 Python 图表服务
./start.sh charts
```

如果用户不想使用脚本，也可以手动将各个服务依次启动。那么，用户需要打开三个终端窗口：

```
# 终端 1 — Go 后端：
cd your/BlockEmulator-web/path/backend
go run .
# 等待一段时间，看到以下输出说明启动成功：
# BlockEmulator Web backend listening on http://localhost:9091
```

首次运行 `go run .` 命令，会自动下载 Go 依赖（gopkg.in/yaml.v3），可能需要几十秒。

```
# 终端 2 — React 前端：
cd your/BlockEmulator-web/path/frontend
npm install        # 仅首次需要
npm run dev
# 看到以下输出说明启动成功：
# VITE v7.x.x  ready in xxx ms
# Local:   http://localhost:5173/
```

```
# 终端 3 — Python 图表服务：
cd your/BlockEmulator-web/path/python-backend

# 首次执行：创建虚拟环境并安装依赖
python3 -m venv venv
source venv/bin/activate      # macOS/Linux
# 或 venv\Scripts\activate    # Windows
pip install -r requirements.txt

# 启动服务
python app.py
# 看到以下输出说明启动成功：
# Chart server → http://localhost:5001
```

9.3.3 验证服务是否全部启动成功

- **检查 1：Go 后端**。用户在浏览器中访问：

  `http://localhost:9091/api/config` 或者在终端执行：

```
curl -s http://localhost:9091/api/config | head -20
```

  预期效果是页面显示出一段类似以下形式的 JSON 文本：

```
{
  "ok": true,
  "data": {
    "Chain": {
      "shard_num": 4,
      "node_num": 4,
      ...
    }
  }
}
```

- **检查 2：React 前端**。用户在浏览器中访问：`http://localhost:5173`。预期效果是用户能看到一个完整的 Web 页面：左侧是配置表单（Chain / Consensus Node / Supervisor / Network 四个区块），右侧顶部有操作按钮（Refresh/Save Config/Load Config/Start/Stop），下面是 Status 卡片。用户如果看到空白页或是加载错误，需要确认 Go 后端（9091）已经启动，因为前端加载时会立即请求后端配置。

- **检查 3：Python 图表展示服务**。用户在浏览器中访问：

  `http://localhost:5001/api/health` 或者在终端执行：

```
curl -s http://localhost:5001/api/health
```

  预期效果是页面将显示包含以下内容的 JSON 文本：

```
{
  "ok": true,
  "data_available": false,
  "csv_path": null,
```

```
  "row_count": 0
}
```

初次启动时，如果看到 `data_available: false` 是正常现象，因为用户还没有运行过实验，此时还没有记录实验结果的 CSV 数据可用于绘制图表。

- **一键检查所有端口**。对于 MacOS 和 Linux 的用户，也可通过以下命令在终端一键检查所有服务是否启动成功：

```
for port in 9091 5173 5001; do
  if lsof -i :$port > /dev/null 2>&1; then
    echo "port $port is listening"
  else
    echo "port $port is NOT listening"
  fi
done
```

对于 Windows 用户，也可以在 PowerShell 中运行以下命令一键检查所有服务是否启动成功：

```
@(9091, 5173, 5001) | ForEach-Object {
  $conn = Get-NetTCPConnection -LocalPort $_
      -ErrorAction SilentlyContinue
  if ($conn) { "port $_ is listening" } else { "port $_
      is NOT listening" }
}
```

9.3.4 停止所有服务

- **停止使用脚本启动的服务：**用户可直接按 `Ctrl + C`，脚本会自动终止所有三个后台服务。另外，启动脚本中内置了 `trap cleanup EXIT INT TERM`，因此即使用户直接关闭终端窗口，后台进程也会被清理。

- **停止手动启动的服务：**用户只需要在每个终端窗口中分别按 `Ctrl + C`，即可停止在对应终端已启动的服务。

9.4 BE-Web 的界面布局与功能区介绍

用户在成功启动 React 前端和 Go 后端服务之后，打开浏览器访问 `http://localhost:5173`，将会看到如图 9.2 所示界面布局。

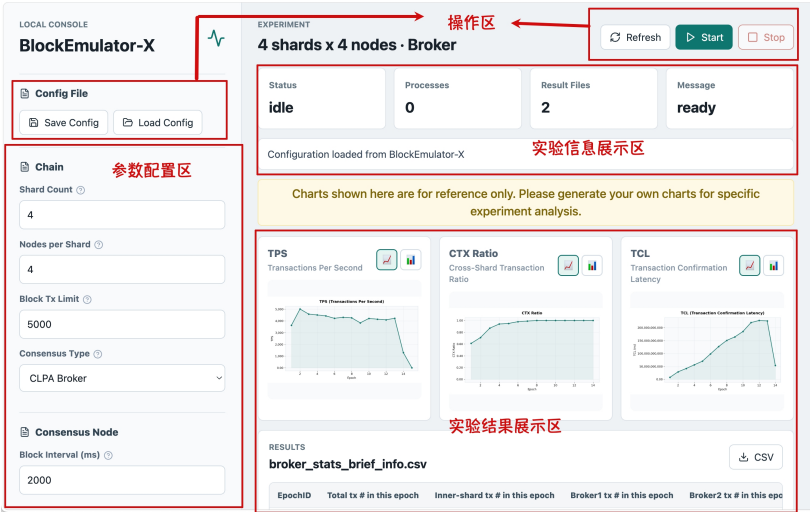


图 9.2: BE-Web 界面展示

BE-Web 主要包含四个区域：左侧参数配置区、右侧顶部操作区、右侧中间实验信息展示区和右侧底部实验结果展示区。以下将介绍各个区域。

9.4.1 左侧：实验参数配置区

参数配置区主要包含四个配置区域，每个区域包含对应可修改的配置项。每个参数配置项的输入框旁边的“?”图标，是该配置项介绍的提示符号。用户可将鼠标移到该问号图标上，但不要点击，则会自动弹出浮动提示框，会显示该参数的详细解释。

参数配置区域几个重要类别的参数设置区，从上到下依次为：

- **Chain（链参数）：**用户可修改分片数、各分片节点数、区块大小、共识类型。
- **Consensus Node（共识节点参数）：**用户可修改出块间隔。
- **Supervisor 参数：**用户可修改交易注入总数量、交易注入速率、CLPA 重配置时间间隔、交易来源、交易数据集 CSV 文件路径。
- **Network（网络参数）：**用户可修改带宽、延迟。

对于可以填写大数值的配置项，例如交易注入总数量等，BE-Web 也支持用户使用科学计数法（即计数单位为“1e6”）的方式来填写。另外，BE-Web 只支持配置一些重要的实验参数。如果用户需要调整其他更多实验参数，仍然需要在 BE-X 项目中的 `config.yaml` 文件中进行设置。

9.4.2 右侧：顶部操作区

操作栏会显示当前配置的简要信息（例如 4 shards × 4 nodes · Relay），以及 5 个操作按钮，各个按钮的功能可参考表格 9.1。

表 9.1: 操作区按钮说明

按钮	功能	何时可用
Refresh	从 BE-X 重新加载配置，会覆盖当前修改	随时
Save Config	将当前配置下载为 `blockemulator-config.json`	随时
Load Config	从本地 `.json` 文件加载配置	随时
Start	启动实验	仅实验未运行时
Stop	终止实验	仅实验运行时

9.4.3 右侧：实验信息展示区

实验信息展示区主要包括四张状态卡片，每 3 秒会自动刷新一次，各状态卡片的详细解释见表 9.2。

表 9.2: 状态卡片说明

卡片	含义	可能的值
Status	实验运行状态	`idle`（空闲）、`running`（运行中）、`finished`（已完成）、`failed`（失败）、`stopped`（已停止）
Processes	当前运行的进程数	数字，例如 `5`（表示 1 个 Supervisor 和 4 个节点）
Result Files	结果 CSV 文件数	数字，例如 `3`
Message	当前状态描述	例如 `"ready"`、`"experiment is running"`、`"experiment finished"`

而状态卡片下方的文本框中也会显示一些用户进行操作后的相关信息，例如用户在保存配置完成之后会显示 `Configuration downloaded` 等类似的提示信息。

9.4.4 右侧：底部实验结果展示区

该区域包含的内容较多，主要包括 3 个主要功能：图表展示、结果表格、与日志输出。

图 9.3: 实验结果图表展示

1. **图表展示：**该区域会展示三张并排的图表卡片：TPS、CTX Ratio、TCL。在实验未运行完成时，每张卡片显示 “Waiting for experiment data...”，并且在实验完成后

会自动加载通过 Python 绘制的 PNG 格式的图表图片。每张卡片均支持折线图和柱状图展示，用户可点击每张卡片右上角的折线图和柱状图的图表进行切换，如图 9.3 所示。另外，需要用户注意的几点是：

- 需要提前启动 Python 图表服务，否则图表无法显示。启动方式见章节 9.3。
- 图表中的横轴 Epoch 指的是配置项中设置的 CLPA 算法进行一次账户重配置的时间间隔，若用户运行实验时选择的是 Static 方式，将只能看见一个 Epoch 的数据。
- 页面中显示的图表仅供用户参考，具体的实验图仍需要用户自己绘制.

2. **结果表格：**实验运行结束后，会显示实验产生的 CSV 格式的结果数据。表格上方会显示文件名和 CSV 文件下载按钮，用户可点击下载按钮下载该结果文件到本地，如图 9.4 所示。页面中展示的表格支持水平滚动，用户可据此查看更多的信息。

图 9.4: 实验结果展示

3. **日志输出：**实验结果展示区的最底部是 BlockEmulator-X 运行过程中产生的日志，该日志是实时显示的。在默认情况下，日志文本框内输出的是所有节点运行产生的日志，用户可通过日志输出区右上角的下拉框选择对应的节点日志进行观看，如图 9.5 所示。

图 9.5: 查看日志

9.4.5 用户上传交易数据集文件，用于驱动实验运行

当实验参数配置项 `Transaction Source` 被选择为 `CSV Source` 时，用户需上传交易数据集文件，用于驱动实验运行。交易数据集文件的上传支持两种方式：

1. **手动输入文件路径：**用户可直接点击输入框，输入 CSV 文件在 BE-X 目录下的路径，例如 `./data/txs.csv`。
2. **上传本地文件（推荐）：**用户点击输入框右侧的上传按钮后，系统将弹出文件选择对话框，在对话框中找到用户需要的 `.csv` 文件后，选中并点击“打开”，即可上传文件至 BE-X。文件上传成功后，输入框中将自动填入在用户机器上的文件路径，并且在通知区的通知栏中也将显示 `"File uploaded: your_file_name.csv"`。

若出现上传失败的情况，用户可从以下几个方面进行检查：

- 确认文件名的后缀为 `.csv`；
- 确认文件大小少于 50 MB；
- 确认 Go 后端的 `9091` 端口正常运行；
- 确认后端工作目录有写入权限。

9.4.6 管理参数配置：保存、加载、上传

当用户在 BE-Web 页面上修改任何一个实验参数配置项时，修改后的参数会立即生效并存储在浏览器的本地状态中。但这个修改后的参数不会自动写入 BlockEmulator-X 的配置文件中。只有在用户点击 `Start` 按钮启动实验时，前端才会将当前最新配置参数发送给后端。后端再据此写入 `generated_config.yaml` 并更新到 BlockEmulator-X 中。也就是说，在页面上改完一个参数不等于它已经保存，也不等于它已生效。如果用户刷新了页面，修改的配置项会丢失（除非你之前保存了参数配置文件）。为此，BE-Web 为用户提供了“保存”和“加载”实验配置参数的功能。

1. **保存配置到本地文件：**在用户调整好各项参数后，点击右侧顶部操作区的 `Save Config` 按钮，此时浏览器自动触发下载，文件名固定为 blockemulator-config.json。下载位置默认是浏览器中设置的文件保存位置。在保存完成后，页面顶部的信息框中会显示绿色提示：`"Configuration downloaded"`。保存后的文件示例如下：

```
{
  "Chain": {
    "shard_num": 4,
    "node_num": 4,
    "limit": 5000,
    "consensus_type": "static_relay"
  },
  "consensus_node": {
    "block_interval": 2000
  },
  "supervisor": {
    "tx_number": 100000,
    "tx_injection_speed": 20000,
    "epoch_duration": 50,
    "tx_source_type": "random_source",
    "tx_source_file": "",
    "exclude_contract_txs": true
  },
  "network": {
    "bandwidth": 1000000,
    "latency": 0
  }
}
```

2. **从本地文件加载参数配置：**用户点击右侧顶部操作区中的 `Load Config` 按钮之后，系统会弹出文件选择对话框，用户可找到之前保存的.json 配置文件选中后，点击“打

开”。然后前端会自动读取文件内容并解析 JSON 文件，然后将解析后得到的内容更新到所有配置项中。

- **加载成功时：**页面右侧的信息框中会显示出绿色的提示信息，表示配置已成功加载：`"Configuration loaded from filename.json"`。
- **加载失败时：**页面右侧的信息框中会显示黄色错误提示：“`Failed to parse config file:` 错误详情”。这通常是因为文件不是有效的 JSON 格式。用户可以先用文本编辑器打开文件进行检查。
- **容错处理：**如果用户上传的 JSON 文件中缺少某些字段（例如旧版本保存的配置不包含最新版本中新增的字段），系统会自动用默认值补齐，不会导致加载失败或页面崩溃。

9.5 运行实验：从开始到结束

9.5.1 Step1：启动前的最终检查

用户在点击 `Start` 按钮启动 BE-X 运行实验之前，应该对以下内容进行确认：

- **三个服务正常运行：**在浏览器中 `http://localhost:5173` 正常显示界面，页面左侧配置项数据正常加载（不是空的默认值）。如果未成功启动可参考章节 9.3。
- **所有配置项参数正确：**需要用户自己逐一检查分片数、节点数、共识类型、交易数等是否正确。
- **如果选了 CSV Source：**如果用户在参数配置项中的 `Transaction Source` 选择的是 `CSV Source`，那么用户需要确认 `CSV Path` 已填写（上传后自动填入或手动输入），填写方式可参考章节 9.4.5
- **BlockEmulator-X 路径正确：**如果用户在之前启动后端时出现 `BlockEmulator-X root not found` 错误，可参考章节 9.2.3 进行解决。
- **无实验正在运行：**右侧实验信息展示区中 `Status` 卡片显示 `idle` 或 `finished`，`Start` 按钮处于可用状态（没有灰色禁用）。

9.5.2 Step2：点击 Start 按钮，启动实验

用户在确认实验前的准备工作无误后，可点击页面右侧顶部操作区中的 `Start` 按钮启动 BE-X 运行实验。在点击之后，`Start` 按钮将瞬间变为灰色（禁用状态），实验过程中无法再次点击。另外，页面中的信息框中也会显示 “`Experiment started`”。

9.5.3 启动过程中后端做了什么？

当用户点击 `Start` 按钮后后，前端发送一个 `POST/api/experiments/start` 请求给后端，并且携带完整配置的 JSON 数据。然后后端将依次执行以下步骤：

1. **验证配置：**后端会调用 `ValidateConfig()` 检查所有必填字段和值范围。如果验证失败，则返回 400 错误，并在前端显示错误信息。`Start` 按钮重新恢复可用。
2. **重置实验目录：**后端会删除 `backend/workdir/exp/` 目录及其上所有内容（上一次实验的日志、结果都被清除），并创建新的空目录。
3. **保存配置文件：**后端会读取 BlockEmulator-X 的原始 `config.yaml` 模板，并将 Web 端配置的参数覆盖进去，生成 `backend/workdir/generated_config.yaml` 文件。
4. **生成 IP 表：**后端会为每个分片中的每个节点分配 127.0.0.1 上的端口号（从 32217 开始递增），并生成 `backend/workdir/generated_ip_table.json` 文件。
5. **编译 BlockEmulator-X：**后端会在 BlockEmulator-X 根目录下执行 `go build ./...` 命令。如果编译失败，则实验状态变为 `failed`，并且错误信息会显示在 `Message` 状态卡片中。
6. **依次启动所有进程：**后端将分别运行以下命令启动每个分片的每个共识节点进程和一个 supervisor 进程。

```
# 启动共识节点进程
go run cmd/consensusnode/main.go -config ... -ip_table
    ... -shard_id N -node_id M
# 启动 supervisor 进程
go run cmd/supervisor/main.go -config ... -ip_table ...
    -shard_id=0x7fffffff -node_id=0
```

所有进程的 stdout 和 stderr 重定向到 `backend/workdir/experiment.log` 文件中。

表 9.3: 常见错误及原因

错误信息	可能原因
`go build failed: ...`	BlockEmulator-X 源码存在编译错误
`validation error: ...`	配置参数验证失败
进程 panic（查看日志）	BlockEmulator-X 运行时发生错误

9.5.4 Step3：实验结束

实验结束存在实验正常结束和异常结束两种情况。

- **实验正常结束：**当所有进程退出(即所有交易注入完毕且被处理完),后端会自动检测到并更新状态。在页面中主要体现为进程数量为 0, 实验状态由 `running` 变为 `finished` 等。
- **实验异常结束：**如果任何节点进程异常退出（如 crash、panic 等），`Status` 卡片变为 `failed`，`Message` 卡片显示具体的错误信息。常见的失败原因见表格 9.3。除进程原因外，用户也可通过操作区中的 `Stop` 按钮手动停止实验。用户在点击 `Stop` 按钮后，后端会向所有子进程的进程组发送 `SIGTERM` 信号。`Status` 卡片从 `running` 变为 `stopped`。并且 `Message` 会显示 “`stopped by user`”。另外，停止实验并不会删除已生成的结果文件。用户仍然可以查看部分结果和日志。

9.6 查看实验结果

当实验运行完成之后，用户可直接在实验结果展示区查看对应的实验结果。这里主要介绍一下图表绘制和日志查看功能。

9.6.1 图表绘制服务

图表绘制的功能已在章节 9.4.4 中已经介绍过，这里不再赘述。除在 BE-Web 端页面查看所有生成的图表外，用户也可根据特定的 URL 访问查看对应的图表。具体对应关系可参考表 9.4。

表 9.4: Python 图表绘制服务接口

URL	效果
http://localhost:5001/api/charts/tps?type=line	TPS 折线图 PNG
http://localhost:5001/api/charts/tps?type=bar	TPS 柱状图 PNG
http://localhost:5001/api/charts/ctx_ratio?type=line	CTX Ratio 折线图 PNG
http://localhost:5001/api/charts/ctx_ratio?type=bar	CTX Ratio 柱状图 PNG
http://localhost:5001/api/charts/tcl?type=line	TCL 折线图 PNG
http://localhost:5001/api/charts/tcl?type=bar	TCL 柱状图 PNG
http://localhost:5001/api/charts/combined	三合一组合图 PNG
http://localhost:5001/	独立 HTML 预览页面，展示所有 6 张图及组合图

9.6.2 查看日志

不同节点对应的日志切换的功能已在章节 9.4.4 中介绍过了，这里主要介绍一下日志内容的**时效性**。日志文件会每 3 秒自动刷新一次。并且，每次刷新时，后端只会返回日志文件末尾 50KB 数据，为了避免前端内存溢出和网络拥堵。如果用户切换了日志源，前端会立即请求新日志源的内容，不需要等到下一个 3 秒周期再返回结果。

实际上，在启动实验之前，可选择的节点日志列表是根据当前配置（例如 4 分片 × 4 节点 = 16 个节点 + 1 个 supervisor）进行计算的。在实验启动后，后端会扫描项目中 `backend/workdir/exp/logs/` 目录来获取日志源列表。如果当前选中的日志源不再可用（例如你在看 `shard=0_node=0` 的日志，但该节点的日志文件因某种原因不存在了），前端会自动将展示的节点日志重置为 `all`。

9.7 其他补充信息

9.7.1 BE-Web 后端（端口 9091）API 接口说明：表 9.5

表 9.5: Go 后端 REST API 接口说明

方法	路径	说明
GET	/api/config	获取当前配置
POST	/api/config	验证并保存配置
POST	/api/config/validate	仅验证配置
GET	/api/config/saved	列出已保存配置
POST	/api/config/saved	保存命名配置
GET	/api/config/saved/{id}	加载指定配置
DELETE	/api/config/saved/{id}	删除指定配置
POST	/api/ip-table	生成 IP 地址表
POST	/api/upload/tx-source	上传 CSV 交易文件
POST	/api/experiments/start	启动实验
POST	/api/experiments/stop	停止实验
GET	/api/experiments/status	获取实验状态
GET	/api/experiments/logs?source=x	获取日志（末尾 50KB）
GET	/api/experiments/logs/sources	获取日志源列表
GET	/api/results	获取实验结果 CSV
GET	/api/results/download/{file}	下载指定结果文件

9.7.2 完整的默认参数配置值

```
{
  "chain": {
    "shard_num": 4,
    "node_num": 4,
    "limit": 5000,
    "consensus_type": "static_relay"
  },
  "consensus_node": {
    "block_interval": 2000
  },
  "supervisor": {
    "tx_number": 100000,
    "tx_injection_speed": 20000,
    "epoch_duration": 50,
    "tx_source_type": "random_source",
    "tx_source_file": "",
    "exclude_contract_txs": true
  },
  "network": {
    "bandwidth": 1000000,
```

```
    "latency": 0
  }
}
```

9.7.3 BE-Web 项目中环境变量速查表：表 9.6

表 9.6: 环境变量说明

变量名	默认值	说明
BLOCK_EMULATOR_X_ROOT	自动检测 ../block-emulator-x	BlockEmulator-X 根目录
BLOCK_EMULATOR_WEB_ADDR	9091	Go 后端监听地址
VITE_API_BASE	http://localhost:9091	前端连接 Go 后端的基础 URL
VITE_CHART_API_BASE	http://localhost:5001	前端连接图表服务的基础 URL
PORT	5001	Python 图表绘制服务监听端口
BLOCKEMULATOR_WORKDIR	../backend/workdir	图表服务读取结果的工作目录

9.8 使用过程中可能遇见的问题

9.8.1 Go 后端启动失败

Go 后端启动失败时，完整的错误信息如下：

```
BlockEmulator-X root not found at ../block-emulator-x: stat
    ../block-emulator-x/config.yaml: no such file or directory
```

原因：后端在 ../block-emulator-x/ 和 ../../block-emulator-x/ 这两个位置都没找到 config.yaml 文件。

解决方法：

1. **确认 BE-X 的目录位置：**在 BlockEmulator-Web 目录下执行以下命令：

```
ls ../block-emulator-x/config.yaml
```

2. **设置环境变量：**在执行完以上命令后，如果提示 `No such file or directory`，可使用以下命令设置环境变量：

```
export BLOCK_EMULATOR_X_ROOT=/your/blockemulator-x/path
```

3. **重新启动后端：**在正确设置环境变量之后，使用以下命令重启后端：

```
cd backend && go run .
```

9.8.2 端口被占用

当端口被占用时，完整错误信息类似如下：

```
listen tcp :8080: bind: address already in use
```

原因：之前启动的后端进程或其他程序占用了该端口。

解决方法：首先使用以下命令查找占用端口的进程：

```
lsof -i :9091    # 替换为被占用的端口（9091/5173/5001）
```

```
(base) xjb@JianbodeMac-mini ~ % lsof -i :9091
COMMAND     PID USER   FD   TYPE             DEVICE SIZE/OFF NODE NAME
Microsoft   598  xjb   26u  IPv6 0x4475cb197c158bab      0t0  TCP localhost:5666
9->localhost:xmltec-xmlmail (ESTABLISHED)
Microsoft   598  xjb   27u  IPv6 0xe33f40b1ad4043ec      0t0  TCP localhost:5667
0->localhost:xmltec-xmlmail (ESTABLISHED)
Microsoft   598  xjb   28u  IPv6 0x28e4f6d806d3f6cd      0t0  TCP localhost:5667
1->localhost:xmltec-xmlmail (ESTABLISHED)
Microsoft   598  xjb   31u  IPv6 0xbc2bd30c859a33fa      0t0  TCP localhost:5667
2->localhost:xmltec-xmlmail (ESTABLISHED)
backend   43132  xjb    3u  IPv6 0x88b07eae192f88e4      0t0  TCP *:xmltec-xmlma
il (LISTEN)
backend   43132  xjb    5u  IPv6 0xb11e4420a9f8e325      0t0  TCP localhost:xmlt
ec-xmlmail->localhost:56669 (ESTABLISHED)
backend   43132  xjb    6u  IPv6 0x34f314cd2711e471      0t0  TCP localhost:xmlt
ec-xmlmail->localhost:56670 (ESTABLISHED)
backend   43132  xjb    7u  IPv6 0x20a31a0b89c67cd0      0t0  TCP localhost:xmlt
ec-xmlmail->localhost:56671 (ESTABLISHED)
```

图 9.6: 查看被占用端口

使用以上命令后会有如图 9.6 所示输出。 然后使用以下命令终止该进程：

```
kill 12345    # 替换 12345 为上面查到的 PID
```

如果用户不想终止进程，也可使用以下命令更换端口：

```
# Go 后端换到 8090
BLOCK_EMULATOR_WEB_ADDR=:8090 go run .

# Python 图表换到 5002
PORT=5002 python app.py
```

不过需要用户注意的是，如果更换了端口，React 前端也需要知道新的端口。前端可通过环境变量 `VITE_API_BASE`（默认 http://localhost:9091）和 `VITE_CHART_API_BASE`（默认 http://localhost:5001）连接后端。如果换了端口，需要同步修改这些环境变量，或者在项目中的 `frontend/vite.config.js` 文件中配置，亦或者使用启动脚本中内置的默认端口。

第十章　使用 BE 可能会遇到的技术问题

10.1　用户二次开发中遇到的 Bug 及解决方案

开源 BlockEmulator 至今，我们已经解决了不少代码上的 bugs，其中的一些 bugs 与协议细节有关，还有一些与代码设计本身有关。我们将遇到的一些问题，以及它们的解决方案整理到了 GitHub 的 Issue Log [1]。

BlockEmulator 是一个支持用户二次开发的实验平台。在二次开发过程中，用户可能会遇到各种技术问题。我们将收集到的一些典型技术问题公开在这里，在大家遇到类似问题时希望能够您提供一些解决思路。

问题标题	**问题描述**	**解决方案**
PBFT 共识间隔设置	PBFT 的消息在共识间隔很低的时候，会出现乱序。	使用 Go routine 并行处理消息，从而消息便可以按照预期顺序执行。
`CLPACommitteeModule` 出现死循环	将分片数目设置为 1 时，Supervisor 节点会在第一次运行 CLPA 算法后卡住。	加上判断：分片数目为 1 时，Supervisor 节点不执行 CLPA 算法。

[1] `https://github.com/HuangLab-SYSU/block-emulator/blob/main/docs/issueLogRoot.md`

问题标题	问题描述	解决方案
AddAccount 函数抛出空指针异常	当 AddAccount 函数访问完 MPT 后，如果 MPT 并没有添加新的树节点，那么程序就会抛出空指针异常。	每次调用 AddAccount 函数时，检查 MPT 是否被添加了新的树节点。

10.2 PBFT 共识间隔参数的异常设置

2024.10.22 测试：2024 年 9 月提交在 GitHub 的代码可以将 PBFT 出块间隔设置为 0 且正常运作。我们将在这一节分享这部分代码的曾经遇到的问题以及它的解决方案。

10.2.1 问题描述

PBFT 共识协议的出块间隔，即 Block_Interval 参数被设置为 0 或很低的值时，该分片节点收到的消息可能会乱序。这种情况会导致错位的区块无法通过区块合法性验证，进而导致共识中止。

10.2.2 引发原因 - BlockEmulator 中的消息处理逻辑

BlockEmulator 被设计为一个事件驱动（event-driven）型的模拟器。

当共识节点收到消息时，它会触发消息处理函数“handleMessage”，根据消息头“msgType”的类型，选择处理消息的函数（handlePrePrepare，handlePrepare，handleCommit）。

引发原因：在之前存在 Bug 的版本中，节点并没有使用 go routine 并发处理消息。也就是说，节点会按照接收消息的顺序来依序处理消息。但

是，在“Block_Interval”参数很小的情况下，消息的到达顺序是无法预测且不合人意的，所以消息错位就会导致 PBFT 共识的处理逻辑错误。

比如在收到 PrePrepare 消息之前收到了同一个 proposal 的 Commit 消息，那么节点先处理 Commit 消息，则会发现无法识别到这个 proposal，所以 handleCommit 函数无法帮助节点 commit 这个 proposal。哪怕之后处理了该 proposal 的 PrePrepare 消息，由于上述 Commit 消息已经处理过了，节点不会再触发 handleCommit 函数。这会导致该节点迟迟无法完成 commit 操作。

```
// 旧版本存在 BUG 的 BlockEmulator 的消息处理函数

func (p *PbftConsensusNode) handleMessage(msg []byte) {
    msgType, content := message.SplitMessage(msg)
    switch msgType {
    // pbft inside message type
    case message.CPrePrepare:
        p.handlePrePrepare(content)
    case message.CPrepare:
        p.handlePrepare(content)
    case message.CCommit:
        p.handleCommit(content)

        // ......
}
```

10.2.3 解决方案

该问题的解决方案主要是利用 Go routine 和几个 PBFT 的字段来控制 PBFT 的消息处理顺序，防止共识节点由于消息错位导致共识失败。

利用 Go routine 并行的消息处理

在当前已修复 bug 的版本中，共识节点在处理 PrePrepare，Prepare，Commit 消息时，它会使用 go routine 启动一个线程，并发地处理消息。

```
// 已修复 BUG 的 BlockEmulator 的消息处理函数
```

```
// 该函数利用 Go routine 处理各个 PBFT 共识消息

func (p *PbftConsensusNode) handleMessage(msg []byte) {
    msgType, content := message.SplitMessage(msg)
    switch msgType {
    // pbft inside message type
    case message.CPrePrepare:
        // use "go" to start a go routine to handle
            this message, so that a pre-arrival
            message will not be aborted.
        go p.handlePrePrepare(content)
    case message.CPrepare:
        go p.handlePrepare(content)
    case message.CCommit:
        go p.handleCommit(content)
        // ......
}
```

用于控制 PBFT 消息处理顺序的辅助字段

从代码上看，程序是由这 3 个字段来控制 PBFT 共识协议的进度（pbftStage，pbftLock，conditionalVarPbftLock）。其中，变量 conditionalVarPbftLock 使用了 sync.Cond，关于 go 中条件变量（conditional variable）的使用可以见：https://pkg.go.dev/sync#Cond。

简而言之，conditional variable 能够同时释放多个被阻塞的线程，BlockEmulator 用它保证节点收到的消息被按序执行。

```
type PbftConsensusNode struct {
    // ......
    // pbft stage wait
    pbftStage                atomic.Int32 //
        1->Preprepare, 2->Prepare, 3->Commit, 4->Done
    pbftLock                 sync.Mutex
    conditionalVarpbftLock sync.Cond
    // ......
}
```

接下来介绍这三个变量在代码中的作用。

1. pbftStage

这个变量用来表示当前 PBFT 共识处于哪个阶段。或者说，这个变量表明了目前该节点可以处理哪些消息（1->Preprepare,2->Prepare,3->Commit,4->Done）。

2. pbftLock & conditionalVarpbftLock

   这两个变量是需要一起使用的。在创建 PbftConsensusNode 时，代码会将 pbftLock 和 conditionalVarpbftLock 绑定。

```
// 以 &p.pbftLock 为参数创建 p.conditionalVarpbftLock
p.conditionalVarpbftLock = *sync.NewCond(&p.pbftLock)
```

节点的消息处理逻辑

当节点收到一个消息后，它会启动一个 go routine 线程，来处理消息。

```
func (p *PbftConsensusNode) handlePrepare(content []byte) {
    // 对消息进行解码
    // ......
    // 控制消息按照共识流程的顺序被处理
    curView := p.view.Load()
    p.pbftLock.Lock()
    defer p.pbftLock.Unlock()
    for p.pbftStage.Load() < 2 && pmsg.SeqID >= p.sequenceID && p.view.Load() == curView {
        p.conditionalVarpbftLock.Wait()
    }
    defer p.conditionalVarpbftLock.Broadcast()

    // if this message is out of date, return.
    if pmsg.SeqID < p.sequenceID || p.view.Load() != curView {
        return
    }

    // 该类消息的处理逻辑
    // ......
    // 如果收到足够多的 PrePare 消息，那么进入 Commit 阶段
    p.pbftStage.Add(1)
}
```

以处理 prepare 消息的函数为例，这个线程可能会受到阻塞. 第 6 行使用 pbftLock 进行了加锁，第 9 行使用 conditionalVarpbftLock.Wait() 进行阻塞。也就是说，如果第 8 行 for 循环的条件判断为 true，那么这个进程将会被 Wait() 方法阻塞。

- 首先是第 8 行的 for 循环条件。p.pbftStage.Load()<2 这个条件确保了，共识节点处于 pbftStage = 1（preprepare）阶段时，Prepare 消息不会被处理（如果被处理就相当于消息乱序了）。pmsg.SeqID>= p.sequenceID 条件确保了，p.pbftStage.Load()<2 这个条件阻塞的仅是当前这一轮 PBFT 共识的消息。

- 如果一个消息处理函数跳出了 for 循环，它会首先检查该消息是否已经过时（out of date，由第 14 行的代码进行判断），如果过时则直接结束函数，否则按照该类消息的处理逻辑处理。

- 在该类消息处理逻辑的末尾，如果节点可以进入下一阶段（比如，节点已经收到了当前 PBFT 轮次的足够多的 Prepare 消息），那么第 23 行代码会触发 p.pbftStage.Add(1)。

- 最后，由于第 11 行使用了 defer 指令（deferp.conditionalVarpbftLock.Broadcast()），在程序返回时，该程序还会用 Broadcast() 方法，解除其他所有线程的 wait() 阻塞。这样，其他进程会再次进行第 8 行的 for 循环判断，并且尝试跳出循环。

为了方便理解，我们提供了图 10.1 作为解决方案的解释。

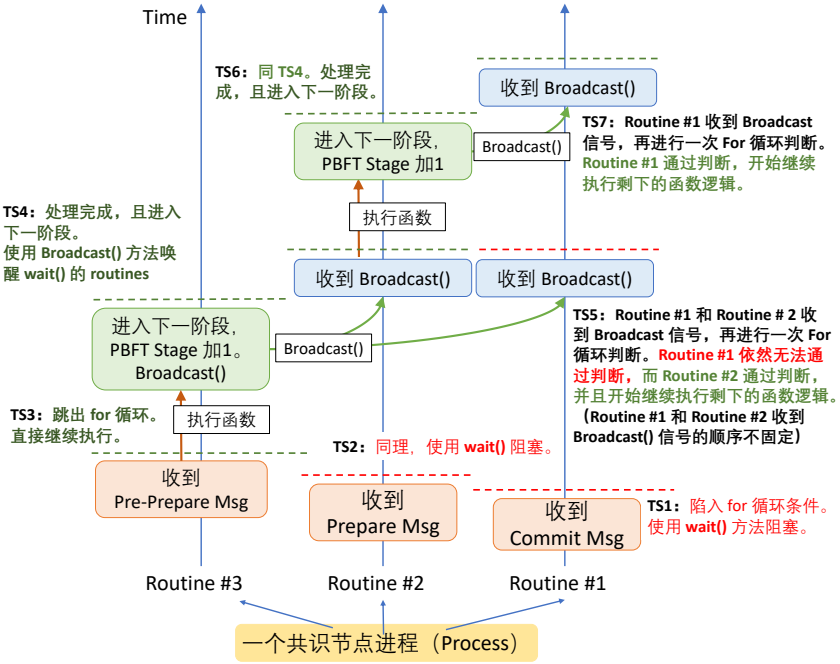


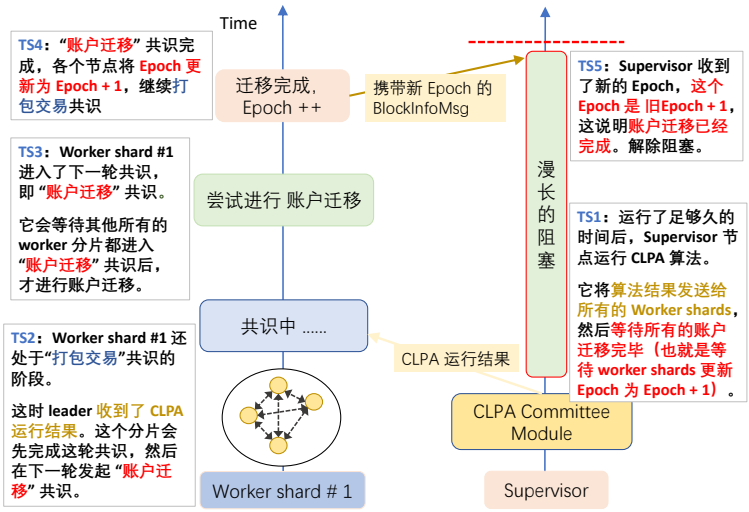
图 10.1: 使用 Go Routine 解决 PBFT 消息乱序的方案。图中的 TS 是 time stamp 的缩写，表示事件发生的时间。

10.3 CLPACommitteeModule 出现死循环

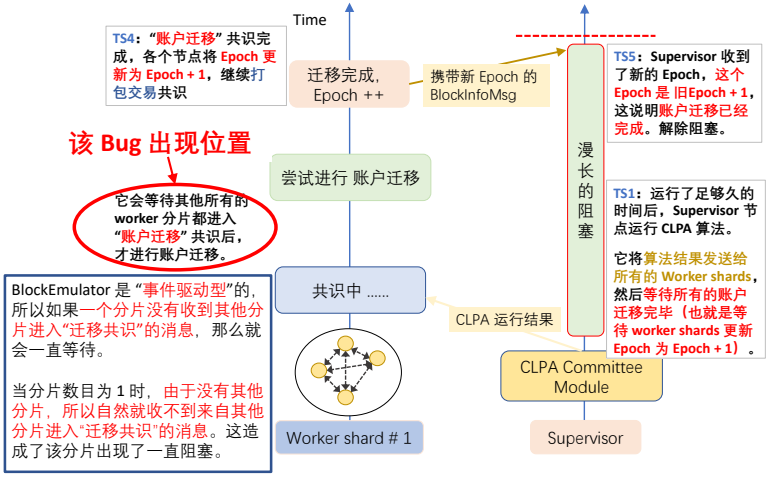
该 issue 链接为:https://github.com/HuangLab-SYSU/block-emulator/issues/42，感谢这位提问者提出问题并且给出了对应的解决方案!

10.3.1 问题描述

使用 CLPA 类算法跑实验时，如果将分片数目设置为 1，那么 CLPACommitteeModule 会卡住不动。



(a) CLPA 共识协议中，节点的正常运行逻辑



(b) CLPA 协议触发这个 bug 的原因。

图 10.2: CLPA 共识协议运行时，节点的运行逻辑以及 bug 触发的原因。

10.3.2 代码定位

如图 10.2 中描述的代码逻辑与 bug 出现原因表现的那样，CLPA 类方法的代码执行逻辑是这样的：

Supervisor 执行 CLPA 算法，然后将结果分发给 worker 分片，最后阻塞自己。然后 worker 分片根据 CLPA 算法的结果更新本地数据、进行账户迁移。这一系列操作完成后，worker 分片会将自己本地的 epoch 自增 1。然后，在下一次出块时，worker 分片会将区块信息发送给 Supervisor，其中包含了自己当前的 epoch。而 supervisor 的 `CLPACommitteeModule` 收到了 worker 发送的 `BlockInfoMsg`，发现 worker 分片已经进入了新的 Epoch，所以 supervisor 也更新本地的 `ccm.curEpoch`。此时，`ccm.curEpoch==clpaCnt`，supervisor 跳出阻塞条件，继续执行。

下面是结合代码的解释：卡住不动的代码位于下方代码块的第 15 17 行。在分片数目为 1 的情况下，`ccm.curEpoch` 始终不等于 `clpaCnt`，所以代码陷入了死循环。

```
1       // 原先出现 Bug 的代码位于
            supervisor\committee\committee_clpa.go
2   if !ccm.clpaLastRunningTime.IsZero() &&
        time.Since(ccm.clpaLastRunningTime) >=
        time.Duration(ccm.clpaFreq)*time.Second {
3       ccm.clpaLock.Lock()
4       clpaCnt++
5       mmap, _ := ccm.clpaGraph.CLPA_Partition()
6
7       ccm.clpaMapSend(mmap)
8       for key, val := range mmap {
9           ccm.modifiedMap[key] = val
10      }
11      ccm.clpaReset()
12      ccm.clpaLock.Unlock()
13
14      for atomic.LoadInt32(&ccm.curEpoch) !=
            int32(clpaCnt) {
15          time.Sleep(time.Second)
16      }
17      ccm.clpaLastRunningTime = time.Now()
18      ccm.sl.Slog.Println("Next CLPA epoch begins. ")
19  }
```

那么，ccm.curEpoch 为什么无法等于 clpaCnt?

在下面这份代码的第 5 行发生了 ccm.curEpoch 的修改，从中可见，ccm.curEpoch 是通过 BlockInfoMsg 的 Epoch 字段进行修改。所以，如果 BlockInfoMsg的 Epoch 字段一直没有修改，那么 ccm.curEpoch 也不会被修改，上述的死循环就会出现。

```
1 // 对 ccm.curEpoch 产生修改的代码位于
      supervisor\committee\committee_clpa.go
2
3 func (ccm *CLPACommitteeModule) HandleBlockInfo(b
      *message.BlockInfoMsg) {
4     ccm.sl.Slog.Printf("Supervisor: received from
          shard %d in epoch %d.\n", b.SenderShardID,
          b.Epoch)
5     if atomic.CompareAndSwapInt32(&ccm.curEpoch,
          int32(b.Epoch-1), int32(b.Epoch)) {
6         ccm.sl.Slog.Println("this curEpoch is
              updated", b.Epoch)
7     }
8     ......
9 }
```

为什么 BlockInfoMsg 的 Epoch 字段没有被修改？因为 BlockEmulator 是事件驱动型（event-driven）。从下面的代码可以看到，（leader 节点）代码会卡死在第 6 行的循环里。当 leader 节点卡死后，view-change 就会触发，选举出新的 leader，新的 leader 忽视上一轮的账户迁移，所以 Epoch 保持不变。不变的 Epoch 导致了 Supervisor 端的死循环。

```
1 // 该代码位于
      consensus_shard\pbft_all\pbftInside_moduleCLPA.go
2
3 func (cphm *CLPAPbftInsideExtraHandleMod)
      HandleinPropose() (bool, *message.Request) {
4     if cphm.cdm.PartitionOn {
5         cphm.sendPartitionReady()
6         for !cphm.getPartitionReady() {
7             time.Sleep(time.Second)
8         }
9         // send accounts and txs
10        cphm.sendAccounts_and_Txs()
11        // propose a partition
12        for !cphm.getCollectOver() {
13            time.Sleep(time.Second)
```

```
            }
            return cphm.proposePartition()
        }
        ...
}
```

10.3.3 解决方案

最简单的解决方案是，仅当分片数目大于 1 时，才运行 CLPA 算法。

```
// 修复 Bug 的代码位于
    supervisor\committee\committee_clpa.go
if params.ShardNum > 1 &&
    !ccm.clpaLastRunningTime.IsZero() &&
    time.Since(ccm.clpaLastRunningTime) >=
    time.Duration(ccm.clpaFreq)*time.Second {
    ccm.clpaLock.Lock()
    clpaCnt++
    mmap, _ := ccm.clpaGraph.CLPA_Partition()

    ccm.clpaMapSend(mmap)
    for key, val := range mmap {
        ccm.modifiedMap[key] = val
    }
    ...
}
```

10.4 AddAccount 函数抛出空指针异常

10.4.1 问题描述

在使用 CLPA 算法时，共识节点在进行账户迁移时，调用 `AddAccount` 函数，而这个函数有概率会抛出异常。

10.4.2 原因分析

函数在调用 `./chain/blockchain.go` 文件中的 `AddAccount(ac []string, as []*core.AccountState, miner int32)` 函数时，如果**更新**

的账户状态已经存在于 MPT 中，那么方法 `err = bc.triedb.Update(trie.NewWithNodeSet(ns))` 就会抛出异常。

Tips：在 `AddAccount` 中，一共有 3 个参数，第一个参数 `ac` 指的是账户地址，第二个参数是 `as` 指的是账户状态，第三个参数是 `miner` 指的是当前的矿工。其中，第一个参数 `ac` 和第二个参数 `as` 是以数组的形式给出的，具有相同 index 的 `ac` 和 `as` 指的是同一个账户。比如，账户状态 `as[1]` 对应了账户地址 `ac[1]` 的状态，账户状态 `as[2]` 对应了账户地址 `ac[2]` 的状态。

10.4.3 问题复现

修复之前的代码情况

修复前的代码是这样的：

```
// 完整代码位于 chain/blockchain.go

func (bc *BlockChain) AddAccounts(ac []string, as
    []*core.AccountState, miner int32) {
    // ......

    // handle transactions to build root
    rt := bc.CurrentBlock.Header.StateRoot
    if len(ac) != 0 {
        // ......

        rrt, ns := st.Commit(false)
        err = bc.triedb.Update(trie.NewWithNodeSet(ns))
        if err != nil {
            log.Panic(err) // 由这一行抛出的错误
        }
        err = bc.triedb.Commit(rrt, false)
        if err != nil {
            log.Panic(err)
        }
        rt = rrt.Bytes()
    }

    // ......
}
```

修复前的代码仅仅是检查了 `len(ac) != 0`，防止 MPT 未更新导致的 `err = bc.triedb.Update(trie.NewWithNodeSet(ns))` 报错。

复现 bug 的测试代码

可以通过运行 `chain/blockchain_test.go` 中的 `TestBlockChain` 单元测试函数，来复现这个 bug。

```go
// 完整代码位于 chain/blockchain_test.go

func TestBlockChain(t *testing.T) {
    // 初始化账户
    accounts := []string{"000000000001",
        "00000000002", "00000000003", "00000000004",
        "00000000005", "00000000006"}
    as := make([]*core.AccountState, 0)
    for idx := range accounts {
        as = append(as, &core.AccountState{
            Balance: big.NewInt(int64(idx)),
        })

    // ......

    // 添加账户 & 账户状态
    CurChain.AddAccounts(accounts, as, 0)
    CurChain.PrintBlockChain()

    // 添加重复的账户 &
        账户状态（由于是重复的账户状态，所以底层 MPT
        不会更新）
    CurChain.AddAccounts([]string{accounts[0]},
        []*core.AccountState{as[0]}, 0)
    CurChain.PrintBlockChain()

    // ......
}
```

在**旧版本**中，这个测试用例不会导致 MPT 更新。但它绕开了 `len(ac) != 0` 的检查，从而导致报错。

运行单元测试代码 TestBlockChain 后，出现报错，报错信息如下：

```
--- FAIL: TestBlockChain (0.01s)
panic: runtime error: invalid memory address or nil
    pointer dereference [recovered]
        panic: runtime error: invalid memory address
            or nil pointer dereference
[signal 0xc0000005 code=0x0 addr=0x0 pc=0x7ff7bf8f42bc]

...

FAIL    blockEmulator/chain     0.141s
```

如此，我们便复现了该 bug 出现的场景。

修复之后的代码

修复之后的代码仅仅是加上了一行 ns!=nil 的判断：

```
// 完整代码位于 chain/blockchain.go

func (bc *BlockChain) AddAccounts(ac []string, as []*
    core.AccountState, miner int32) {
    // ......
    // handle transactions to build root
    rt := bc.CurrentBlock.Header.StateRoot
    if len(ac) != 0 {
        // ......
        rrt, ns := st.Commit(false)
        // 增加了一行 ns 不为 nil 的判断
        // if ns is nil, the err = bc.triedb.Update(
            trie.NewWithNodeSet(ns)) will report an
            error.
        if ns != nil {
            err = bc.triedb.Update(trie.NewWithNodeSet(
                ns))
            if err != nil {
                log.Panic(err)
            }
            err = bc.triedb.Commit(rrt, false)
            if err != nil {
                log.Panic(err)
            }
            rt = rrt.Bytes()
        }
    }
    // ......
}
```

再运行测试代码便不会出错：

```
go test -timeout 30s -run ^TestBlockChain$
    blockEmulator/chain
=== RUN   TestBlockChain
--- PASS: TestBlockChain (0.02s)
PASS
ok      blockEmulator/chain     0.144s
```

10.4.4 触发 bug 的场景复盘

上述讨论都是分析 bug 在区块链的数据层面是如何触发的。

而 BlockEmulator 中，触发这种 bug 并不是一个共识节点导致的，而是整个共识流程导致的。接下来我们将会利用图10.3 中的例子，分析可能出现这种 bug 的场景。

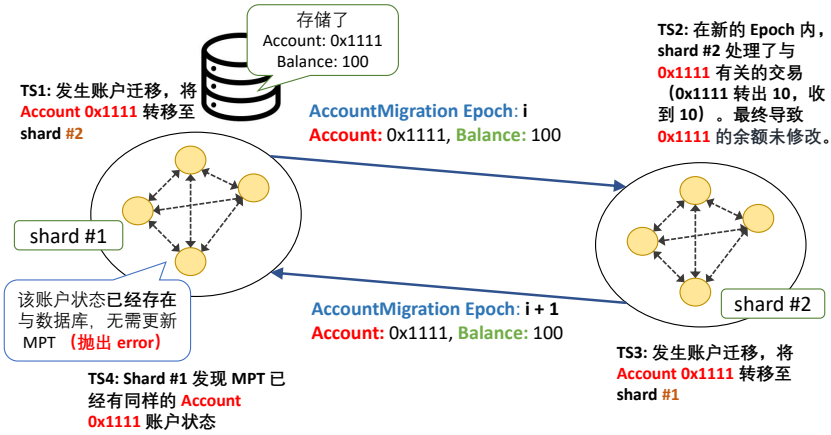


图 10.3: 一个抛出空指针异常的例子：代码中 MPT 更新账户余额失败。

1. 在上述场景中，一开始账户 *0x1111* 位于 shard #1 内。Shard #1 内的数据库存储了 `Account:0x1111` 的状态 `Balance:100`（该账户状态是简化后的版本）。

2. 在 TS1 时，区块链系统发生了账户迁移，账户 *0x1111* 将从 shard #1 迁移至 shard #2。

3. 在 TS2 时，账户 *0x1111* 已经位于 shard #2 中。Shard #2 的共识节点将会处理与 *0x1111* 有关的交易。这里举了一个例子，***0x1111* 转出 10，又收到 10**。这**将会导致 *0x1111* 的最终余额并没有发生变化**。（因为当前版本的 BlockEmulator 并没有考虑手续费。）

4. 在 TS3 时，区块链系统又发生了账户迁移，账户 *0x1111* 将从 shard #2 迁移回 shard #1。

5. 在 TS4 时，shard #1 内的共识节点利用 `AddAccount` 函数处理新到来的账户 *0x1111* 的状态 `Balance:100`。**`AddAccount` 函数发现这个状态已经存在于底层数据库中，所以不需要做出 MPT 的更新，故而抛出 error**。由此，造成了上述 bug 的触发。

第十一章　历史版本更新

开源至今，BlockEmulator 的各次版本迭代更新记录如下。

11.0.1　2026/May 13: BlockEmulator v2.0

2025 年下半年到 2026 年初，黄华威研究组的叶光同学重构了 BlockEmulator v1.0 的代码，推出一个进化的新版本——BlockEmulator-X [1]（后文某些地方简称为 BE-X）。这个版本（BE-X）的解释与使用方法，请参考章节 7。

此外，熊剑波同学整理了在 BE-X 上对智能合约给定用户自定义交易集进行测试的使用案例，请参考章节 8。

11.0.2　2026/May 12: Broker2Earn 分支

陈钦德同学（2026 年 5 月博士毕业）添加了 Broker2Earn （某些地方简称为 B2E）算法的使用说明与调用方法，并演示了如何在 BE v1.0 版本中调用 B2E 算法进行实验。请参考章节 6。

11.0.3　2024/11/14

1. **可执行文件被移动至 GitHub Release 中**。

[1] BE-X 的 GitHub 页面：`https://github.com/HuangLab-SYSU/block-emulator-x`

我们将预先编译好的可执行文件转移到了 GitHub 的 release 部分[1] 。这是因为，如果直接将可执行文件放到 GitHub 的 Code 部分，每次提交都会占用约 100 MB 的空间，导致 GitHub 仓库的空间消耗快速增长。因此，未来可执行文件将被放到 GitHub 的 Release 部分。从此以后，执行 `gitclone` 时，默认不会拉取这些文件。

2. **修复 bug**。

   我们修复了位于 `./chain/blockchain.go` 文件下 `AddAccounts` 函数内的一个 bug。这个 bug 导致了该函数在访问了 MPT 但没有更新它时，会抛出一个空指针异常。为了解决这个 bug，我们增加了一个检查 `newNodeSet` 是否为 `nil` 的判断。**如果您对这个 bug 的原因感兴趣，可以访问这个文件：AddAccount 抛出空指针异常** [2]。

11.0.4 2024/11/05

1. **新文档**: 我们在 GitHub 上新增了一个问题日志[3]。

11.0.5 2024/10/29

1. **修复 bug**: 我们发现在 `main.go` 中通过命令行定义的 `shardNum` 参数无法传递给 `params.ShardNum`。这个 bug 来源于 2024/09/01 的更新，它可能导致 CLPA 算法的错误的结果。**目前这个 Bug 已经被修复**。

[1] https://github.com/HuangLab-SYSU/block-emulator/releases

[2] https://github.com/HuangLab-SYSU/block-emulator/blob/main/docs/IssueLogs/addaccount_nil_pointer_dir/addaccount_nil_pointer.md)

[3] https://github.com/HuangLab-SYSU/block-emulator/blob/main/docs/issueLogRoot.md

2. **修复 bug**: 当用户设定 `shardNum=1` 并且使用 CLPA 相关的共识协议时, supervisor 节点可能会进入死循环。**目前，当分片数目为 1 时，CLPA 划分算法将不会触发**。我们对这个 bug 的发生场景进行了详细的分析，分析内容可以见 10.3 节。

3. **用来控制 Blocksize 的新方法**: 现在，用户可以根据需求灵活控制区块大小。具体来说，用户既可以以 **交易数目**为单位，也可以以 **字节数（bytes）**为单位控制 Blocksize。在 `paramsConfig.json` 文件中，用户通过设置 `UseBlocksizeInBytes` 参数即可选择具体的控制方式。当 `UseBlocksizeInBytes` 被设置为 1 时，共识节点在打包区块时将采用 **字节数**作为单位，并根据另一个参数 `BlocksizeInBytes` 指定具体的区块大小。

   (致谢: GitHub issue #40[1], GitHub issue #42[2])

11.0.6 2024/09/01

1. **IP 表**: 我们提供了一个 IP 表文件 `./ipTable.json` 用来简化 IP 配置操作。用户可以通过这个 IP 表修改每个节点的 IP 地址。请注意，`./ipTable.json` 中的第一级 Key 指的是 “ShardID”（即分片序号），第二级 Key 指的是 “NodeID”（即节点序号）。

2. **参数配置文件**: 我们提供了一个配置文件 `paramsConfig.json` 用来简化参数配置操作。用户可以在文件中设置不同的参数，包括共识方法，结果输出目录，数据来源文件等。

[1] `https://github.com/HuangLab-SYSU/block-emulator/issues/40`
[2] `https://github.com/HuangLab-SYSU/block-emulator/issues/42`

3. **分片内的 PBFT view-change 机制**: 我们更新了一个分片内的 PBFT view-change 机制。由于跨分片通信机制的复杂性，目前这个机制仅支持一个分片内部的 view-change。

11.0.7 2024/07/19

1. **用户手册**: 我们提供了中文和英文版本的用户手册，以便于用户可以更快地上手 **BlockEmulator**。

2. 三个 **Python 画图示例代码**: 在运行完 **BlockEmulator** 并且收集了实验结果之后，用户可以使用 `./figurePlot` 文件夹下的代码进行绘图。**请注意，画图代码中的文件路径是基于 BlockEmulator 根目录 `./` 的相对路径，所以请在 `./` 目录下运行画图代码**。

11.0.8 2024/07/12

1. **更细化的测试指标**: 我们增加了更细节的测试指标，从而让用户可以观测更加**细粒度**的实验结果。这些指标可以在 `params.DataWrite_path` 路径下找到（在 `./params/global_config.go` 中可以找到这个路径的定义）。

11.0.9 2024/07/10

1. **用户自定义的实验结果输出目录**: 用户目前可以自定义实验结果的输出路径了。用户可以通过命令行的 `-d` 和 `--dataRootDir` 选项控制输出目录。

   下面是两个例子:

```
1     1) go run main.go -g -d TestDir
2     2) go run main.go -n 1 -N 4 -s 3 -S 4 -m 3
          --dataRootDir TestDir
```

Tip: 在 2024/09/01 的更新后，实验结果的输出目录被移到了配置文件 `paramsConfig.json` 中。

11.0.10 2024/07/06

1. **可执行文件**: 我们为 **BlockEmulator** 新增了 **预编译的可执行文件**。利用这些可执行文件，用户可以直接运行 BlockEmulator，而无需提前下载 *Golang* 和相关的库文件。此外，用户还可以自行编译源代码，我们也提供了编译源代码的脚本。编译 Golang 代码的脚本位于 `./zPrecompileScripts` 文件夹中。

2. **运行可执行文件**: 下面三种操作系统的可执行文件分别是:

   1. **Windows**: `./blockEmulator_Windows_Precompile.exe`
   2. **Linux**: `./blockEmulator_linux_Precompile`
   3. **MacOS**: `./blockEmulator_darwin_Precompile`

   用户可以利用这些可执行文件生成一个 **可执行文件的批处理脚本**。例如，Windows 系统下可以执行这个命令：`./blockEmulator_Windows_Precompile.exe -S 2 -N 4 -g --shellForExe`

   这里, `--shellForExe` 表明生成的批处理脚本是针对 **可执行文件**的。然后, `WinExe_bat_shardNum=2_NodeNum=4_mod=CLPA_Broker.bat` 文件便生成了，用户可以在 Windows 系统上运行它。

3. **兼容性**: 本次更新不会影响任何现有代码。

11.0.11 2024/07/01

1. **更改数据集**：我们用新的数据集 `./selectedTxs_300K.csv` 替换了原先的数据集 `./TestTx_1M.csv`。新的数据集内的交易来自于以太坊的真实历史数据。与历史交易有关的算法和协议在这个数据集上能表现得更好。

11.0.12 2024/06/12

1. **Fine-tune Lock**: **Fine-tune Lock**(INFOCOM'24) 的源码已经被加入到 **Fine-tune Lock** 分支中。

11.0.13 2024/06/11

1. **修复 Bug**: 修复了 PBFT 共识协议中的一些 bugs，这些 bugs 可能导致共识节点丢弃那些由于不稳定网络而收到的乱序消息。

11.0.14 2024/05/20

1. **新增数据集**：我们提供了一个数据集 `./TestTx_1M.csv`，用户可以在这个数据集上快速开始实验。

11.0.15 2024/03/01

1. **新增 FAQ**: 我们增加了 FAQ.md [1]，其中记录了一些常见的问题。

[1] `https://github.com/HuangLab-SYSU/block-emulator/blob/main/docs/FAQ.md`

11.0.16 2023/12/12

1. **新特性**: 我们为 Linux 系统新增了 shell 生成函数。当用户使用命令: `go run main.go -g`，代码将会生成.bat 和.sh 文件。

11.0.17 2023/07/21

1. **问题**: 账户迁移操作可能导致交易池的顺序被打乱。

   • **原因**: 原先的交易迁移算法虽然节省了遍历的开销，但是将交易池的顺序打乱了。

   • **解决方案**: 我们重写了交易迁移算法，这样，它能在不影响开销的情况下，保证交易池的顺序。

11.0.18 2023/07/18

1. **问题**: 当分片数目或节点数目过多时，TCP 连接将会报错。

   • **原因**: 更新之前，由于 TCP 连接被实现为短连接形式，每次数据发送后连接都会立即关闭。这导致两个共识节点之间会同时存在大量**未完全关闭**的 TCP 连接，这些连接处于 TIME_WAIT 状态。长时间运行 BlockEmulator 后,由于大量 TCP 连接处于 TIME_WAIT 状态，大量端口被占用，导致新的 TCP 连接无法建立，从而引发程序报错。

   • **解决方案 1**: 我们将短连接替换为了长连接（*keep-alive connection*），从而避免了 TCP 频繁地被建立和关闭。简单来说，我们设计了一个连接复用池，两个共识节点的 TCP 连接会被不断复用，直到系统运行完毕。

• **解决方案 2**: 用户可以将命令拆分，并且在多台机器上运行 BlockEmulator。相关的 IP 设置可以在 `params/global_config.go` 和 `build/build.go` 中定义。

（**在 2024/09/01 的更新后，用户应该在 `ipTable.json` 中定义各个节点的 IP 地址**。）

11.0.19 2023/06/22

1. **问题**: PBFT 共识协议会在大规模实验时中断（数据集很多或分片数目很多的情况）。

   • **原因**: 原先的 `HandleforSequentialRequest` 函数实现有 bugs，这会导致 PBFT 被乱序地接收，并且被错误地处理。

   • **解决方案**: 我们修复了函数内逻辑上的错误，目前这个函数可以正确地工作了。

11.0.20 2023/05/25

1. **新特性**: 我们在 `./query` 目录下新增了查询函数。

   当实验运行完后，用户可以使用 `./query` 目录下的函数检查运行的正确性，包括账户余额，区块信息和交易信息。

11.0.21 2023/05/24

1. 2023/05/21 修复 bug 的 **下一步工作**

   • 2023/05/21 的更新: **计划的下一步工作**: 我们将会在不使用“虚拟交易”的情况下修复这个 bug，因为当前的实现方案比较耗时。

- 目前，这个函数 `blockChain.AddAccount` 可以正确地运行，并且不调用 `GetUpdateStatusTrie`，这样程序的内存和时间开销更小了。

11.0.22 2023/05/21

1. **问题**: 函数 `blockChain.AddAccount` 无法作用于底层数据库存储。

   - **原因**: 这个函数的实现无法修改 Merkle root。

   - **解决方案**: 我们使用“虚拟交易”的方式来替代 `AddAccount` 操作。对于每一个新增的账户，程序会先生成一笔虚拟交易，然后使用 `GetUpdateStatusTrie` 函数来处理虚拟交易，这样就可以比较曲折地完成添加账户的操作了。

   - **计划的下一步工作**: 我们将会在不使用“虚拟交易”的情况下修复这个 bug，因为当前的实现方案比较耗时。

2. **问题**: 在 CLPA + Broker 共识方法中，交易被迁移到错误的分片。

   - **原因**: 判断账户归属于哪个分片的函数错误，这导致了错误的划分结果。

   - **解决方案**: 我们在 Broker 交易中新增了 `SenderIsBroker` 字段，用来标识交易的发送方是否是 Broker 账户。我们利用这个新增变量，修改了 `accountTransfermod_Broker.go` 文件中的 `sendAccounts_and_Txs` 函数，解决了错误划分的问题。

11.0.23 2023/05/19

1. **问题**: 某些账户的余额初始化错误.

- **原因**: 更新之前, 账户余额的初始值是通过一个浅拷贝指令 (“=”) 来定义的, 而 Balance 是一个指针 (`*big.Int`), 这会导致账户余额被错误修改。
- **解决方案**: 我们将浅拷贝替换为了深拷贝（通过 “adding-zero” 的方式)。

第十二章　用户社区的建设与维护

12.1　微信群社区的问题解答

2023 年 5 月我们开源了 BlockEmulator，期望它可以为尽量多的区块链研究者带来便利。为此，我们设置了一个微信群（群名称：BlockEmulator 交流群）。感兴趣的研究者可联系 HuangLab 助理杨青林博士（WeChat ID: Qinglinyang ）联系加入社群。

自项目开源以来，有一些用户已经加入了我们的微信群（截至 2026 年 5 月 11 号，群里有 463 名成员），并且他们在群里提出了一些问题和建议。在这一节中，我们会整理一部分微信群中曾经被问到过的典型问题以及研究组成员的解答，供大家参考。

- **2024/11/13**：群里的两位用户反馈说，在数据集较大时，使用 BlockEmulator 的 CLPA 相关共识协议，会出现报错。（用户附上了报错的截图，其中大意是 “无效的内存地址和空指针解引用”。）

  问题处理经过：起初，我们通过截图中的报错信息，定位到了出现报错的代码，但是反复进行测试后，并没有复现出这个 bug。通过与用户的沟通，我们得知这个 bug 是在实验数据集很大时才触发的。因此，我们调大了数据集并且重新进行了测试，并且复现出了这个 bug。后来，我们发现这个问题发生在 CLPA 共识算法的迁移过程中，这时节点调用的函数 `addAccount` 触发了对空指针的访问。这个报错的触发是由于我们对以太坊 MPT 的不恰当调用，我们将这个 bug 进

行了尽快地修复，而且将错误的具体信息和出现场景记录在了章节 10.4 中。

- **2024/8/21**：群里的一位用户反馈说，他在 BlockEmulator 中注入了一百万条交易，但是最终只跑了 5 个 epochs，而且 5 个 epochs 仅仅处理了大约八十万条交易。因此，这位用户猜测可能是 epoch 总数被固定在了 5 个，导致最终没有跑完所有交易。

  关于设置 Epoch 的解释：我们向用户解释了 BlockEmulator 的 epoch 数目并非被固定为 5 个。事实上，epoch 是按照时间（50 秒）递增的，也就是每 50 秒一个 epoch。这里的 50 秒是通过 `./paramsConfig.json` 里的 `ReconfigTimeGap` 来设置的。

  交易缺漏的原因：后续，通过我们和用户的共同排查，发现这位用户所使用的数据集内有一百万条交易。但是，BlockEmulator 在注入交易时，会将智能合约交易筛去，这导致了实际注入交易的数目仅有大约八十万条，这导致了所谓的 “交易缺漏” 现象。

- **2024/6/25**：一位用户在运行 BlockEmulator 时，出现了报错 “error tx”。这位用户根据报错信息定位到了代码的具体行数：`consensus_shard/pbft_all/accountTransfer_module.go:145` 文件中的 `proposePartition` 函数（第 144 和 145 行）。

```go
// 代码位于 consensus_shard\pbft_all\accountTransfer_module.go 文件中。
func (cphm *CLPAPbftInsideExtraHandleMod) proposePartition() (bool, *message.Request) {
    // ......
    if !tx.Relayed && cphm.cdm.ModifiedMap[cphm.cdm.AccountTransferRound][tx.Sender] != cphm.pbftNode.ShardID {
        log.Panic("error tx")
    }
    // ......
```

用户的进一步解释：用户在 `consensus_shard/pbft_all/pbft.go:168` 这个位置（`handleMessage` 方法第一行）加了个网络时延模拟操作 `time.Sleep(time.Second)`，从而导致了上面的问题，如果不加这个操作的话就不会出现此问题。

我们的回复：经过我们对代码报错的分析，我们认为这个报错发生在“账户迁移”阶段。此问题是由于 sleep 了一秒，导致了消息处理速度过慢。这进一步地导致了，哪怕下一轮的“账户划分表”（partitionMap）已经更新，但是上一轮的交易却没有迁移完毕。这样的话，两次“账户迁移”阶段发生了重叠，代码无法处理这种意料之外的情况，才产生了报错。

我们的解决方案：我们新增了 CLPA 相关算法里的 CLPA Epoch 判断逻辑。具体而言，如果上一轮账户迁移没有结束，那么下一轮账户迁移便不会开始，这就避免了两次“账户迁移”阶段重叠引发的问题。

用户实现网络时延的错误之处：通过和用户沟通，我们发现，他之所以想要新增一行代码 `time.Sleep(time.Second)`，是为了模拟网络的延迟。然而，由于 `handleMessage` 部分的代码是串行执行的，如果在其中加入 `time.Sleep(time.Second)`，会导致第一个消息的延迟为 1 秒，第二个消息为 2 秒，第三个消息为 3 秒，以此类推。这种实现方式使时延不断累加，与用户预期的功能完全不符。

我们后续的更新：正确的实现方式是使用 Go Routine 来控制网络时延。由于 Go Routine 是**并发**执行的，`time.Sleep(time.Second)` 便不会因**串行**执行而导致累加，这就避免了上述问题。为此，我们更新了 BlocKEmulator 的代码，将网络延迟作为 `./paramsConfig.jso`

n 文件中的参数 Delay，让用户能够更方便地控制和模拟网络延迟，提升了系统的灵活性与用户体验。

- **2024/3/13**：一位用户在 BlockEmulator 用户群中发起提问：如何查看 ./expTest/database 文件里面的内容？在这个文件中，数据库文件都是乱码，用户是否需要使用其他的读取数据库的软件呢？

  我们的回答：./expTest/database 中的文件确实并不能直接阅读。事实上，BlockEmulator 是使用数据库的接口来对这些文件进行增删查改。我们确实考虑到了用户在做完实验后想要检查数据库文件的场景，因此，我们特地写了 ./query 文件中的内容。用户可以根据这个文件中的内容，了解到 BlockEmulator 是如何利用数据库接口与数据库进行交互的。除此之外，./query 内还有着一些单元测试代码，用户还可以借助这些代码检查 BlockEmulator 运行的正确性。

  具体而言，单元测试代码会: i) 读取数据集文件，并且将会单机执行改文件内的交易，计算出每个账户的最终余额。ii) 读取 BlockEmulator 运行后的 database 文件，从 database 文件中读取每个账户的余额。iii) 比较上述二者是否相同，如果 BlockEmulator 运行正确，那么上述二者应该是相同的。

12.2 GitHub 项目 Issues 页面的问答

自 2023 年 5 月我们在 GitHub 上公开 BlockEmulator 项目源代码以来，不少用户也在 GitHub 项目 issues 页面上提出了很多问题。这些问题无一例外都促进了 BlockEmulator 的更新和完善。感谢所有曾经向我们发送过反馈的用户。与用户一起共建这个项目，这也是我们研究组期望看到

的情景。在这一章节，我们挑选出了一些 GitHub issues 上一些有代表性的问题，并给与详细的回复与解释。供读者参考。

- **Issue #42**[1]：

  问题概述：运行 CLPA 相关的算法时，如果分片数目设置为 1，则 BlockEmulator 会出现奇怪的死锁现象。目测是 partition 部分的代码所导致的死锁。

  解决方案：问题的提出者在提出问题时，也提供了解决方案：在运行 partition 前先进行判断，如果分片数目为 1，那就不执行 partition。

  我们的后续审查：我们十分感谢这位用户提出了这个 bug 并且提供了对应的解决方案。我们也很好奇这个 bug 出现的原因，为此，我们重新对 partition 这一部分的代码进行了审查。经过审查，我们发现，BlockEmulator 出现这一问题的原因不仅仅是 partition 这一模块的原因，它还涉及到了账户迁移的共识过程。为此，我们特地撰写了 10.3 节来详细地阐述这个问题的触发原因。

- **Issue #41**[2]：

  问题概述：看起来 BlockEmulator 中使用了 Rate Limiting [3] 库来模拟网络带宽（我不确定这是否正确）。在 Linux 中，我们可以使用 traffic control [4] 来实现这个功能。

  我们的答复：我们确实使用了 Go 的 Rate Limiting 库来模拟网络带宽的限制。由于我们提供的代码并不能影响操作系统，所以 Rate

[1] https://github.com/HuangLab-SYSU/block-emulator/issues/42
[2] https://github.com/HuangLab-SYSU/block-emulator/issues/41
[3] https://go.dev/wiki/RateLimiting
[4] https://github.com/thombashi/tcconfig

Limiting 是我们的一种解决方案。对于用户而言，直接控制操作系统的流量确实是一个更好的选择。

追问：您已经在应用程序级别限制了带宽（使用 Go 的 Rate Limiting 库）。我担心这是正确的做法吗？如果我通过 Rate Limiting 库将每台机器的带宽限制在 100kbps，这台机器真的有 100kbps 吗？使用 Rate Limiting 库后如何获取机器的实际带宽？

我们的答复：

i) 网络带宽管理通常使用操作系统级工具，并需要管理权限。BlockEmulator 作为模拟器，不应该具备直接影响某台机器真实带宽的功能，这样可能会很危险。因此，尽管 Rate Limiting 库在模拟带宽控制方面可能不太准确，但我们依然选择它为应用程序级别的解决方法。

ii) 如果您用 Rate Limiting 库限制了某台机器的带宽为 100 kbps，实际上这台机器并不会只有 100 kbps。事实上，Rate Limiting 库仅仅是限制了程序对 TCP listener 的写入速度。因此，使用 Rate Limiting 库时，BlockEmulator 不会影响机器的实际带宽，而只会在应用层控制对某个 iostream 的读写速度。

iii) 使用 Rate Limiting 库后，您可以使用一些**网络监控工具**来获取机器的真实带宽，比如 iperf，netstat 等。

- **Issue #40**[1]：

问题概述：修改 PackTxs 函数，当达到一定大小时停止，是否更合理？（问题提出者在问题的详情区域附上了他实现的 PackTxs 函数。）

[1] https://github.com/HuangLab-SYSU/block-emulator/issues/40

我们的答复：感谢您的建议！在目前的 BlockEmulator 中，我们使用 BlockSize，也就是一个区块中的交易数目来限制区块大小。而您的代码通过区块的数据大小（以 Bytes 为单位）来限制区块大小，这确实是一种更合理的方案。我们会将这种实现方式添加到开源代码中，以便用户根据自己的需求选择具体的方法。

后续的更新：我们将“以 Bytes 为单位限制区块大小”作为了 2024/10/29 的更新内容。

- **Issue #33**[1]：

  问题概述：请问为什么 Relay 模式下，设置固定数量的分片，改变节点数量时，TPS 和延迟变化很小很小呢?

  我们的答复：您好，感谢您的问题。在分片区块链中，区块链的分片数目等同于并行数目，固定分片数目，也就固定了系统的并行程度，所以哪怕变化分片内的节点数目，TPS 和延迟也变化较小。一个分片内的节点数目关乎该分片的安全性，换言之，一个分片内节点数目越多，便能够容忍更多恶意节点，相对来说也就越安全。

  后续的补充：节点数量增大时，分片内的 PBFT 耗时应该也会随之增加。但是据提问者所说，TPS 和延迟变化很小。这可能是因为实验是在单机环境下运行，此时 PBFT 的网络共识开销并不会受到带宽的影响，所以导致了提问者所说的现象。

- **Issue #25**[2]：

  问题概述：运行 BlockEmulator 时出现了这个问题：cannot use 0x7fffffff

[1] https://github.com/HuangLab-SYSU/block-emulator/issues/33
[2] https://github.com/HuangLab-SYSU/block-emulator/issues/25

(untyped int constant 34359738367) as int value in assignment (overflows)。

我们的答复：抱歉，错误出现在 `.\partition\partition_CLPA.go` 文件夹中的 160 行，我们使用了 0x7fffffff 来表示 INT_MAX，但是此处多打了一个 f，超出了 32 位 int 的表示上限。这导致了该行代码在某些系统上会报错。目前我们已经修改了此部分代码为 math.MaxInt，请您拉取最新的代码。

- **Issue #21**[1]：

  问题概述：请问 Broker 的地址和 `broker` 文件下的数据有什么关系？`broker/broker` 文件里的数据是如何生成的？

  我们的答复：该文件内记录的是 Broker 账户的地址，这些账户的地址是从数据集中选出的。您可以根据自己采用的数据集，对这个文件里的内容进行替换，从而让不同的账户担任 Broker。

- **Issue #14**[2]：

  问题概述：请问这个问题是什么原因造成的？我运行了一次之后，改了一下参数，又运行了一次。就出现了这个报错。难道不能多次运行程序吗，如果想多次运行程序怎么办。（提问者附上了报错的截图，图中大意是“无法获取到资源”（新建文件失败）。）

  另一位用户的答复：我认为可能是因为没有删除之前的日志文件的影响。您可以尝试在运行程序之前删除 `log,record` 文件夹。

  我们的后续改进：这位提问者由于没有删除历史数据，导致第二次运行 BlockEmulator 时发生了报错，这是较为常见的。BlockEmulator

[1] `https://github.com/HuangLab-SYSU/block-emulator/issues/21`
[2] `https://github.com/HuangLab-SYSU/block-emulator/issues/14`

在实验结束后依然会保留历史数据，这样有利于用户对数据进行审查、绘图等操作。然而，BlockEmulator 的实验结果默认存储在一个文件夹中，因此在第二次运行时容易引发文件命名冲突的问题。我们意识到这会影响用户体验，因此将实验结果的输出路径设置为参数 `ExpDataRootDir`，并放到了 `./paramsConfig.json` 中，方便用户根据需求自行配置。

12.3 用户社区其他常见问题的汇总

12.3.1 关于 BlockEmulator 设计方面的问题

a. BlockEmulator 如何保证每个分片内共识节点的状态一致性？

在 blockEmulator 的默认配置中，同一个分片的所有共识节点都运行着同一个版本的 PBFT 共识协议，如何保证这些节点的状态一致性？为了回答这个问题，我们提供了图 12.1 作为解释。

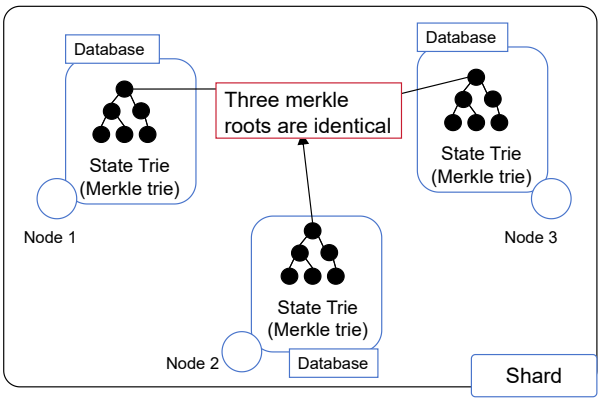


图 12.1: 同一个分片的共识节点通过验证状态树的 merkle root，来判断区块是否正确。

在图 12.1 中所示的分片区块链系统中，每个节点本地都维护着一个状态树（state trie），该树采用 merkle tree 结构。由于哈希函数的抗碰撞性，如果各个节点的状态树根（merkle root）一致，那么我们就可以认为各个节点所维护的状态树一致。反之，如果两个节点的状态树的 root 不同，那么这两个节点在各自本地维护的状态树就是不一样的。

BlockEmulator 的 BlockHeader 数据结构正是包含了“状态树根”(StateRoot）这一字段，它可以用来验证各个共识节点的世界状态树是否一致。

在源码中，区块头 BlockHeader 的数据结构是这样的：

```
// 代码位于 core\block.go 文件下
type BlockHeader struct {
    ParentBlockHash []byte
    StateRoot       []byte
    TxRoot          []byte
    Bloom           bitset.BitSet
    Number          uint64
    Time            time.Time
    Miner           int32
}
```

每个节点在生成新区块时，会将最新的状态树根（state root）记录到区块头的 StateRoot 字段中。当其他节点接收到该新区块后，会根据区块内的交易对本地状态树进行更新，并将更新后的本地状态树根与新区块区块头中的 StateRoot 字段进行比较。如果两者相等，节点才会认为该新区块是正确的。

此外，由于区块链采用链式结构，如果某一高度出现两个不一致的区块，这种不一致性会通过区块头中的 ParentBlockHash 字段传播到后续区块，导致后续区块也出现不一致。因此，区块链的不一致性会随着链式结构向下传递，从而通过这一特性进一步确保了区块链的历史状态一致性。

所以，BlockEmulator 是通过状态树的 merkle root 来保证同一个分片内各个节点的状态一致性。

b. BlockEmulator 这个系统的正确性是如何评估的？

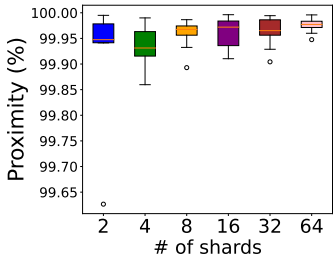
作为一个分片区块链实验平台，BlockEmulator 的结果必须确保正确性和可信度。为此，我们结合理论推导，进行了一组大规模实验，以充分验证 BlockEmulator 的准确性。

为了减轻不相关因素（如网络扰动、数据集分布偏斜等）对理论推导的影响，我们的实验配置如下：

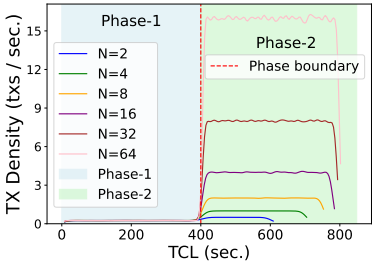
- 为了减轻网络扰动、路由转发等影响，该正确性验证实验将在 LAN 中进行。
- 为了避免数据集的偏斜分布影响 BlockEmulator 性能，我们生成了一个分布随机且均匀的数据集，确保每一个分片负载均衡。
- 我们使用静态的账户划分方法，因为动态的划分方法（如 CLPA）会带来更多的不确定性。
- 为了加强实验说服力，我们将测试两个指标 TPS（transactions per second）和 TCL（transaction confirmation latency）。

理论推导过程可以见 BlockEmulator 论文[1]，这里我们仅给出理论推导与实验结果的比较。图 12.2 呈现了 BlockEmulator 正确性验证实验中的 TPS 和 TCL 分布。在图 12.2(a) 中，BlockEmulator 实验结果的 TPS 与理论推导结果相当接近（99.9%），这说明了 BlockEmulator 的实验结果在 TPS 这个指标上是符合预期的。而图 12.2(b) 则说明了在 BlockEmulator 正确性验证这一实验中，交易在 phase-1 和 phase-2 内都符合均匀分布，这与我们在理论推导中的结果一致。这不仅说明了 BlockEmulator 的 TCL 指标符合预期，还表明了交易的处理流程是符合预测的。

[1] https://arxiv.org/abs/2311.03612



(a) BlockEmulator TPS 的实验结果与理论推导的接近程度 (%)。



(b) BlockEmulator 两个阶段的 TCL 实验结果分布图。

图 12.2: BlockEmulator 的正确性验证实验。

c. BlockEmulator 如何将区块链的基本操作和可替换共识协议进行解耦？

在设计 blockEmulator 时，我们需要尽可能地让代码变得轻量化，而且还要让代码的可重用性变得更高。为此，我们设计了图 12.3中的接口。

其中，ExtraOpInConsensus 接口的设计目的是将区块链操作从共识协议模块中解耦，从而使代码变得易于修改和复用。下面是我们设计 ExtraOpInConsensus 接口的思路。

在大多数区块链系统中，mining，verification，confirmation 三个阶段是很重要的。在许多共识协议被设计的时候，研发者通常会在这三个关键的阶段做创新并添加他们独特的操作步骤。例如，在 mining 阶段，可能会生成一个特殊的区块，例如，负责记录 CLPA 算法中负责辅助状态迁移的专用区块。在 verification 阶段，不同的共识协议以不同的方式验证新区块，例如，在 PoW 协议中，区块的哈希值需要根据 mining difficulty 难度参数进行判断是否成功获得记账权。最后，在 confirmation 阶段，某个新区块的上链可能会触发一些历史交易的回滚。

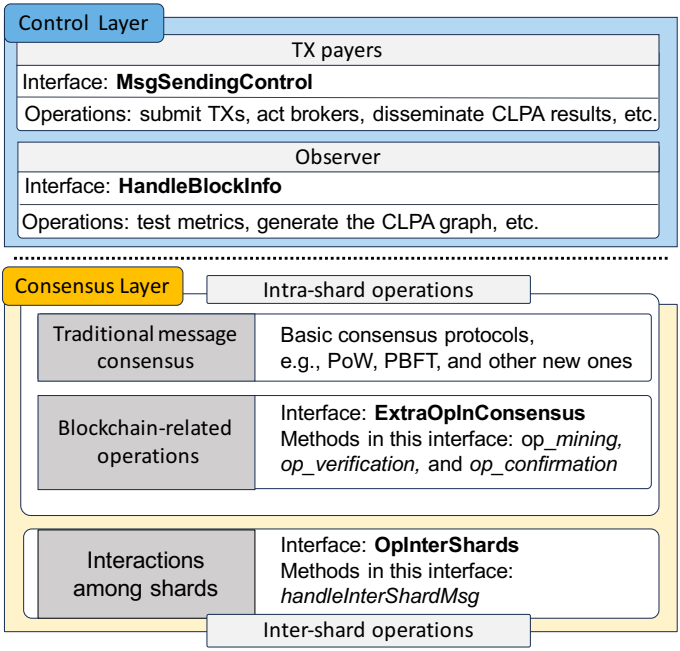


图 12.3: BlockEmulator 的接口设计。

因此，在 BlockEmulator 中，我们将 `op_mining`、`op_verification` 和 `op_confirmation` 这些基础操作从共识协议模块中解耦。通过这种解耦，用户可以在他们自定义的共识协议中设计定制化的操作，而无需担心干扰到消息传播等区块链系统运行层面的事情。此外，如果用户打算实现一个新的共识协议，这三项基本操作（即 `op_mining`、`op_verification` 和 `op_confirmation`）可以方便地被重用、重写。这是因为我们将共识协议设计为 blockEmulator 中的一个可解耦模块。

所以，用户可以根据自己的需求更改上述的接口和方法，无需对其他部分大动干戈，从而进行便捷的二次开发。

12.3.2 关于 BlockEmulator 使用场景的问题

a. BlockEmulator 如何进行“虚实结合”的多机部署？

BlockEmulator 面向的典型测试场景是分片区块链，而分片区块链往往有着大量的节点。将所有节点都部署在一台物理机器上是不切实际的，这对物理机器的性能有很大的要求，所以我们将 BlockEmulator 设计为支持多机部署的模式。在 BlockEmulator 中，多台机器通过 IP 地址建立网络通信，甚至是跨洲际通过广域网（WAN）互相连接为一张大范围实验网络。为了支持这一功能，我们在 2024/09/01 的更新中，提供了便捷配置 IP 的方式。

现在，研究者们可以在 `./ipTable.json` 中配置各个共识节点的 IP 地址。配置完 IP 地址之后，研究者可以使用 `-g` 命令生成批处理文件，此时 BlockEmulator 便按照 IP 地址将不同共识节点归类到若干个批处理文件中。

为了便于用户理解，我们提供了下面的真实网络配置案例。

在图 12.4 的实验中，我们租用了 100 台腾讯云的虚拟机。每个虚拟机配备 4 核的 2.5GHz Intel Xeon CPU，并且每个虚拟机运行一个区块链共识节点。广域网（WAN）网络带宽设定为 100 Mbps。我们在全球 10 个主要城市启动了 100 个虚拟机实例，每个城市（当作一个实验集群）分配了 10 个 BlockEmulator 实例。在每个集群内部，共识节点之间通过集群内部的网络相互连接；集群之间通过真实世界的 WAN 相互连接。

介绍 100 个 BlockEmulator 的配置太过费时，为了简化介绍流程，我们仅以图中的三个虚拟机节点作为介绍样例。其中，Hong Kong 虚拟机的 IP 地址为 `10.1.1.10`，Singapore 虚拟机的 IP 地址为 `10.2.2.20`，Silicon Valley 虚拟机的 IP 地址为 `10.3.3.30`。每个虚拟机会启动两个

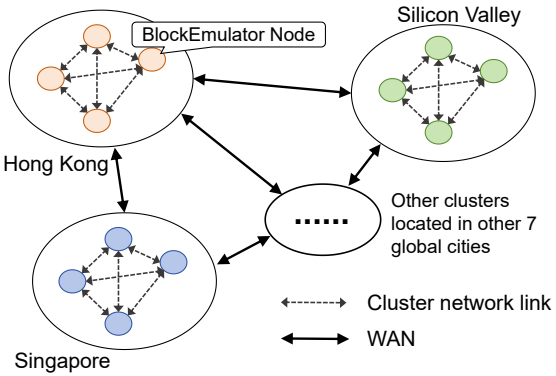


图 12.4: 多机部署 BlockEmulator 的例子。其中 Hong Kong 虚拟机的 IP 地址为 10.1.1.10, Singapore 虚拟机的 IP 地址为 10.2.2.20, Silicon Valley 虚拟机的 IP 地址为 10.3.3.30。

BlockEmulator 共识节点。

在这个简化的例子中,用户可以按照如下步骤来便捷地多机部署 BlockEmulator。

1. 用户按照各个虚拟机节点的 IP 地址配置 `./ipTable.json`。根据上述各个节点的 IP 地址，我们可以这样配置 `./ipTable.json`:

```
{
    "0": {
        "0": "10.1.1.10:32217",
        "1": "10.1.1.10:32227"
    },
    "1": {
        "0": "10.2.2.20:32317",
        "1": "10.2.2.20:32327"
    },
    "2": {
        "0": "10.3.3.30:32417",
        "1": "10.3.3.30:32427"
    },
    "2147483647": {
        "0": "10.3.3.30:38800"
    }
}
```

2. 用户运行 `blockEmulator_Windows_Precompile.exe -g -f -N 2 -S 3`。然后 BlockEmulator 就可以生成 3 个批处理文件，分别可以用于 3 个不同 IP 地址的虚拟机。

   - `windows_exe_run_IpAddr=10_1_1_10.bat`。该批处理文件可以用于启动 IP 地址为 `10.1.1.10` 的共识节点，即上述例子中的 Hong Kong 虚拟机对应的两个共识节点（S0N0 和 S0N1）。
   - `windows_exe_run_IpAddr=10_2_2_20.bat`。该批处理文件可以用于启动 IP 地址为 `10.2.2.20` 的共识节点，即上述例子中的 Singapore 虚拟机对应的两个共识节点（S1N0 和 S1N1）。
   - `windows_exe_run_IpAddr=10_3_3_30.bat`。该批处理文件可以用于启动 IP 地址为 `10.3.3.30` 的共识节点，即上述例子中的 Silicon Valley 虚拟机对应的两个共识节点（S2N0 和 S2N1）。

3. 用户将三个文件（编译好的可执行文件、`./ipTable.json` 和 `./paramsConfig.json` ）发送给每一台虚拟机。

4. 在每一台虚拟机上运行对应的批处理命令，即可启动该虚拟机上的的所有节点。

5. 等待实验运行结束。在实验运行完成后，用户只需要在每一台机器上收集实验数据即可。由于 `./paramsConfig.json` 文件中定义了实验结果输出目录 `ExpDataRootDir`，所以用户们可以利用如 python 的脚本语言，方便地获取虚拟机上的实验数据。

b. 如何确保某一次 BlockEmulator 的运行结果是正确的？

在 BlockEmulator 运行完后，ExpDataRootDir/result/supervisor_measureOutput 文件夹下有一个统计文件 Tx_Details.csv，里面包含了所有交易的各种信息（包括交易提出时间，交易上链时间等）。

用户可以利用这些信息，统计分析各个交易的处理情况，判断交易是否被正确执行。比如，用户可以统计这个文件中的交易总数，判断其是否与注入的交易总数 TotalDataSize 相同；用户可以查看上链次数，判断一笔交易是否是跨分片交易，看看是否符合数据集的情况；用户还可以通过跨分片交易的两次上链时间，判断交易是否被正确按序执行。

请注意，ExpDataRootDir 和 TotalDataSize 并不是固定的，它可以在 paramsConfig.json 中定义。

c. 我该在哪下载数据集？

(更新于 2024/05/20) 为了帮助用户更快上手 blockEmulator，我们生成了 selectedTxs_300K.csv 文件，其中包含三十万条挑选后的历史交易。

我们使用的数据集是来自 **xblock 收集的以太坊历史交易**[1]。

d. 下载完数据集后，我该如何运行 blockEmulator？

具体步骤可参见 blockEmulator 用户手册 [2]。

如果想要更快捷地测试是否 blockEmulator 能正常运行，可以按照用户手册中的内容，采用批处理方式启动预编译好的可执行文件。

[1] https://xblock.pro/xblock-eth.html

[2] https://github.com/HuangLab-SYSU/block-emulator/blob/main/20240906-blockEmulator-UserManual-OpenSource.pdf

e. 为什么我运行 blockEmulator 时，节点的 PBFT 无法达成共识 / 共识停滞不前？

原因可能有如下几种：

1. 启动 blockEmulator 前，未删除上一次运行时产生的数据或日志。

在 **2024/09/01** 的更新中，用户可以在 `./paramsConfig.json` 文件中指定 `ExpDataRootDir` 作为输出文件的目录。只需要确保两次实验的 `ExpDataRootDir` 不一样，那么实验的运行结果便不会造成文件冲突。

解决方案：或者也可以删除 `ExpDataRootDir` 下的 `./log`，`./record`，`./result` 文件夹。

2. 节点中止 / 崩溃。

这种情况产生的原因可能是 *节点端口被占用，设备性能不足（一般是一台机器超过 20 个分片时）*。

端口占用：可以在 `./ipTable.json` 文件中修改被占用节点的 IP。

f. 为什么运行 blockEmulator 之后，测试结果的数据全是 0？

有可能是 Supervisor 节点没有启动 / 启动失败，这导致交易没有发送给共识节点所在的分片。

解决方案：

1. 可以在批量启动时，查看 Supervisor 节点所对应的终端是否出现了报错信息。

2. 可以单独运行 Supervisor 节点，查看并调试 Supervisor 节点。

   Supervisor 节点的启动指令为：`go run main.go -c`。

g. BlockEmulator 是否提供了特定的 P2P 协议？

BlockEmulator 并没有提供特定的 P2P 协议，这是因为我们将它设计为一个用来做区块链实验的工具，而非开发网络协议。

更具体一些讲，我们将 blockEmulator 设计为一个可扩展的实验框架，允许用户能够方便地扩展自定义的功能。虽然我们的重点并不是在 blockEmulator 中实现某些网络协议，但是，我们提供了相关的接口（APIs）与方法 (methods)，使用户能够开发自己版本的协议，包括网络协议、共识协议和其他区块链机制。

为了便于用户配置网络或实现网络协议，我们接下来简要介绍节点如何在给定的网络配置下运行。

1. 在运行 blockEmulator 之前，用户必须定义各个节点的 IP 地址和参数，它们分别在 `ipTable.json` 和 `paramsConfig.json` 两个文件中定义。

2. 在一个共识节点启动时，它将会从 `./ipTable.json` 文件中加载所有节点的 IP 地址，因此在实验期间共识节点不需要进行节点发现以发现其他节点。

在默认情况下，所有 blockEmulator 节点形成的网络采用的是**全连接拓扑**。如果用户想要自定义网络拓扑和消息转发规则，那么用户可以修改 `./networks/p2pNetwork.go` 文件来自由发挥。例如，用户可以手动定义每个节点的邻居节点，以邻接表的形式创建网络拓扑。当节点广播消息时，每个节点仅向其邻居节点发送消息，而邻居节点则帮助它们将消息转发给其他邻居节点。

h. 如何设置与控制 BlockEmulator 实验的网络带宽？

BlockEmulator 在本地单机实验时提供了模拟节点带宽可控的方案，而且十分简单便捷，只需要在 `./paramsConfig.json` 文件中修改 Bandwidth 参数的值即可。Bandwidth 参数的默认单位为 bps，如果想要修改为 Kbps、Mbps 等较大的单位，则只需要在 `./params/global_config.go` 中的 `ReadConfigFile()` 方法中将 `Bandwidth=config.Bandwidth` 修改为 `Bandwidth=config.Bandwidth*1024` 等即可。

下面介绍了这种模拟带宽控制方案的实现原理，以便用户可以更好地自由使用、自行修改以满足实验的需求。建议用户对照 BlockEmulator 的 `./networks/p2pNetwork.go` 中的 `TcpDial()` 和 `InitNetworkTools()` 方法，以及`./networks/upload_limit.go` 进一步学习。

实现带宽控制的方法主要是利用了 `golang.org/x/time/rate` 包中的限流器 (即 `rate.Limiter`) 来控制写入速度。在 BlockEmulator 的 `./networks/p2pNetwork.go` 包中定义了一个用于限制写入速率的包装器 `rateLimitedWriter`，它包装了任何实现了 `io.Writer` 接口的对象（如网络连接 `net.Conn`）。在重写的 `Write(p[]byte)` 方法中，首先计算要写入的数据长度 `n` ；然后进入循环，每次通过 `limiter.Burst()` 尝试获取当前可写入的字节数；使用 `limiter.WaitN()` 方法等待 `limiter` 满足 `writeByteNum`；然后更新 `n` 的值。最后若全部字节都可以被写入时，调用 w.writer.Write(p) 将数据真正地写入。其实也可以通过直接将数据长度 `n` 传入 `limiter.WaitN()` 方法而无需循环：`w.limiter.WaitN(context.TODO(),n)`。

需要使用模拟带宽限制的网络通信时，通过传入带宽参数构造一个 `limiter` ，然后实现连接 `conn` ，调用 `writeToConn()` 方法，传入需要发

送的数据即可。

限制带宽和不限带宽的网络通信的区别就在于前者的 `TcpDial()` 是通过 `writeToConn()` 方法进行数据传输，后者在实现了 TCP 连接后直接调用了 `conn.Write()` 方法传输数据，当然也可以在限制带宽的实现中将带宽设置为无限大。

i. 云端部署时的节点 IP 设置的问题

这个问题来自于天津大学的杨同学，他在 2025 年 6 月 4 日将该问题发送到 BlockEmulator 用户交流群："大家好，我有个问题想请教一下：我在将 BlockEmulator 部署到腾讯云上时，不同区域的节点内网 IP 无法通信，而腾讯云又使用的 EIP，使用公网 IP 会在 net.listen() 处报错了，不知道大家有没有遇到过类似的情况，应该如何解决?"

黄华威老师邀请了研究组的谢葆洲同学回复该问题：

- "您好。使用腾讯云做实验的话，前提是每台机器都有 EIP（弹性公网 IP），因此在 IP 表（iptable.json 文件）中需要配置为每个节点的 EIP 和对应的端口，本地节点使用对端节点的 EIP 是能够正常发起网络通信的（*TcpDial*）。而 *net.listen()* ）处需要设置为监听 *0.0.0.0:xxx* 端口，而不能监听本机的 EIP，因为 EIP 对于云服务器本机是不可见的，云厂商使用 SDN（软件定义网络）将 EIP 映射到云服务器，因此只需监听通配符 *0.0.0.0*，表示监听云服务器的所有网卡。"

杨同学："感谢您的回复，所以需要在 PbftConsensusNode 和 Supervisor 中的 net.listen() 处的 IP 都改为 "0.0.0.0:{自己的端口}"，就可以了是吗?"

谢葆洲："是的。另外，需要在云服务器控制台的安全组规则中放通

（放通的意思是：允许通过）对应的端口，linux 系统如果存在防火墙也需要放通对应的端口。”

杨同学：“好的，我明白了，非常感谢!”

同时，研究组博士后罗肖飞博士也指出“这个问题不是我们 BlockEmulator 系统的问题，是服务器环境配置的问题”，并给出了他的如下见解，一并给读者参考：

- “1. 内网 IP 无法通信：通常是因为腾讯云不同区域之间的内网是隔离的，内网 IP 只能在同一个区域（Region）内通信，跨区域的内网 IP 无法直接通信。”

- “2. 使用公网 IP（EIP）监听报错：当我们尝试使用公网 IP 进行监听时，可能会因为安全组、防火墙设置或者云服务器本身没有绑定 EIP 等原因导致监听失败。另外，有些云服务商要求，如果要监听公网 IP，需要在云服务器控制台上进行额外的配置（比如绑定 EIP 并配置安全组规则允许该端口）。”

希望以上问答对正在遇到类似问题的其他同学一些启发与帮助。

第十三章　未来规划

13.1　BlockEmulator 的后续版本

未来，黄华威研究组计划持续迭代 BlockEmulator，将会陆续嵌入更多研究组的最新研究成果相关的代码，并且会支持更加完善的智能合约执行环境、可视化实验环境等特性。敬请期待。

13.2　BlockEmulator 社区

希望 BlockEmulator 能成为分片区块链社区的标准实验工具。新版本的 BlockEmulator-X 支持任何人提交新代码，审核通过后会被合并到 main 分支，或者被当做一个新的代码分支。期待大家用它来创新与收获。

写在结尾

自从 2023 年 5 月开源之后，HuangLab 与众多社区用户一起见证了 BlockEmulator 的迭代更新，每一次都受益于用户的反馈：他们要么提出一个疑问，要么反馈一个 bug。无论哪种形式，每次反馈无疑都会催促 HuangLab 成员立即展开分析与讨论，最后我们总能解决每一个技术问题。

2024 年下半年，有一天我突然非常好奇：到底有没有用户在使用我们开源的这个工具来做区块链实验？于是我就使用关键词 BlockEmulator 搜索了一下，结果看到两个有意思的发现。

图 13.1: CSDN 上的技术博客。

发现 1：如图13.1示，有网友开始写 BlockEmulator 的技术博客，分享了自己遇到的常见技术问题以及解决方案[1]。

[1] `https://blog.csdn.net/gitblog_00387/article/details/144455816`，《BlockEmulator 常见问题解决方案》，作者：巫舒姗

图 13.2: 知乎文章推荐的几个代表性区块链实验工具，
图片来源于：`https://zhuanlan.zhihu.com/p/671851160`。

发现 2：知乎上有文章[1] 把 BlockEmulator 与 SimBlock、BLOCKBENCH、Ganache、Tendermint、Hyperledger Caliper 等知名工具一起并列推荐为代表性区块链实验和仿真工具，如图 13.2 所示。

感谢对 BlockEmulator 的认可！

但愿它可以帮助更多人。

黄华威

2024 年 12 月 31 日

[1] `https://zhuanlan.zhihu.com/p/671851160`，《区块链实验和仿真工具》，作者：码的艺术家

参考文献

[1] Qinde Chen, Huawei Huang, Zhaokang Yin, Guang Ye, and Qinglin Yang. Broker2Earn: Towards Maximizing Broker Revenue and System Liquidity for Sharded Blockchains. In *Proc. of IEEE Conference on Computer Communications (INFOCOM)*, pages 251–260. IEEE, 2024.

[2] Qinde Chen, Huawei Huang, and Jian Zheng. LiquidityPool: Game-Theoretic Analysis of Stakeholder Revenue in Ranking-Dependent DeFi. In *Proc. of The ACM Web Conference 2026 (WWW'26)*, pages 111–122, Dubai, UAE, April 2026. ACM.

[3] Qiuyu Ding, Rongkai Zhang, Shenglin Yin, PengZe Li, Shengjie Guan, Zhen Xiao, and Jieyi Long. Presto: Optimizing cross-shard transactions in sharded blockchain architecture. In *Proc. of 43rd International Symposium on Reliable Distributed Systems (SRDS)*, pages 139–149. IEEE, 2024.

[4] Huawei Huang, Zhenyi Huang, Xiaowen Peng, Zibin Zheng, and Song Guo. Mvcom: Scheduling most valuable committees for the large-scale sharded blockchain. In *Proc. of 41st International Conference on Distributed Computing Systems (ICDCS)*, pages 629–639. IEEE, 2021.

[5] Huawei Huang, Yue Lin, and Zibin Zheng. Account Migration across Blockchain Shards using Fine-tuned Lock Mechanism. In *Proc. of IEEE Conference on Computer Communications (INFOCOM)*, pages 271–280. IEEE, 2024.

[6] Huawei Huang, Xiaowen Peng, Yue Lin, Miaoyong Xu, Guang Ye, Zibin Zheng, and Song Guo. Scheduling most valuable committees for the sharded blockchain. *IEEE/ACM Transactions on Networking (ToN/TNet)*, 31(6):3284–3299, 2023.

[7] Huawei Huang, Xiaowen Peng, Jianzhou Zhan, Shenyang Zhang, Yue Lin, Zibin Zheng, and Song Guo. BrokerChain: A Cross-Shard Blockchain Protocol for Account/Balance-based State Sharding. In *Proc. of IEEE Conference on Computer Communications (INFOCOM)*, pages 1968–1977. IEEE, 2022.

[8] Huawei Huang, Guang Ye, Qinglin Yang, Qinde Chen, Zhaokang Yin, Xiaofei Luo, Jianru Lin, Jian Zheng, Taotao Li, and Zibin Zheng. BlockEmulator: An Emulator Enabling to Test Blockchain Sharding Protocols. *IEEE Transactions on Services Computing (TSC)*, 18(2):690–703, 2025.

[9] Huawei Huang, Zhaokang Yin, Qinde Chen, Guang Ye, Xiaowen Peng, Yue Lin, Zibin Zheng, and Song Guo. BrokerChain: A Blockchain Sharding Protocol by Exploiting Broker Accounts. *IEEE/ACM Transactions On Networking (ToN)*, 33(4):1930–1945, 2025.

[10] Huawei Huang, Yetong Zhao, and Zibin Zheng. Tmpt: Reconfiguration across blockchain shards via trimmed merkle patricia trie. In *Proc. of 2023 IEEE/ACM 31st International Symposium on Quality of Service (IWQoS)*, pages 1–10. IEEE, 2023.

[11] Huawei Huang, Jian Zheng, Xuanye Zhu, Yue Lin, Ting Cai, Lu Zhou, Zibin Zheng, and Song Guo. ShardCutter: A Blockchain Sharding Protocol Achieving Transaction Workload Balance Across State Shards. *IEEE Transactions on Netwowrking (ToN)*, 34:3393–3407, 2026.

[12] Eleftherios Kokoris-Kogias, Philipp Jovanovic, Linus Gasser, Nicolas Gailly, Ewa Syta, and Bryan Ford. Omniledger: A secure, scale-out, decentralized ledger via sharding. In *Proc. of IEEE symposium on security and privacy (S&P)*, pages 583–598. IEEE, 2018.

[13] Canlin Li, Huawei Huang, Yetong Zhao, Xiaowen Peng, Ruijie Yang, Zibin Zheng, and Song Guo. Achieving Scalability and Load Balance across Blockchain Shards for State Sharding. In *Proc. of 2022 41st International Symposium on Reliable Distributed Systems (SRDS)*, pages 284–294. IEEE, 2022.

[14] Pengze Li, Mingxuan Song, Mingzhe Xing, Zhen Xiao, Qiuyu Ding, Shengjie Guan, and Jieyi Long. Spring: Improving the throughput of sharding blockchain via deep reinforcement learning based state placement. In *Proc. of the 2024 ACM on Web Conference (WWW)*, pages 2836–2846, 2024.

[15] Loi Luu, Viswesh Narayanan, Chaodong Zheng, Kunal Baweja, Seth Gilbert, and Prateek Saxena. A secure sharding protocol for open blockchains. In *Proc. of the 2016 ACM SIGSAC conference on computer and communications security*, pages 17–30, 2016.

[16] Jiaping Wang and Hao Wang. Monoxide: Scale out Blockchains with Asynchronous Consensus Zones. In *Proc. of 20th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, volume 2019, pages 95–112, 2019.

[17] Qian Wang and Yepeng Guan. Transshard: A dynamic transaction-aware sharding scheme for account-based blockchain. *IEEE Access*, 2024.

[18] Junting Wu, Haotian Du, Jin Chen, and Wei Ren. Solving confirmation time in sharded blockchain with pfqn. *Electronics*, 13(7):1220, 2024.

[19] Huang Xinpeng, Jie Wanqing, Zhang Shiwen, Yang Haofu, Qiu Wangjie, Zhang Qinnan, Huang Huawei, Xiong Zehui, Tang Shaoting, Zheng Hongwei, and Zheng Zhiming. ContribChain: A Stress-Balanced Blockchain Sharding Protocol with Node Contribution Awareness. In *Proc. of IEEE Conference on Computer Communications (INFOCOM)*. IEEE, 2025.

[20] Mahdi Zamani, Mahnush Movahedi, and Mariana Raykova. Rapidchain: Scaling blockchain via full sharding. In *Proc. of the 2018*

ACM SIGSAC conference on computer and communications security (CCS), pages 931–948, 2018.

[21] Yuanzhe Zhang, Shirui Pan, and Jiangshan Yu. Txallo: Dynamic transaction allocation in sharded blockchain systems. In *Proc. of IEEE 39th International Conference on Data Engineering (ICDE)*, pages 721–733. IEEE, 2023.

[22] Jian Zheng, Huawei Huang, Yinqiu Liu, Taotao Li, Hong-Ning Dai, and Zibin Zheng. Justitia: An Incentive Mechanism towards the Fairness of Cross-shard Transactions. In *Proc. of IEEE Conference on Computer Communications (INFOCOM)*, pages 1–10. IEEE, 2025.